
Test Document

for

FindIT

Version 1.0

Prepared by Team WebVortex

Group 2:

Rudra Sinha
Siddharth Miglani
Dharajiya Yug Harshadbhai
Pratul Koolwal
Rohan Gauranga Potukuchi
Nischay Agarwal
Vihaan Sapra
Suyash Kapoor
Aarush Narendra Ghatе
Shravan Agrawal

230880
231006
230362
230782
230864
230705
231149
231066
230017
230984

Group Name: WebVortex

rudrasinha23@iitk.ac.in
siddharthm23@iitk.ac.in
dharajiya23@iitk.ac.in
pratulk23@iitk.ac.in
rohangp23@iitk.ac.in
nischaya23@iitk.ac.in
svihaan23@iitk.ac.in
suyashk23@iitk.ac.in
aarushng23@iitk.ac.in
ashravan23@iitk.ac.in

Course: CS253

Mentor TA: Hemang Mohanlal Khatri

Date: 4 April 2025

CONTENTS.....	2
REVISIONS.....	2
1 INTRODUCTION.....	3
2 UNIT TESTING.....	5
3 INTEGRATION TESTING.....	40
4 SYSTEM TESTING.....	80
5 CONCLUSION.....	89
APPENDIX A - GROUP LOG.....	92

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Initial Draft (v1.0)	Rudra Sinha Siddharth Miglani Dharajiya Yug Harshadbhai Pratul Koolwal Rohan Gauranga Potukuchi Nischay Agarwal Vihaan Sapra Suyash Kapoor Aarush Narendra Ghatе Shravan Agarwal	Initiated the document. Added all the necessary details.	04/04/25

1 Introduction

Introduction

The **Lost and Found Campus Website** was designed to facilitate the process of reporting, tracking, and retrieving lost and found items within our campus. Given the personal and urgent nature of lost belongings, the system needed to be **reliable, efficient, and user-friendly**. To ensure a **smooth user experience**, thorough testing was conducted throughout the development process.

This document details the testing strategy, execution methodology, coverage criteria, and the overall effectiveness of the testing process. It provides insights into how we validated the system's **core functionalities, reliability, and robustness** before deployment.

Test Strategy

Our testing strategy was primarily **manual testing**, supported by ThunderClient for API validation. Since the system had **a limited number of routes and failure points**, manual testing was deemed sufficient to verify most functionalities. We specifically focused on:

1. **Functional Testing** – Ensuring that all key features (such as user authentication, reporting lost/found items, and searching for items) performed as expected.
2. **User Interface Testing** – Checking for **layout consistency**, form validation, and **intuitive navigation**.
3. **Error Handling & Edge Cases** – Verifying how the system responds to invalid inputs, missing data, and unexpected user behavior.
4. **Security & Authentication Testing** – Testing login, sign-up, and password recovery mechanisms to ensure **proper access control**.

Testing Timeline

Testing was conducted **in parallel with implementation**, allowing us to identify and resolve issues early. Each feature was tested as soon as it was integrated into the system, ensuring that bugs were fixed before moving on to additional functionalities.

Testers

Since this was **alpha testing**, the **developers themselves performed the testing**, but each developer tested components written by **other team members** to ensure unbiased validation.

Coverage Criteria

We followed a **branch coverage criterion**, meaning we ensured that all possible execution paths were tested at least once. This helped us identify and correct **any logical errors in the control flow**.

Testing Tools Used

- **ThunderClient** – Used for testing API endpoints and verifying request/response handling.
- **Manual Testing** – Performed to validate UI functionality, user flows, and system behavior.

This structured approach helped us **identify and resolve issues efficiently**, ensuring the system was well-prepared for its initial deployment.

2 Unit Testing

For conducting Unit Testing, we have used **Thunder Client**. We have included multiple test cases under each Unit to test all the different functionalities.

1. Sign Up:

This unit pertains to the action of registering a new user into the system

Unit Details: [Auth Class, signup() and verifyOTP()]

a) New email:

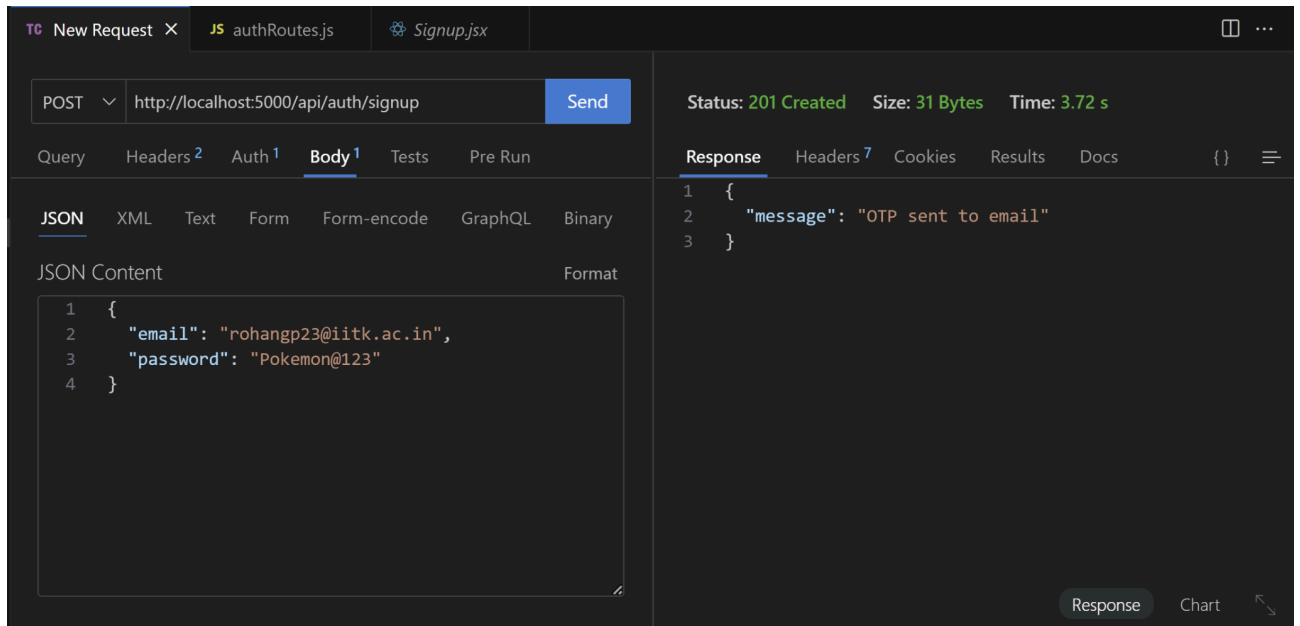
When a user tries to register with an email that isn't already in the database

Test Owner: Rohan Potukuchi

Test Date: 30/3/25

Test Results:

Upon entering the email and password, an OTP is sent to the email ID.



i) Incorrect OTP is entered: Sends an error message

The screenshot shows a REST client interface with a POST request to `http://localhost:5000/api/auth/verify-otp`. The request body is a JSON object: `{ "email": "rohangp23@iitk.ac.in", "otp": "171780" }`. The response status is `400 Bad Request`, with a size of `25 Bytes` and a time of `115 ms`. The response body is a JSON object: `{ "message": "Invalid OTP" }`.

```
POST http://localhost:5000/api/auth/verify-otp
```

```
{
  "email": "rohangp23@iitk.ac.in",
  "otp": "171780"
}
```

Status: 400 Bad Request Size: 25 Bytes Time: 115 ms

```
{
  "message": "Invalid OTP"
}
```

ii) Correct OTP is entered: Successfully signs up

The screenshot shows the same REST client interface with a POST request to `http://localhost:5000/api/auth/verify-otp`. The request body is a JSON object: `{ "email": "rohangp23@iitk.ac.in", "otp": "171781" }`. The response status is `200 OK`, with a size of `31 Bytes` and a time of `132 ms`. The response body is a JSON object: `{ "message": "Signup successful" }`.

```
POST http://localhost:5000/api/auth/verify-otp
```

```
{
  "email": "rohangp23@iitk.ac.in",
  "otp": "171781"
}
```

Status: 200 OK Size: 31 Bytes Time: 132 ms

```
{
  "message": "Signup successful"
}
```

b) Existing Email:

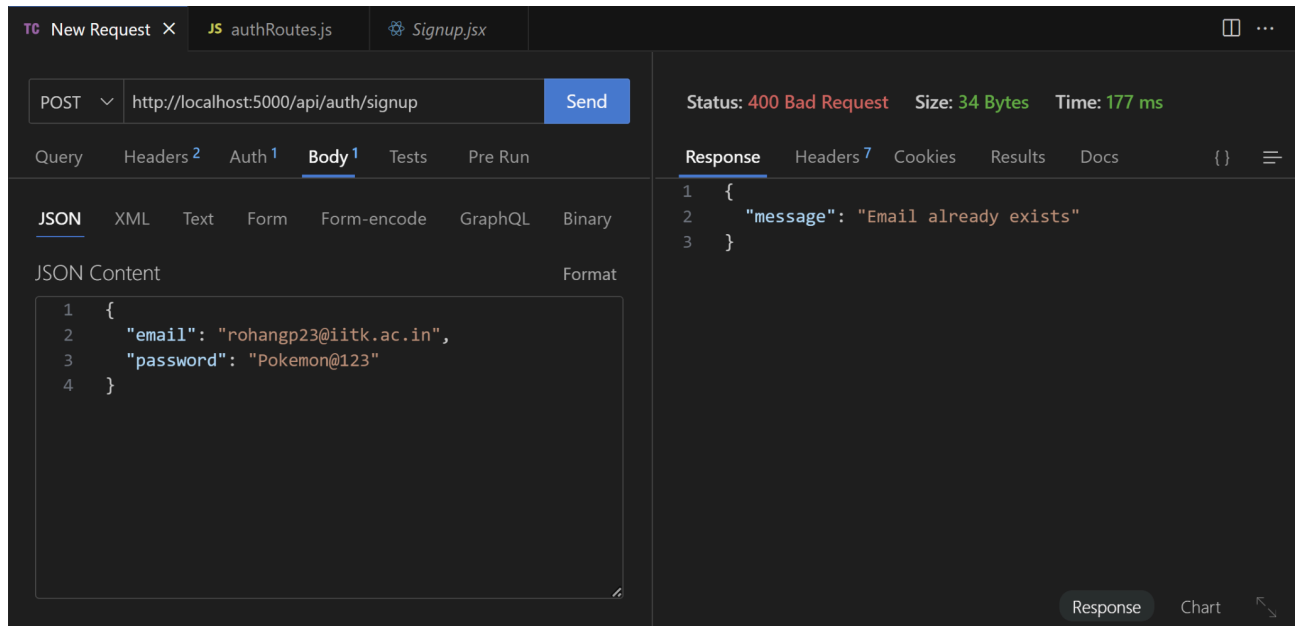
When a user tries to register with an email that is already in the database

Test Owner: Rohan Potukuchi

Test Date: 30/3/25

Test Results:

Sends error message



c) Weak Password:

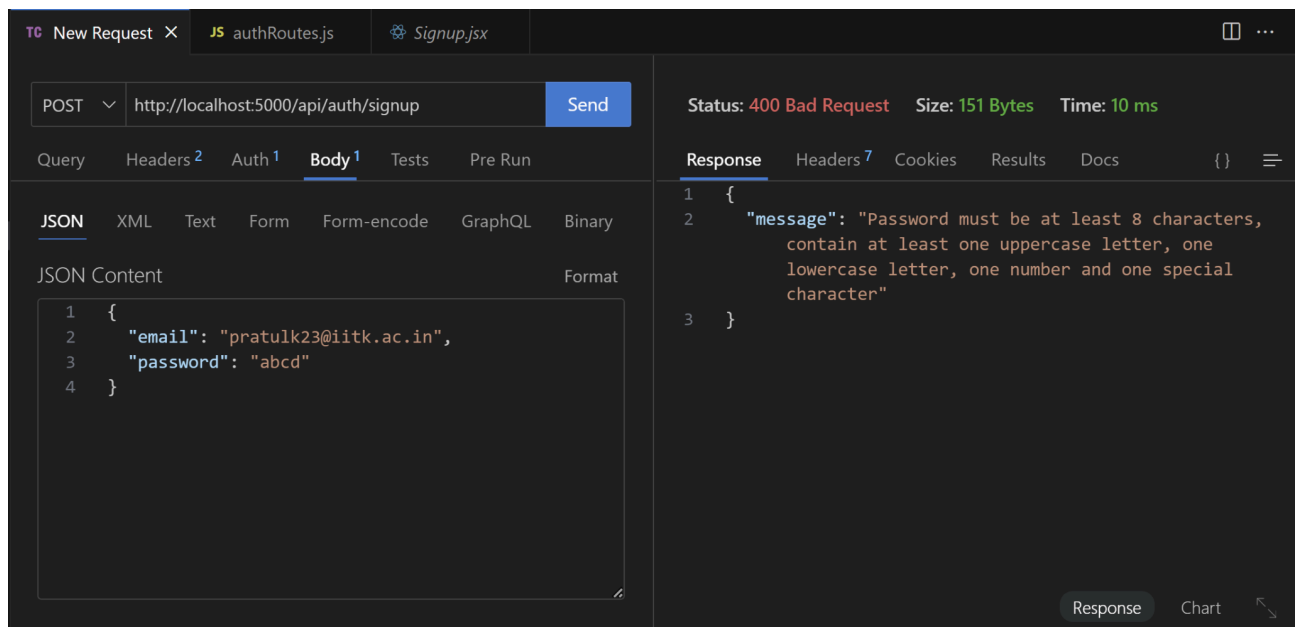
When a user tries to register with a password that is weak

Test Owner: Rohan Potukuchi

Test Date: 30/3/25

Test Results:

Sends error message



Structural Coverage: Branch Coverage is achieved for both functions

2. Login:

This unit pertains to an existing user logging in to the website

Unit Details: [Auth Class, login()]

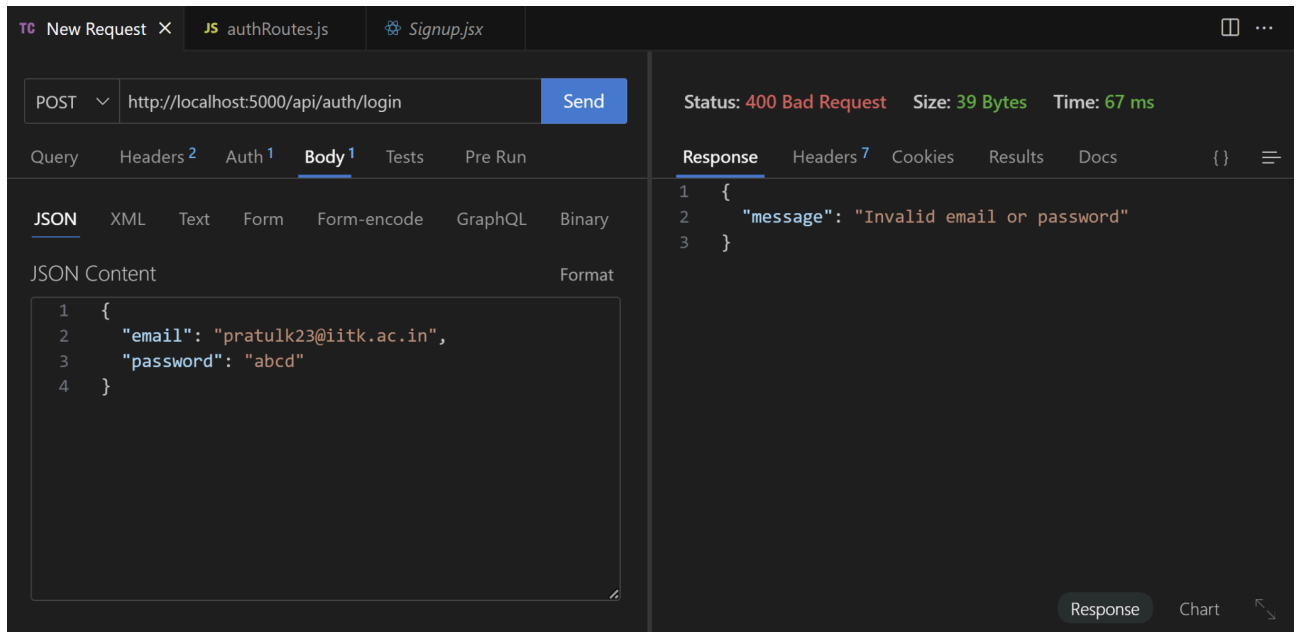
a) User doesn't exist or incorrect password:

Test Owner: Siddharth Miglani

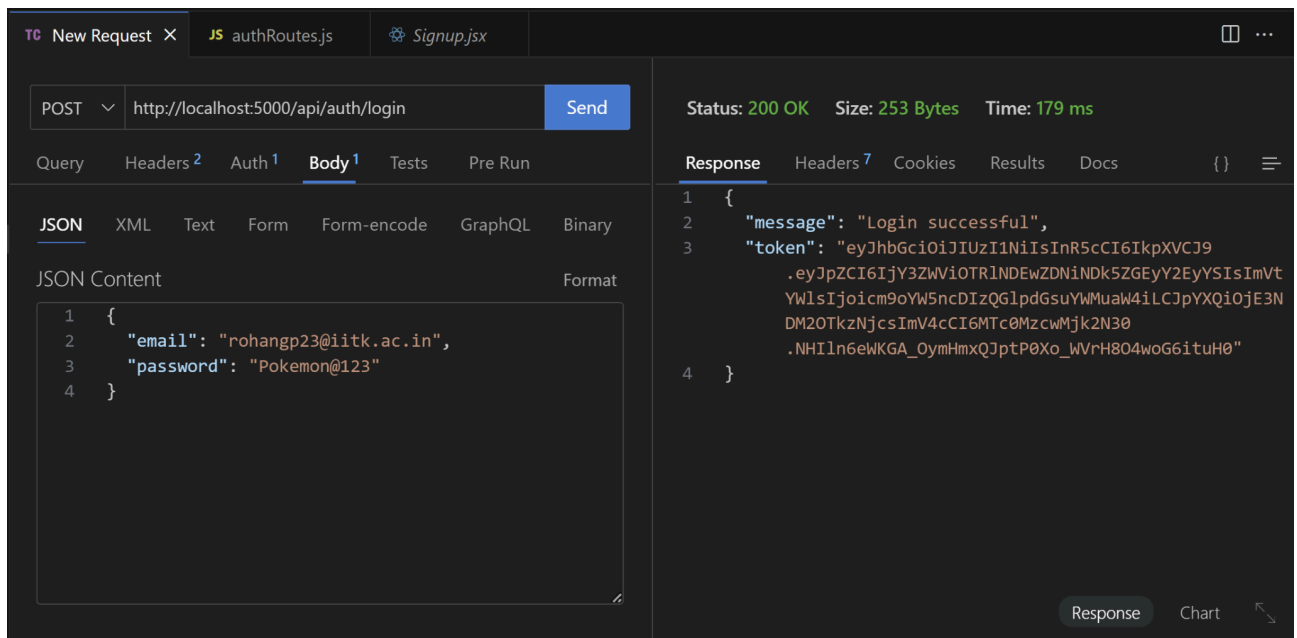
Test Date: 30/3/25

Test Results:

Throws an error

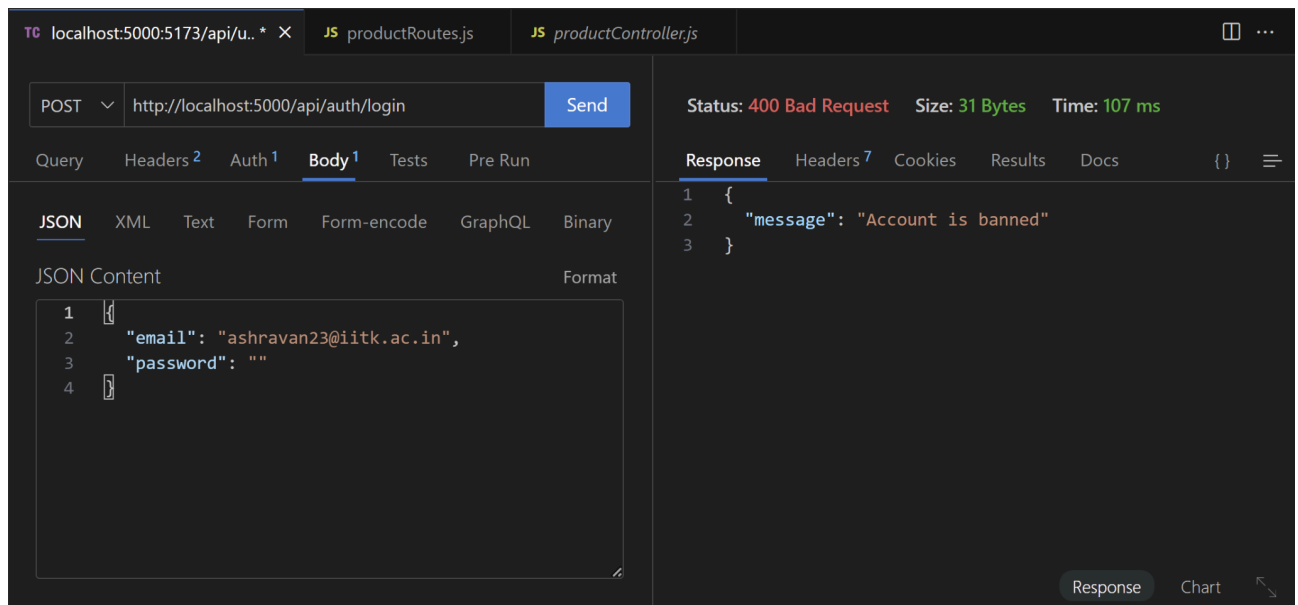
b) User exists:**Test Owner:** Siddharth Miglani**Test Date:** 30/3/25**Test Results:**

Successfully logs in and produces a token



c) User is Banned:**Test Owner:** Rudra Sinha**Test Date:** 4/4/25**Test Results:**

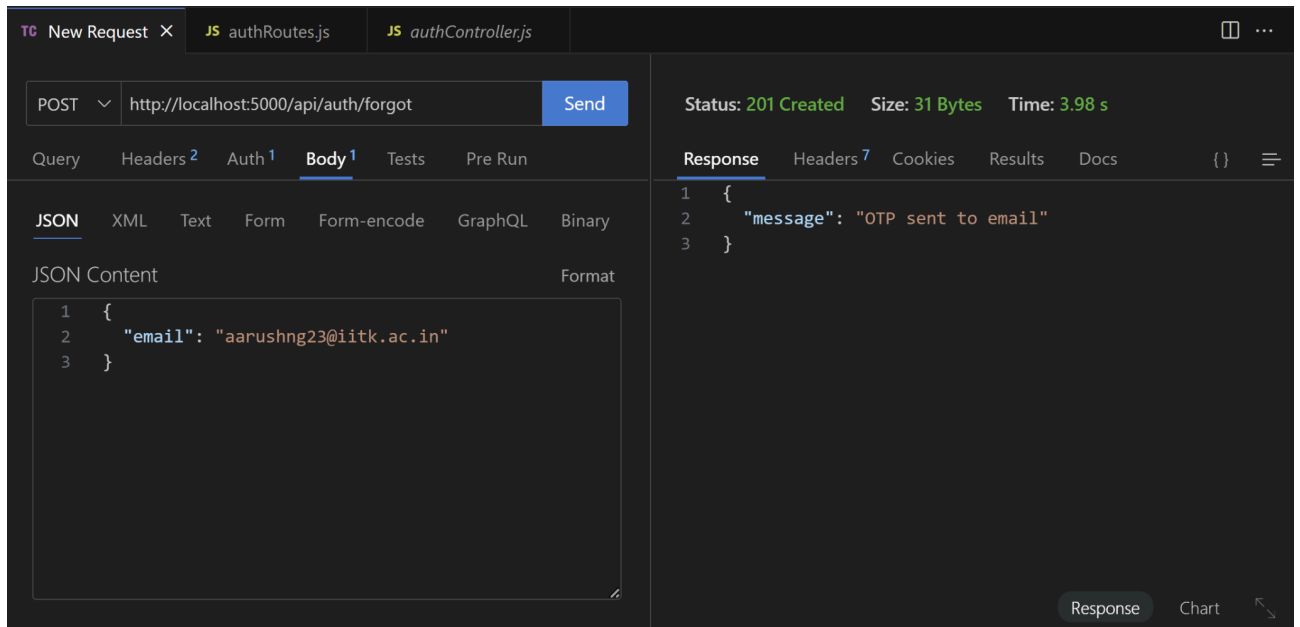
Error is thrown

**Structural Coverage:** Branch coverage is achieved for login()**3. Forgot/Reset Password:**

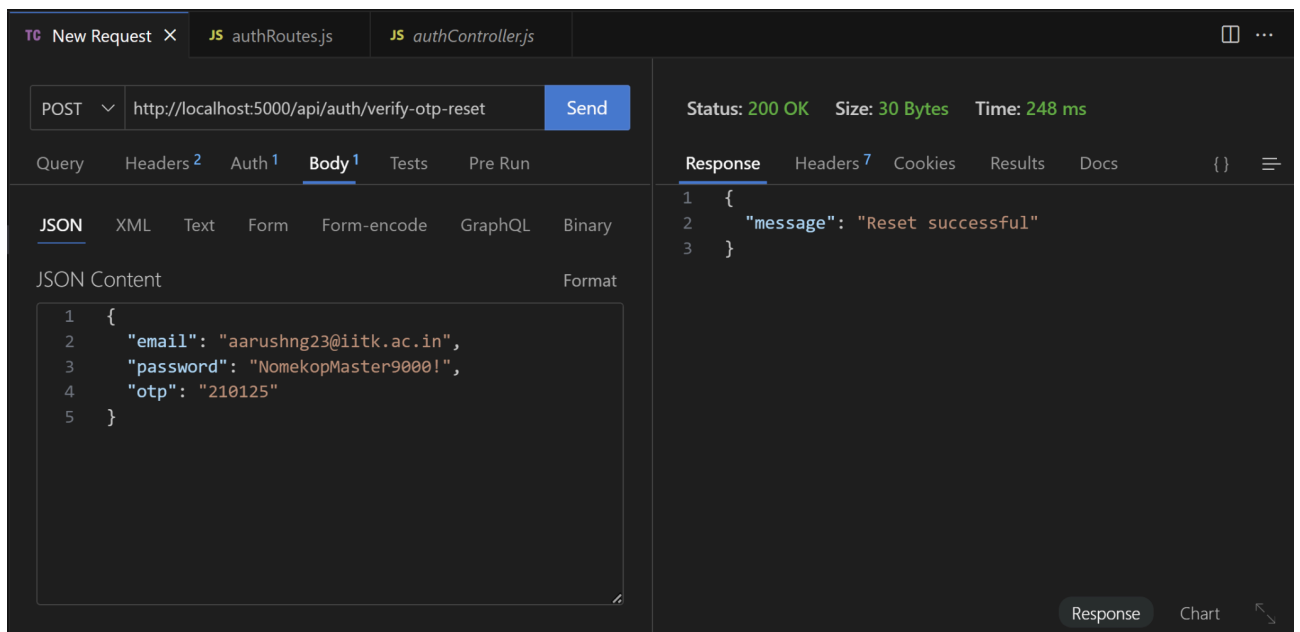
This module is to reset a user's password when they forget it.

Unit Details: [Auth Class, forgot() and verifyOTPreset()]**Test Owner:** Rudra Sinha**Test Date:** 1/4/25**Test Results:**

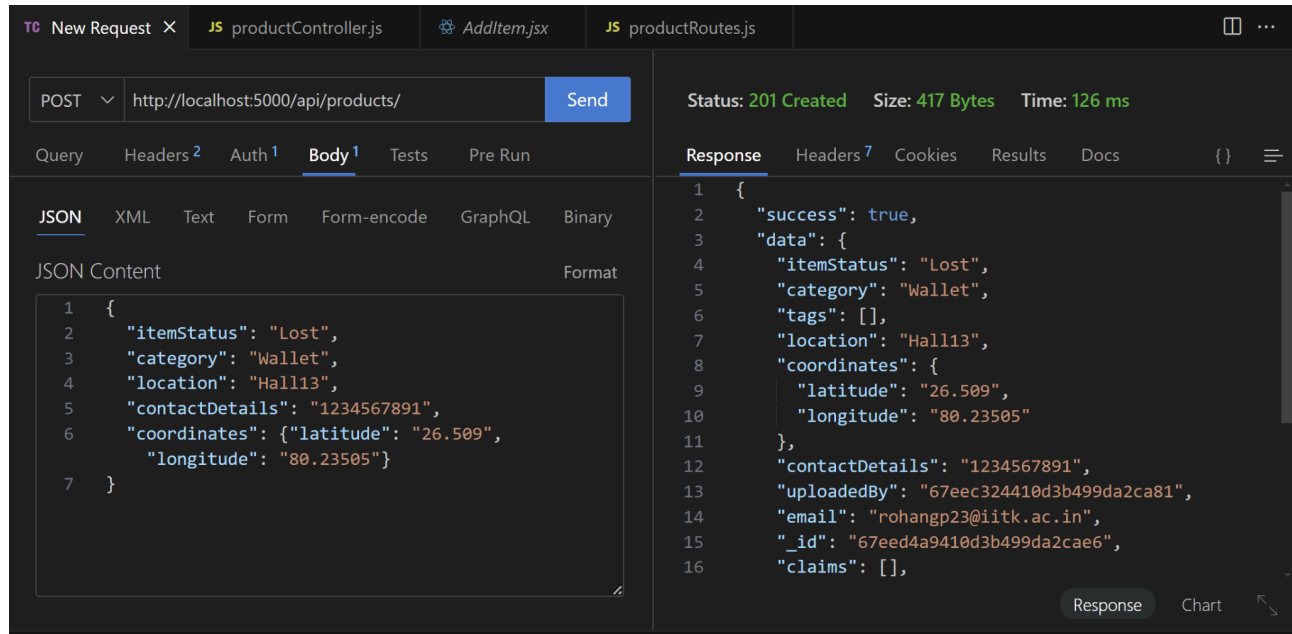
A mail with the OTP is sent successfully



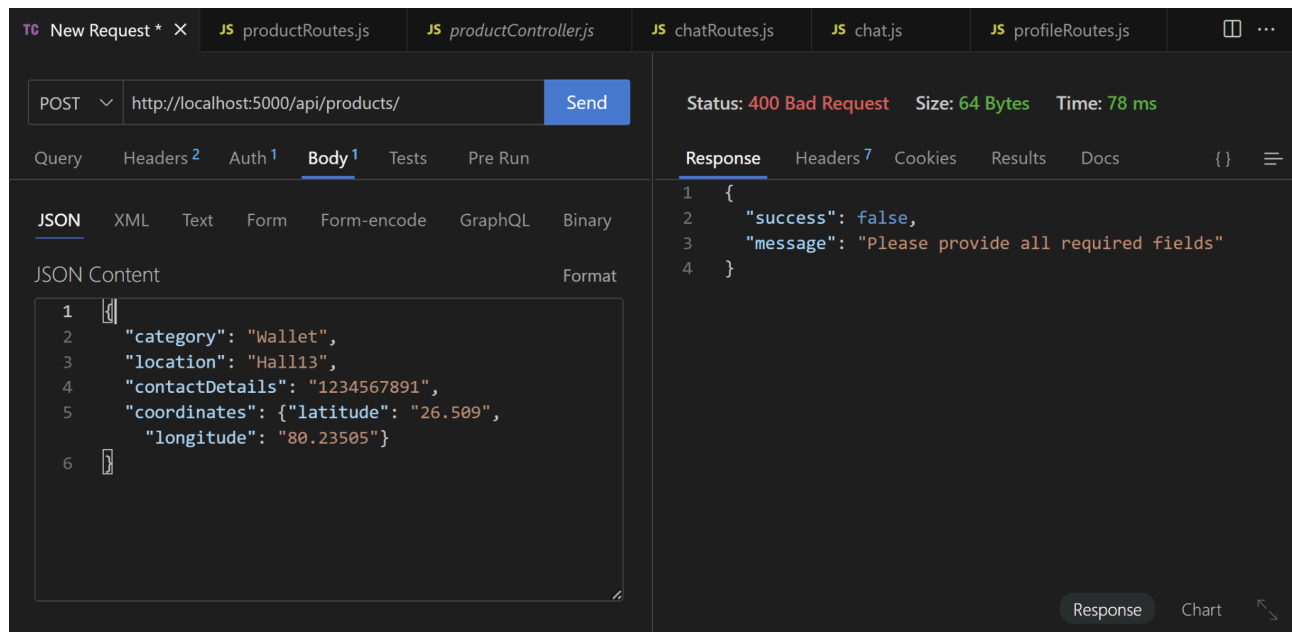
Upon entering correct OTP, password is successfully reset:



Structural Coverage: Complete branch coverage is achieved for the functions



If details are filled partially then error is thrown



Structural Coverage: Functional coverage is achieved for `createProduct()`

6. Delete Product:

Product should get deleted from database

Unit Details: [Product Class, deleteProduct()]

Test Owner: Dharajiya Yug

Test Date: 2/4/25

Test Results:

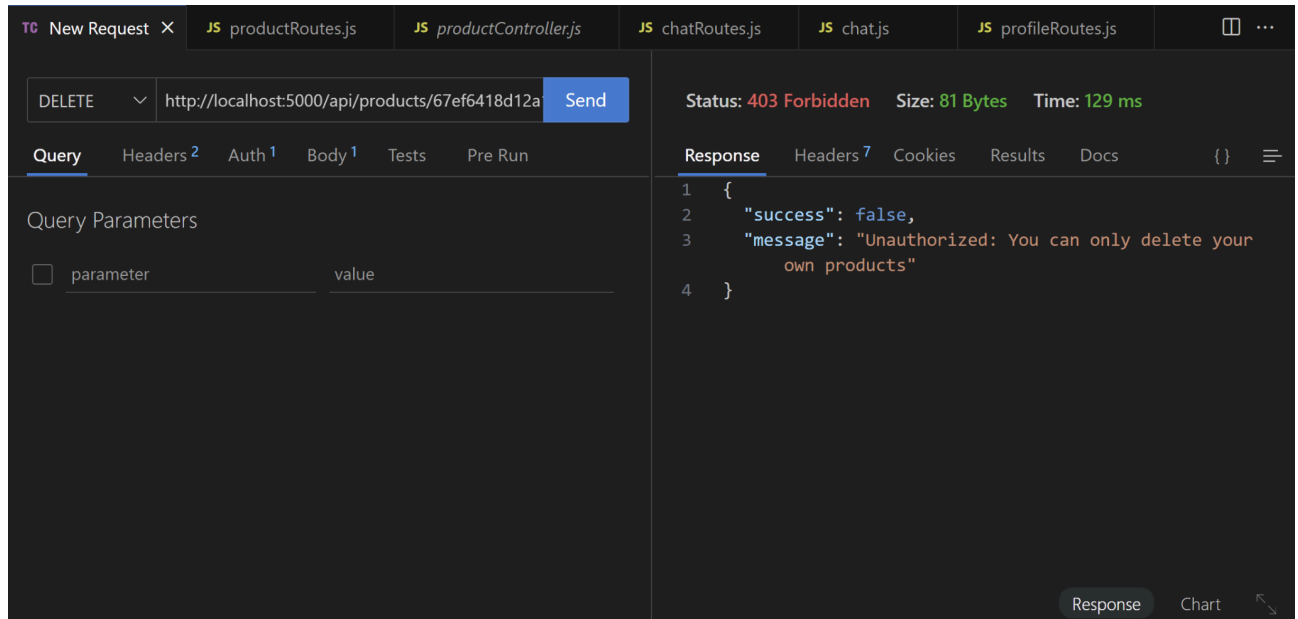
Upon sending a DELETE request with the corresponding product ID, the product is deleted successfully

The screenshot displays a REST client interface with the following details:

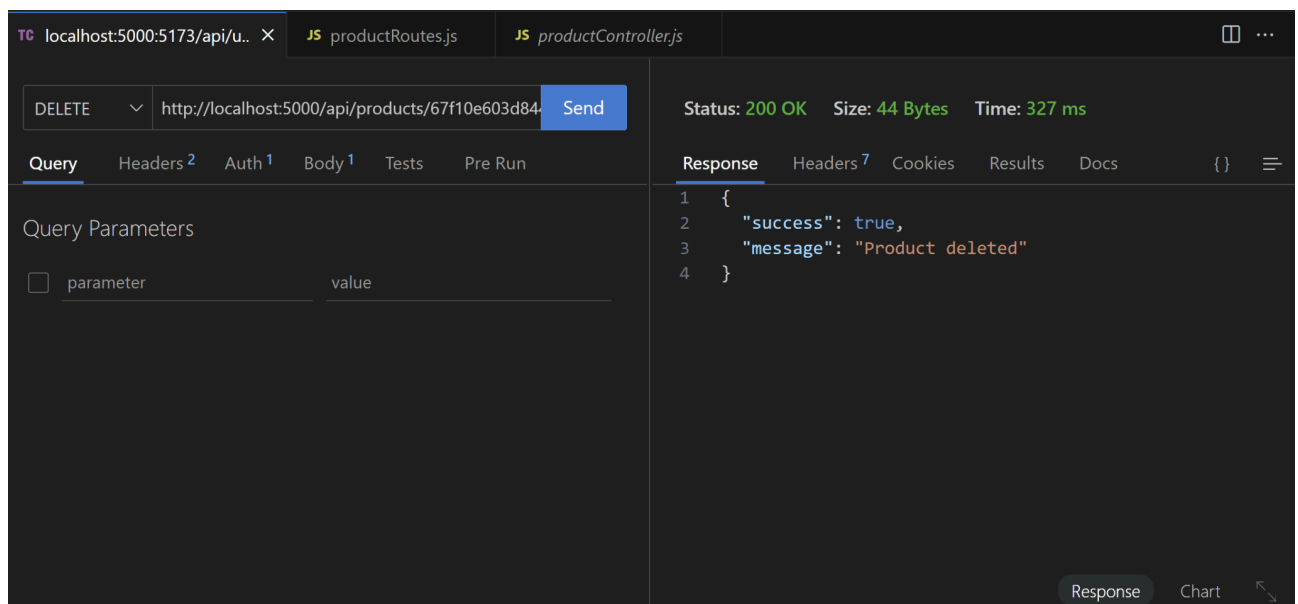
- Request:**
 - Method: DELETE
 - URL: `http://localhost:5000/api/products/67eed4a9410d`
 - Status: 200 OK
 - Size: 44 Bytes
 - Time: 187 ms
- Response:**

```
1 {
2   "success": true,
3   "message": "Product deleted"
4 }
```
- Query Parameters:** A table with columns 'parameter' and 'value' is visible but empty.

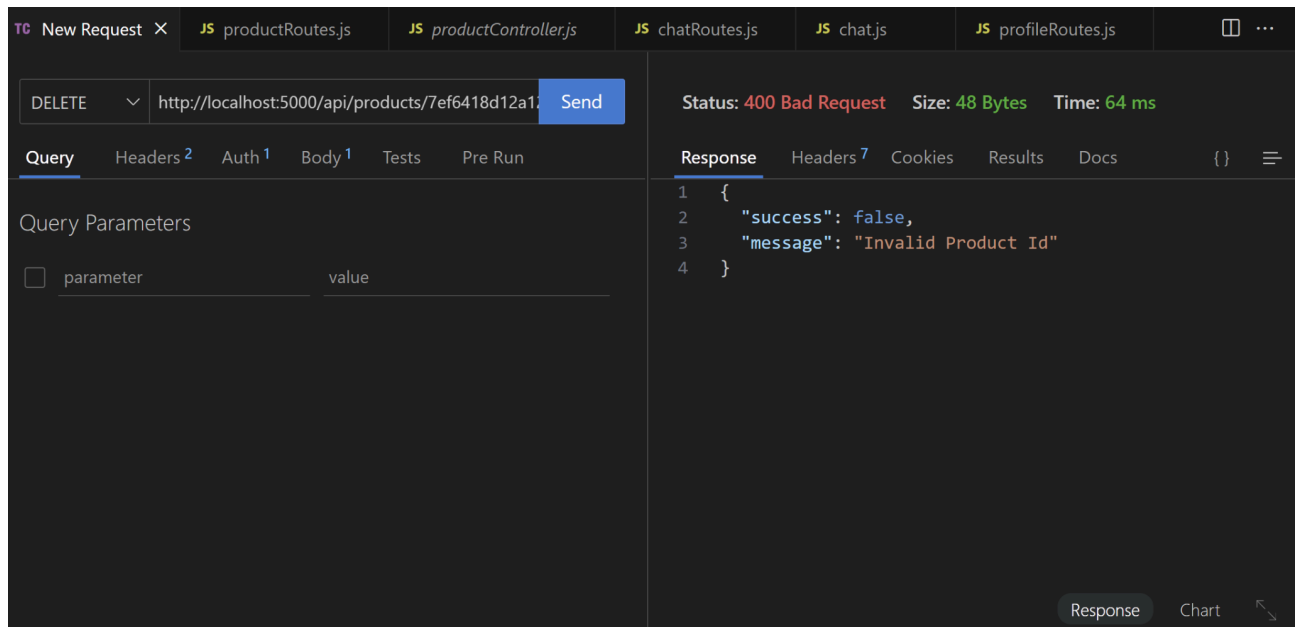
Upon trying to delete another user's product, an error is thrown (if logged in from a regular user's account)



Product of another user is deleted successfully when logged in from an admin account



If invalid product ID is given, error is thrown



Structural Coverage: Full branch coverage is achieved for deleteProduct()

7. Get Product Details:

Gets product details when product ID is given as input

Unit Details: [Product Class, getProductById()]

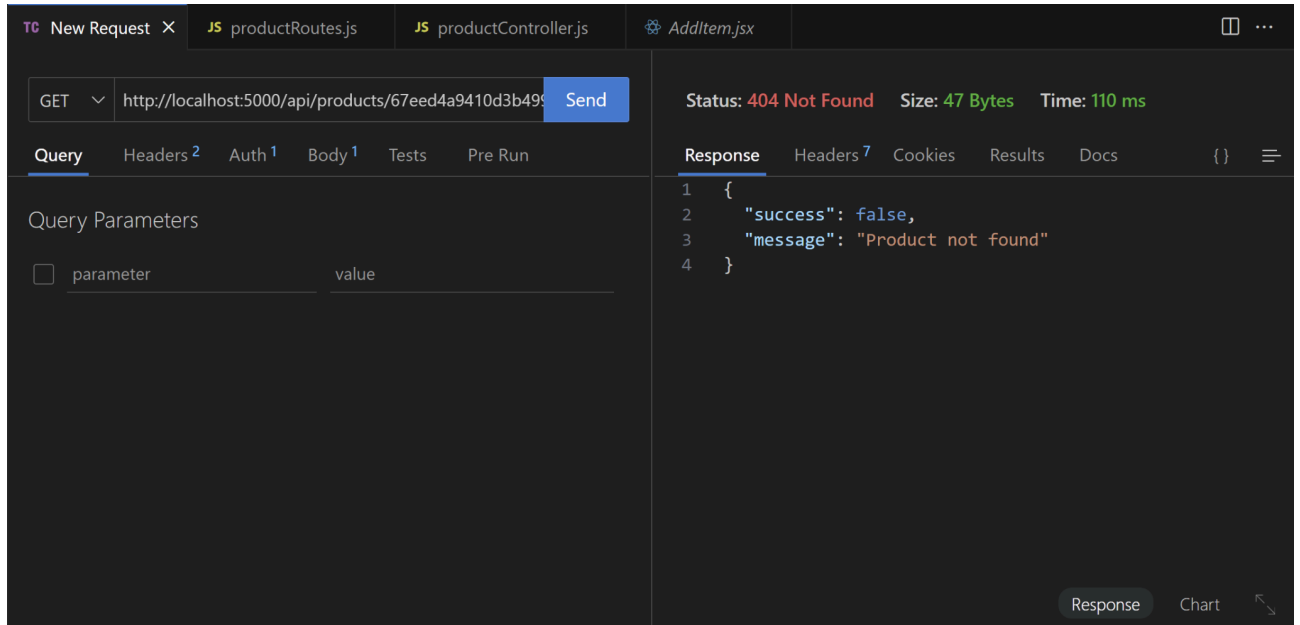
a) If product doesn't exist:

Test Owner: Nischay Agarwal

Test Date: 2/4/25

Test Results:

Outputs an error



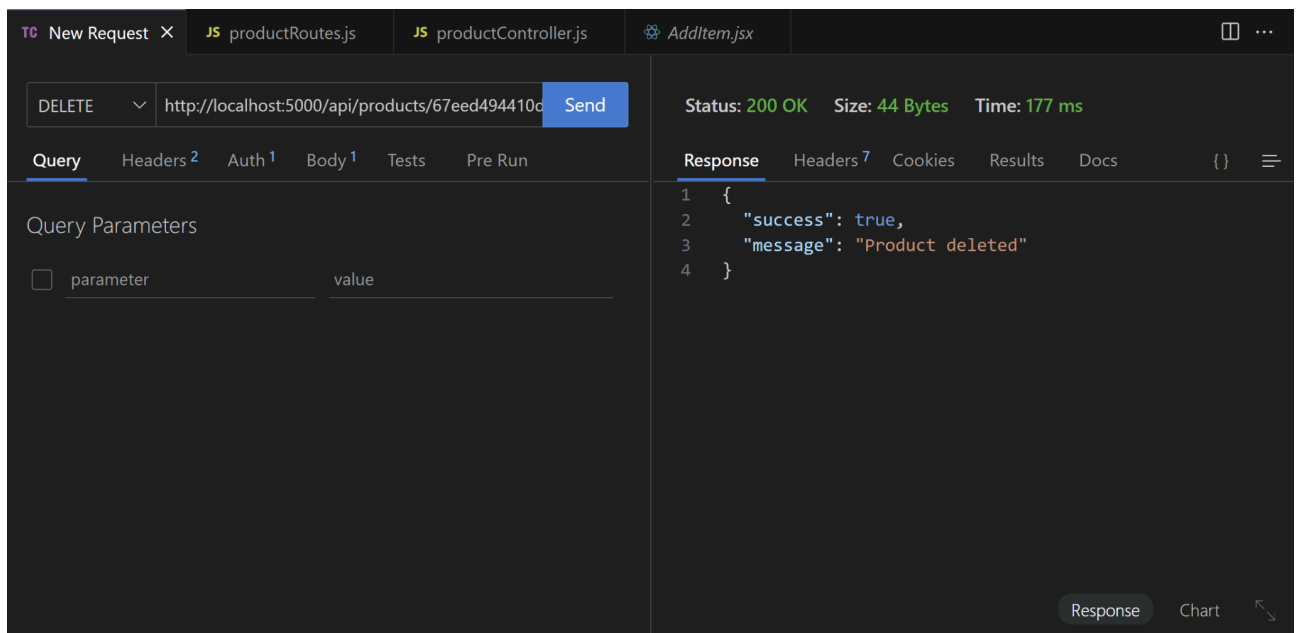
b) If product exists:

Test Owner: Nischay Agarwal

Test Date: 2/4/25

Test Results:

The product is successfully deleted



Structural Coverage: 50% branch coverage is achieved for `getProductById()`

8. My Items:

GETs the items reported by a given user based on their user ID

Unit Details: [Product Class, getProductsByUploader()]

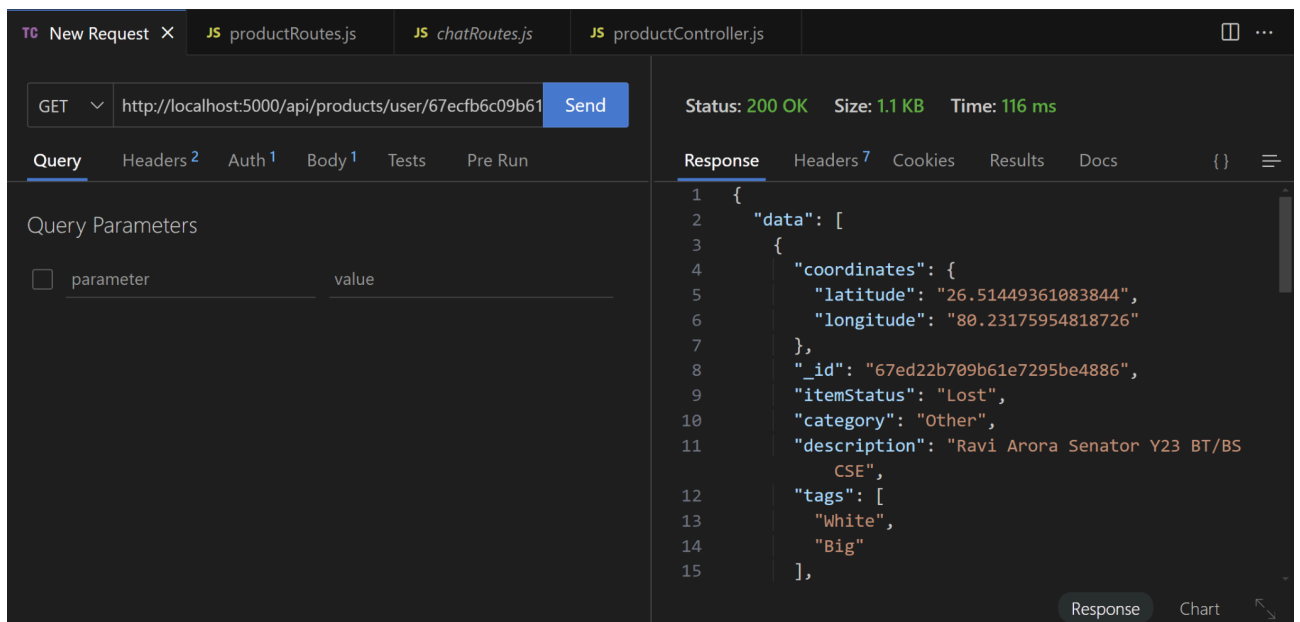
a) If user exists:

Test Owner: Dharajiya Yug

Test Date: 3/4/25

Test Results:

Successfully retrieves user uploaded products



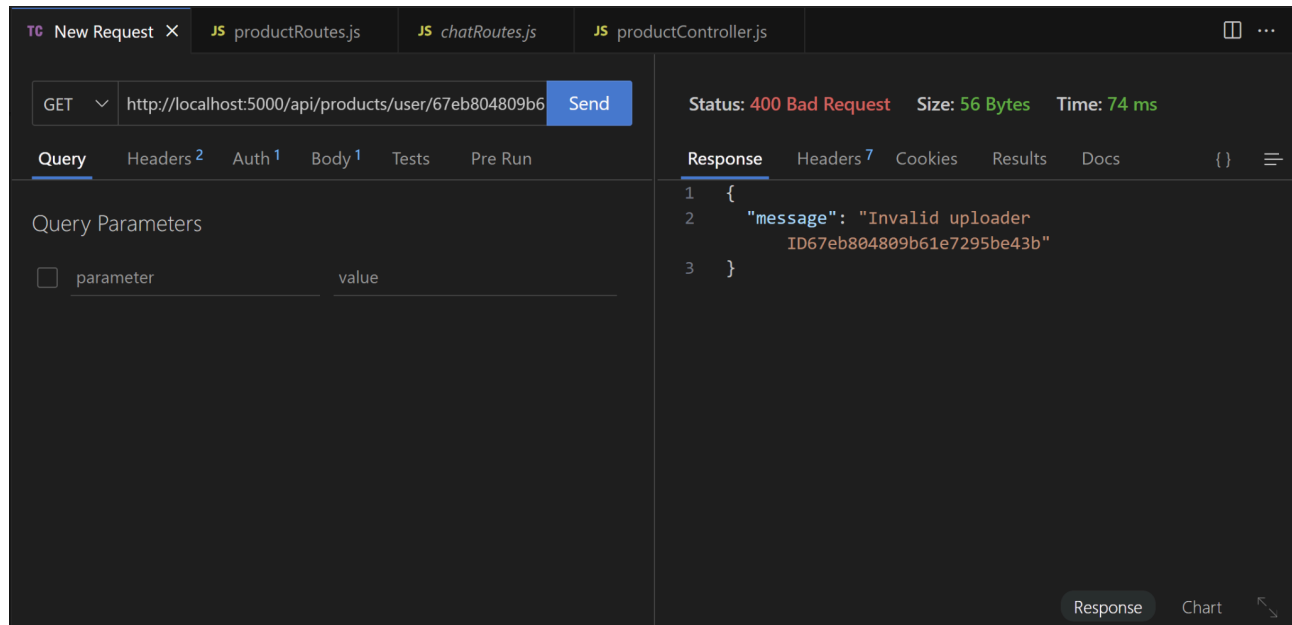
b) If user doesn't exist:

Test Owner: Dharajiya Yug

Test Date: 3/4/25

Test Results:

Throws an error



Structural Coverage: Full Branch coverage is achieved for `getProductsByUploader()`

9. Claim Item:

Tests the functionality of the “Claim” function from both the perspectives of the reporter and claimer

Unit Details: [Product Class, `addClaimRequest()` and `handleClaimRequest()`]

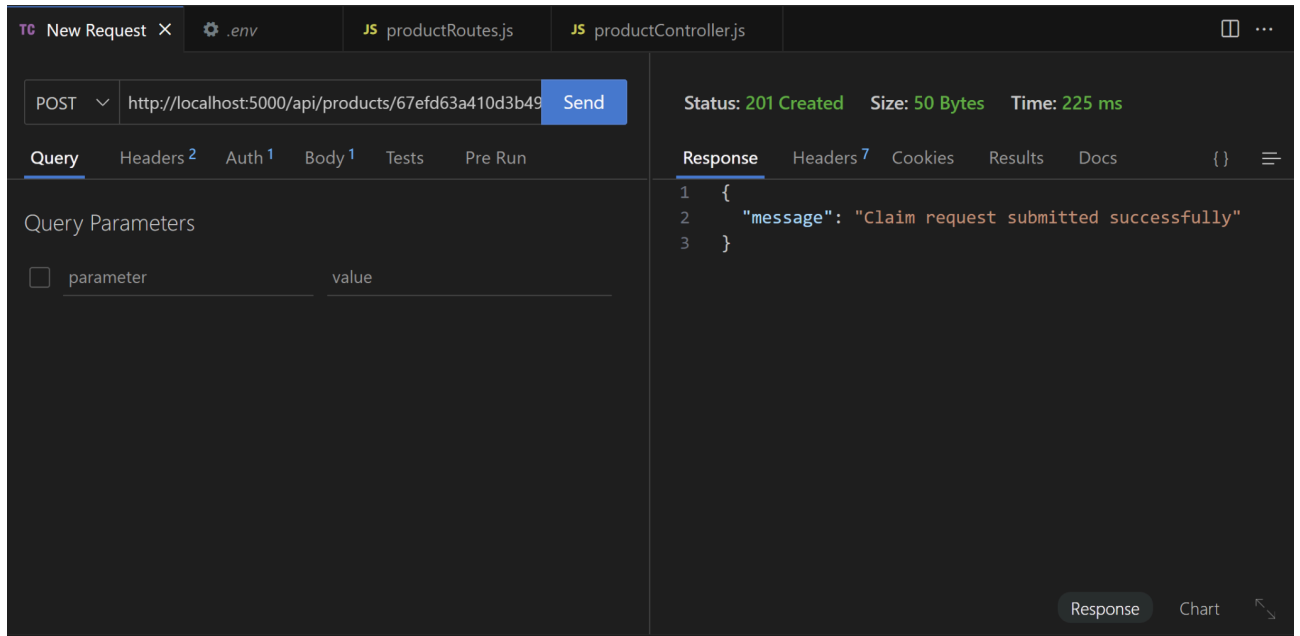
a) Claiming an Item:

Test Owner: Vihaan Sapra

Test Date: 3/4/25

Test Results:

Claim request is correctly placed



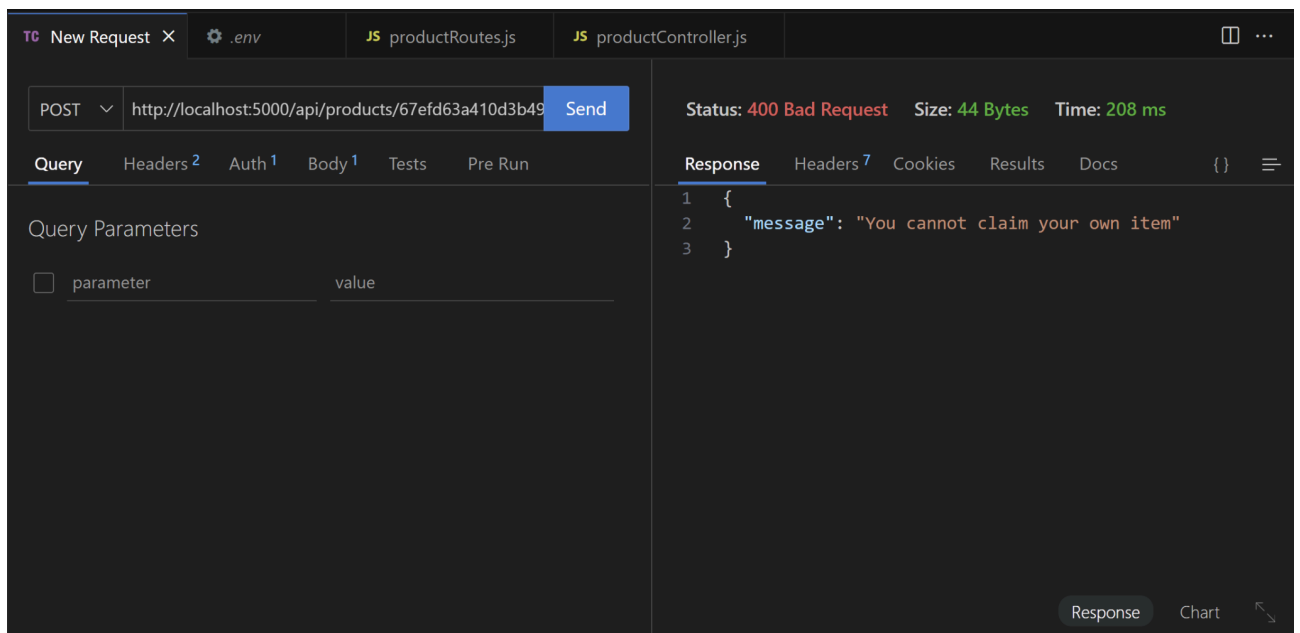
b) Claiming your own Item:

Test Owner: Vihaan Sapra

Test Date: 3/4/25

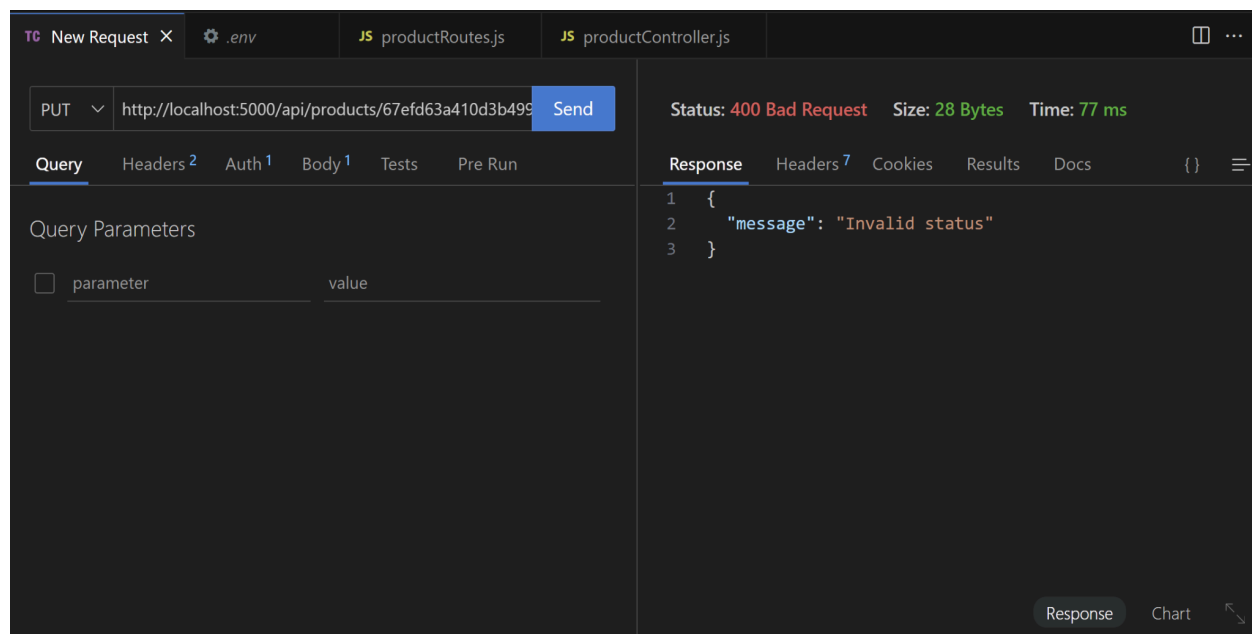
Test Results:

Error is thrown

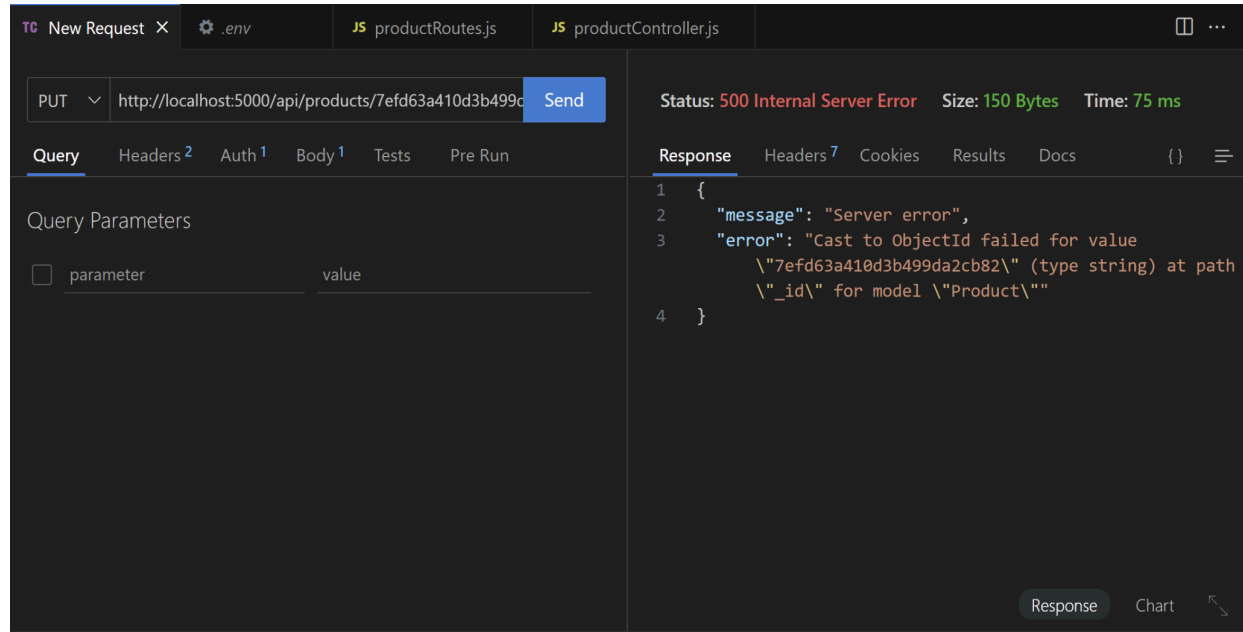


c) Accepting/rejecting a claim:**Test Owner:** Pratul Koolwal**Test Date:** 3/4/25i**Test Results:**1) Invalid status selected:**Test Owner:** Pratul Koolwal**Test Date:** 3/4/25**Test Results:**

Error is thrown

2) Invalid item ID:**Test Owner:** Pratul Koolwal**Test Date:** 3/4/25**Test Results:**

Error is thrown



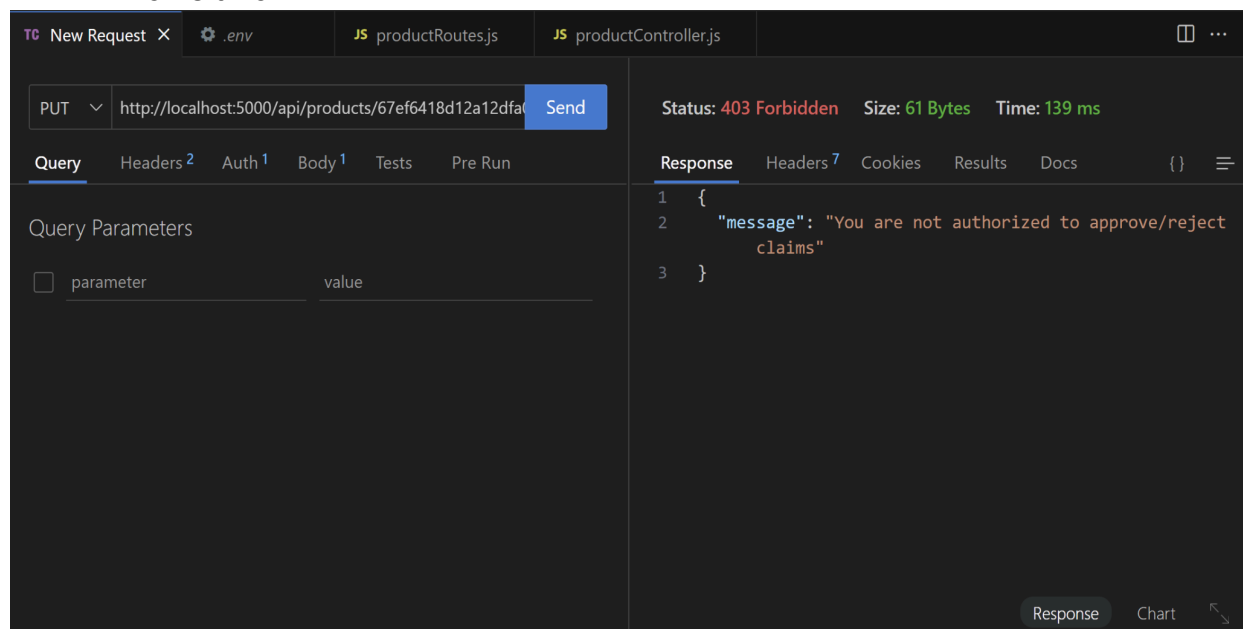
3) Handling a claim on someone else's product:

Test Owner: Siddharth Miglani

Test Date: 3/4/25

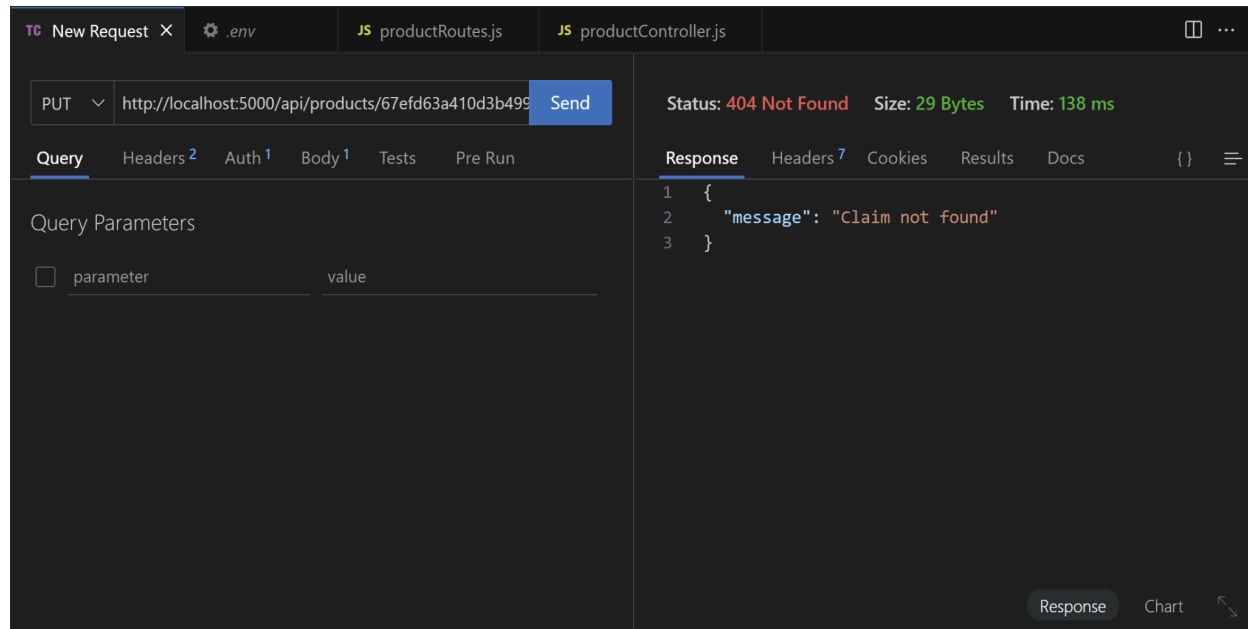
Test Results:

Error is thrown

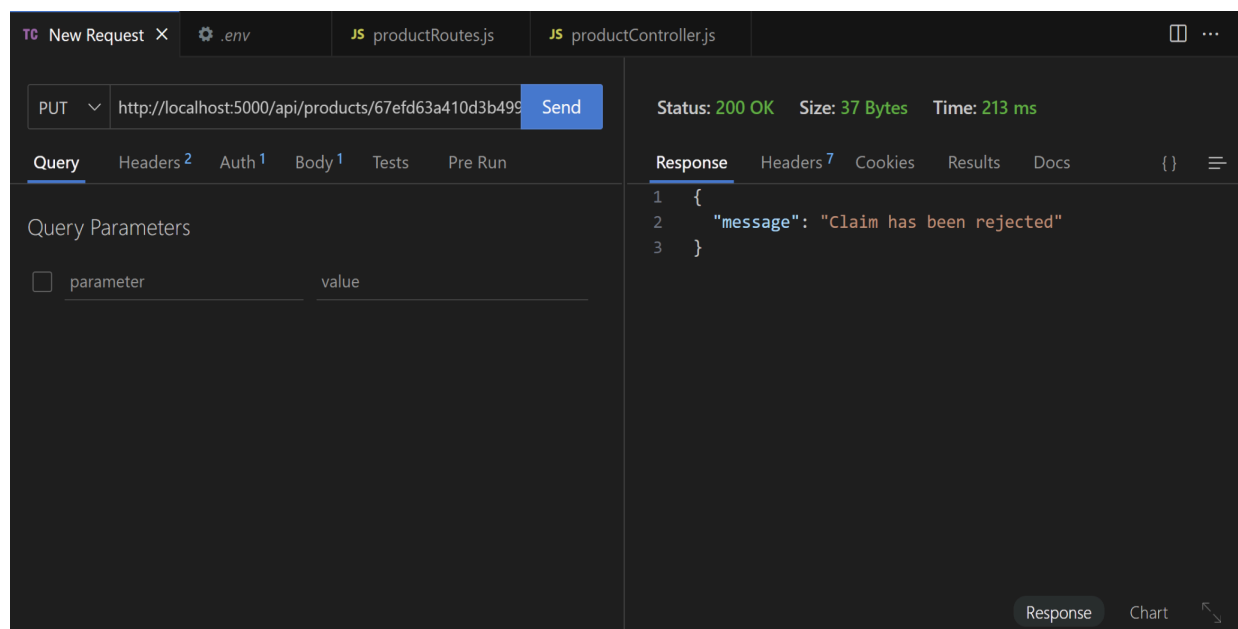
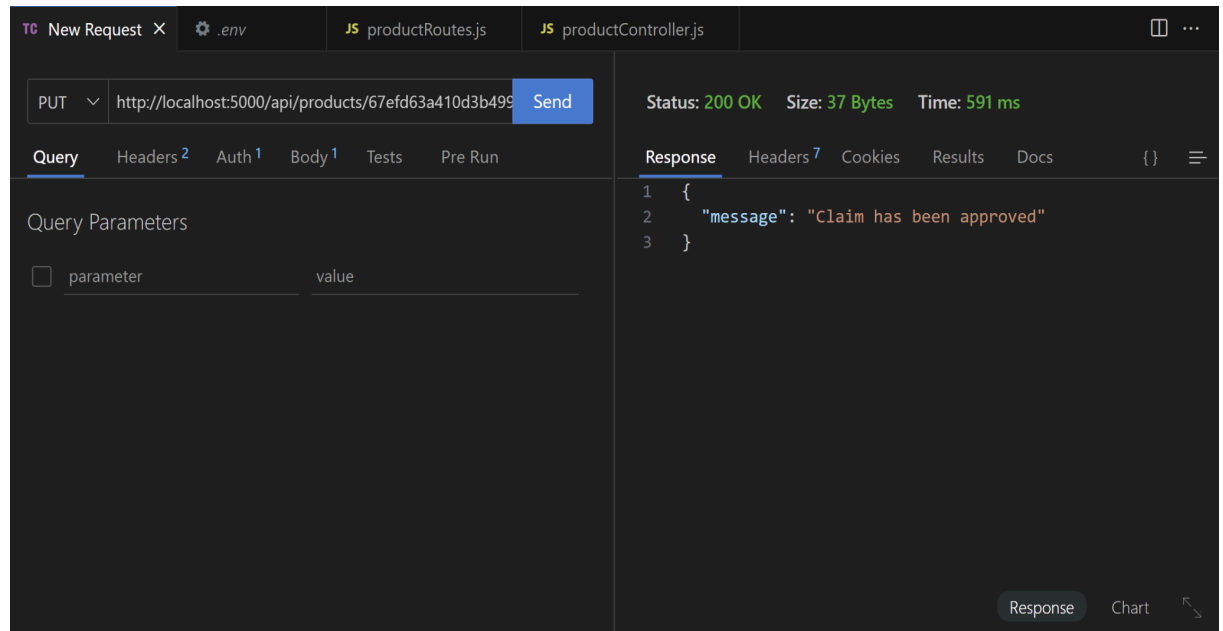


4) Handling an invalid claim:**Test Owner:** Siddharth Miglani**Test Date:** 3/4/25**Test Results:**

Error is thrown

5) Approving/Rejecting a claim normally:**Test Owner:** Shravan Agarwal**Test Date:** 3/4/25**Test Results:**

Claim Status is successfully updated



Structural Coverage: Complete Branch coverage is achieved for addClaimRequest() and handleClaimRequest()

10. Profile Data:

Show Profile details

Unit Details: [User Class, getProfile() and updateProfile()]

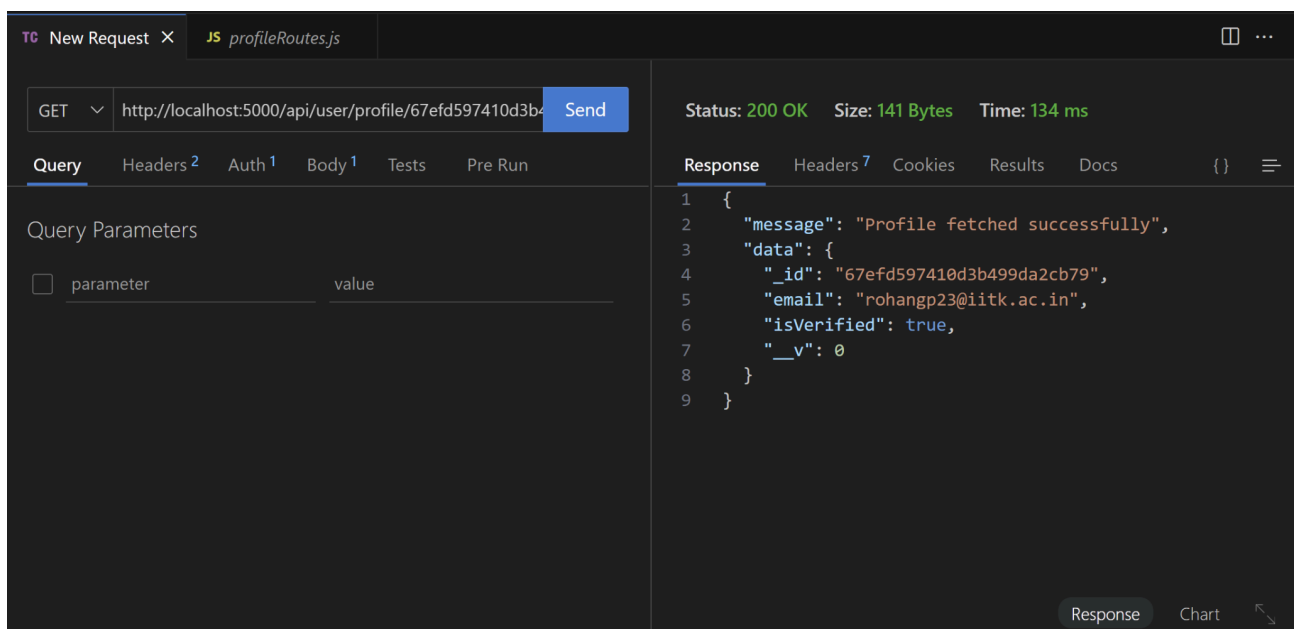
a) Fetch profile details:

Test Owner: Aarush Ghate

Test Date: 4/4/25

Test Results:

Details are successfully retrieved



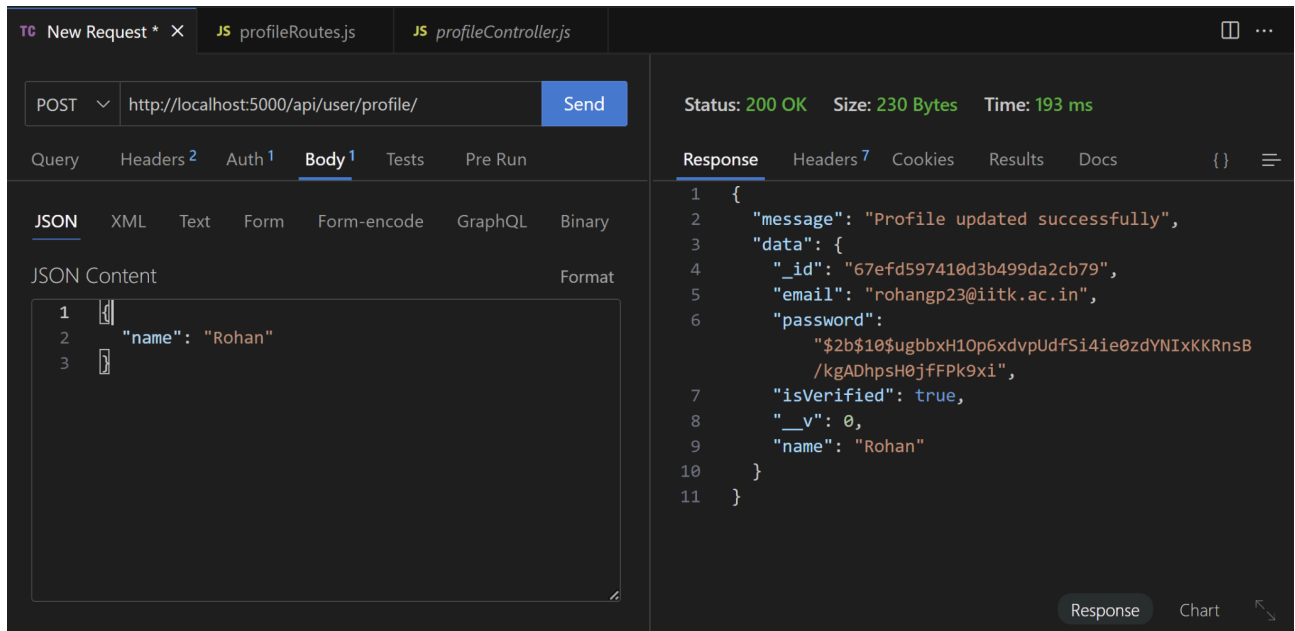
b) Update profile details:

Test Owner: Aarush Ghate

Test Date: 4/4/25

Test Results:

Details are successfully updated



Structural Coverage: 50% Branch coverage is achieved for `getProfile()` and `updateProfile()` respectively

11. Chatting functionality:

Tests if the chatting functionality between users is working as intended

Unit Details: [Chats Class, `sendMessage()`, `getMessages()` and `getPreviousChats()`]

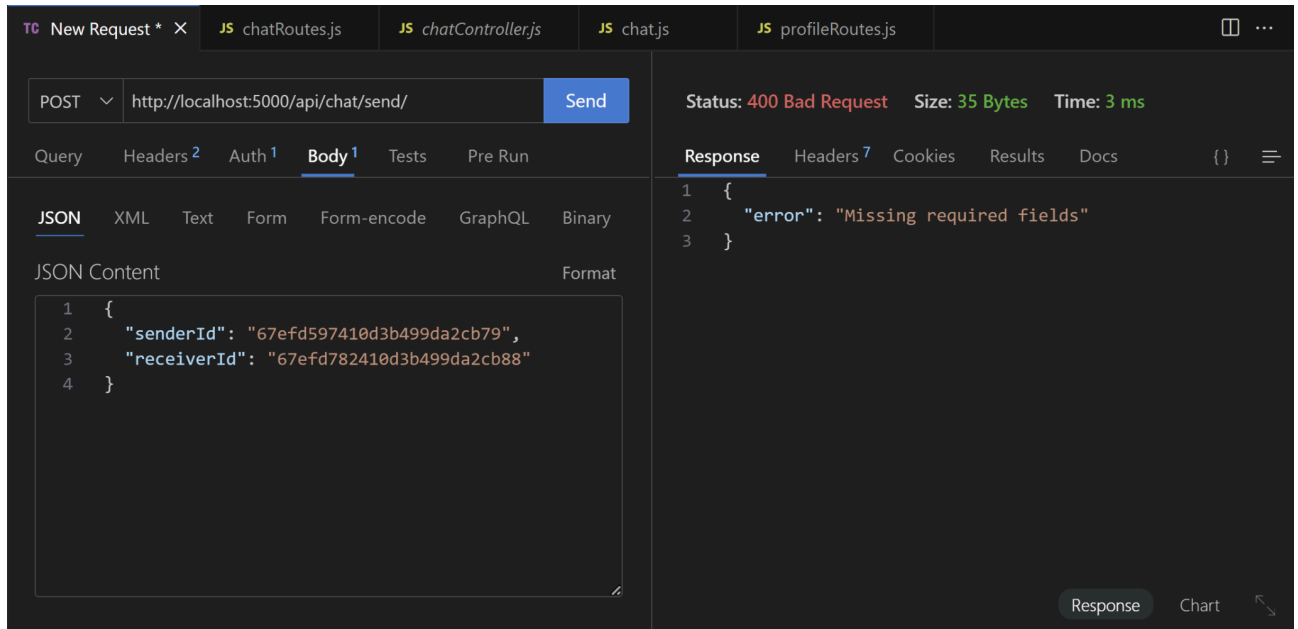
a) If some fields are missing:

Test Owner: Suyash Kapoor

Test Date: 4/4/25

Test Results:

An error is thrown



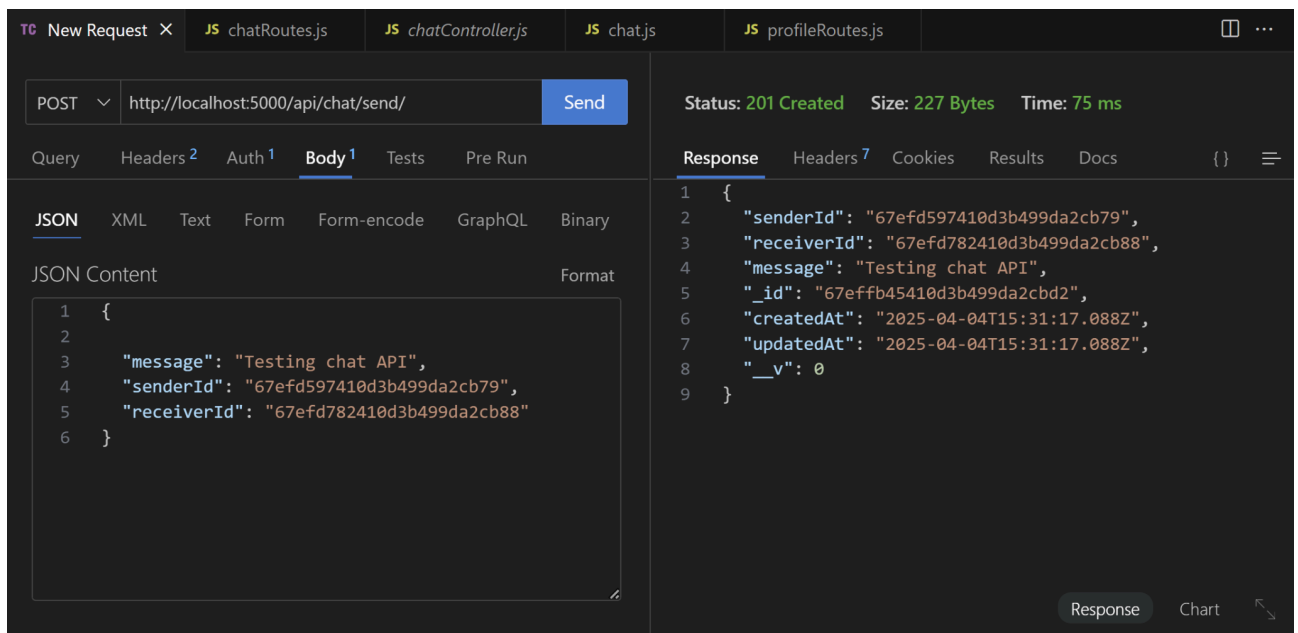
b) Sending message to another user:

Test Owner: Suyash Kapoor

Test Date: 4/4/25

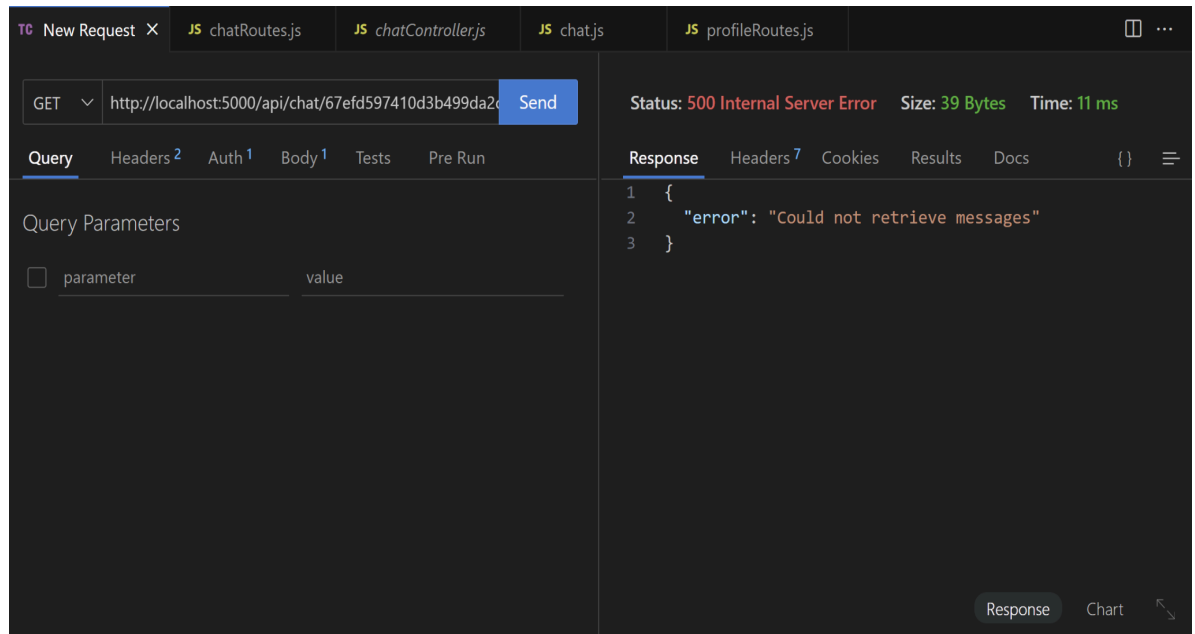
Test Results:

Message is sent successfully

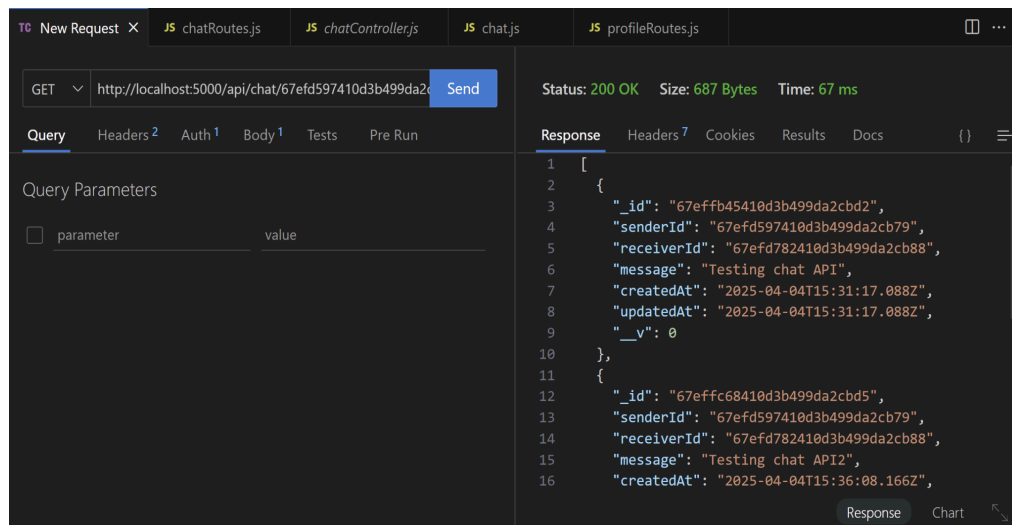


c) Get chat history between 2 users:**Test Owner:** Nischay Agarwal**Test Date:** 4/4/25**Test Results:**1) If both userIDs are not given:

Error is thrown

2) Normal operation:

Charts are successfully displayed



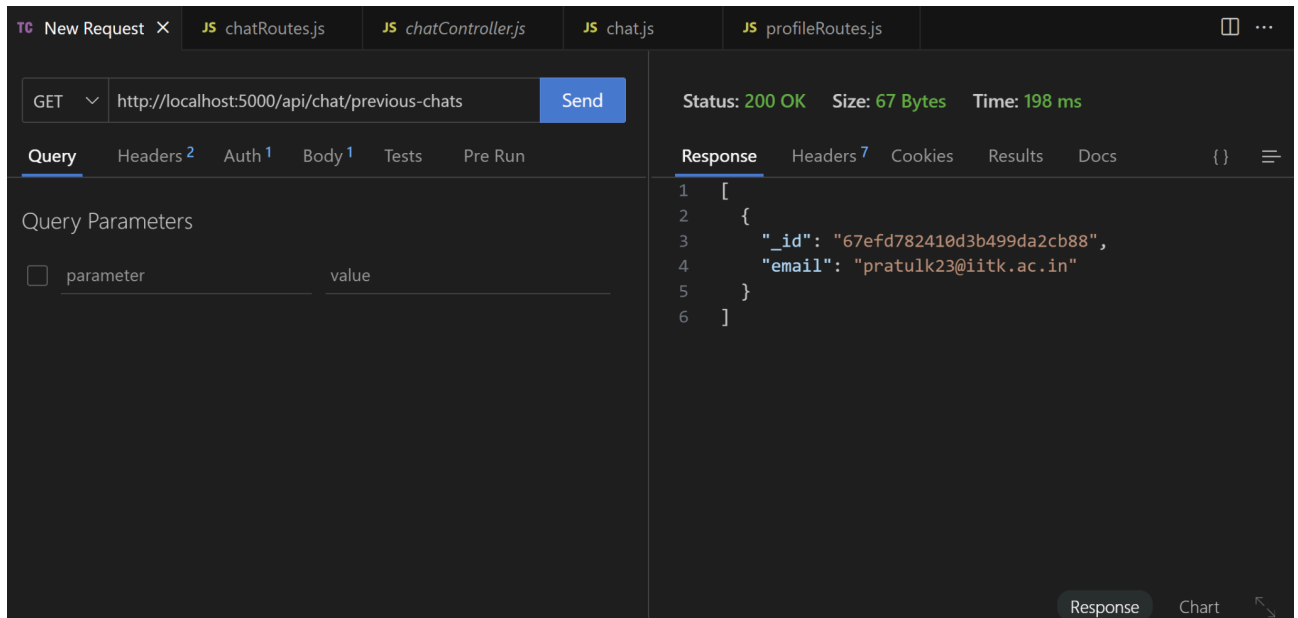
d) Get all the chats of the currently logged in user:

Test Owner: Nischay Agarwal

Test Date: 4/4/25

Test Results:

Successfully gets all the chats of user



Structural Coverage: Complete branch coverage is achieved for all three functions

12. Admin Functionality:

Tests out the extra functionalities provided to the admin user

Unit Details: [User Class, Admin Functions]

a) Banning other users:

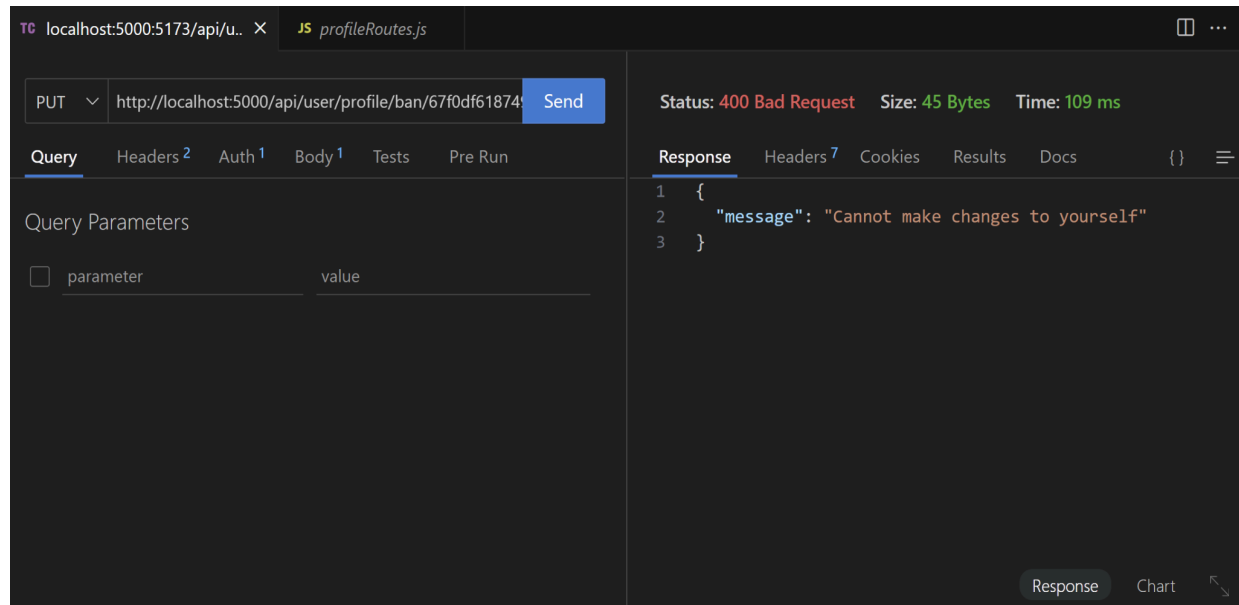
Test Owner: Vihaan Sapra

Test Date: 4/4/25

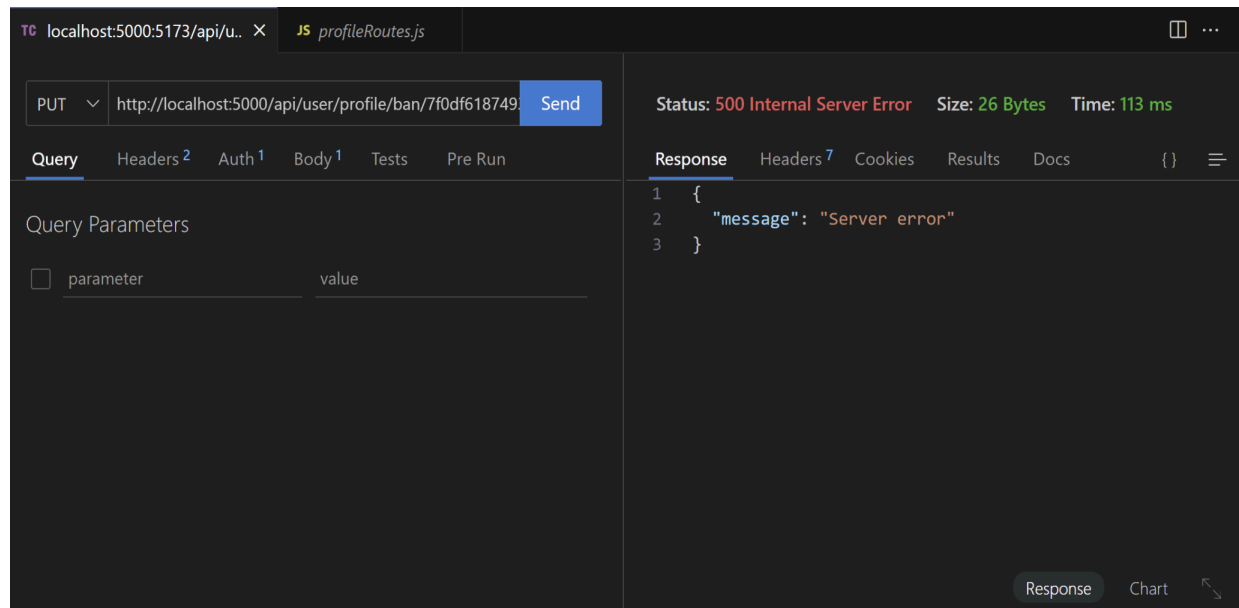
Test Results:

1) Banning yourself:

Error is thrown

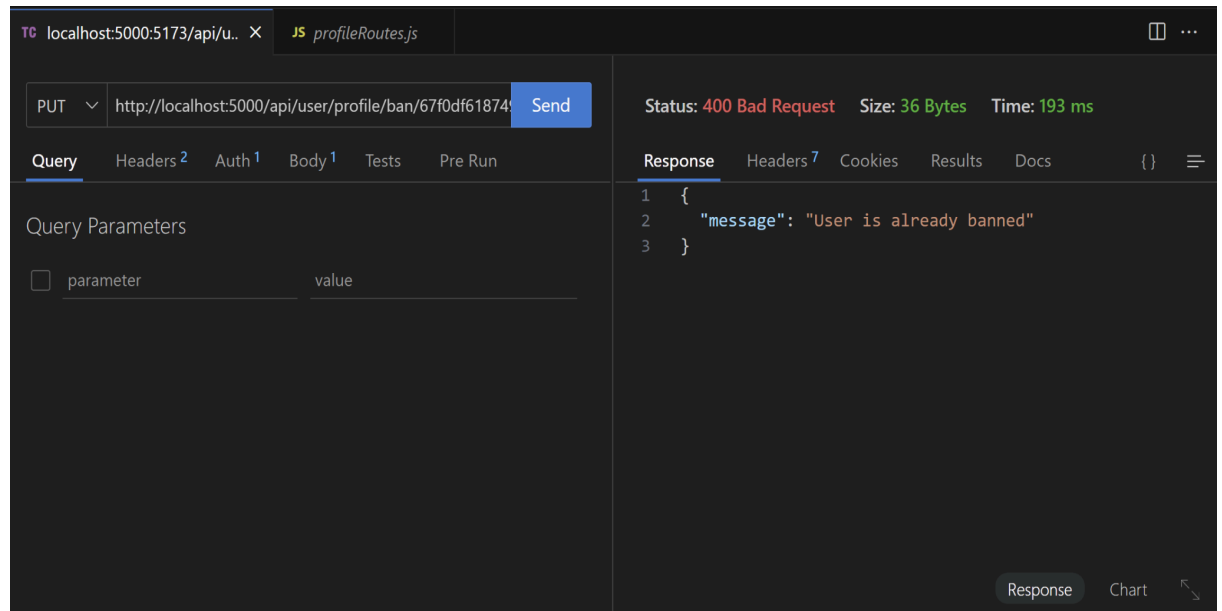
2) Banning unknown user:

Incorrect error is thrown (should have thrown “User not Found” instead threw “Server Error”)



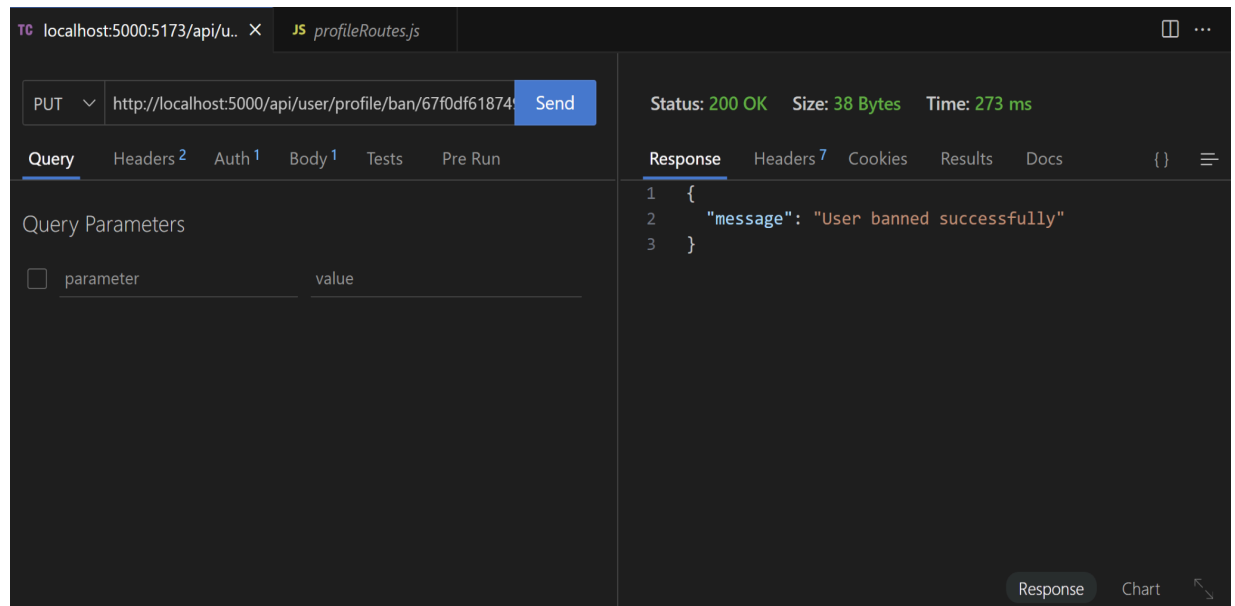
3) Banning user who is already banned:

Error is thrown



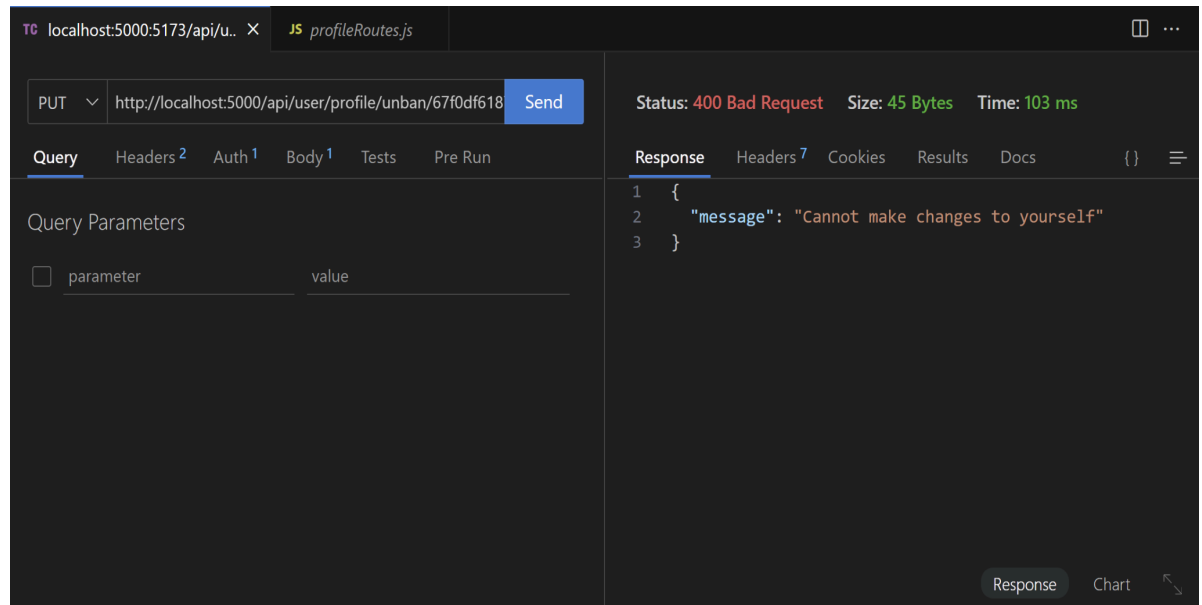
4) Normal function:

User is successfully Banned

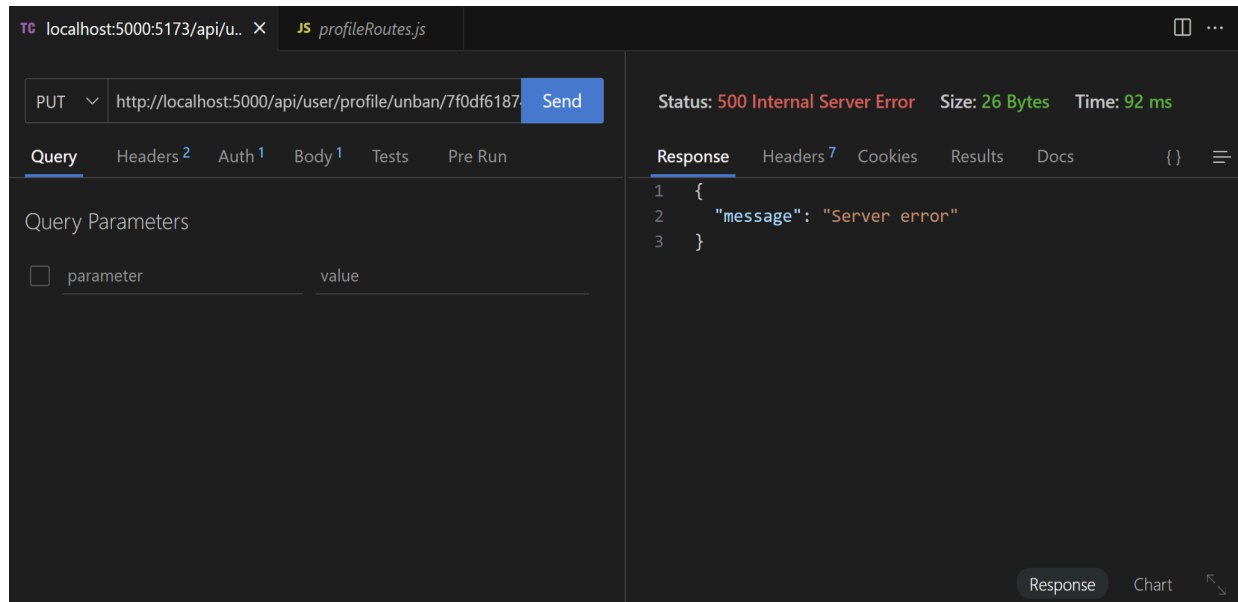


b) Unbanning users:**Test Owner:** Rudra Sinha**Test Date:** 4/4/25**Test Results:**1) Unbanning yourself:

Error is thrown

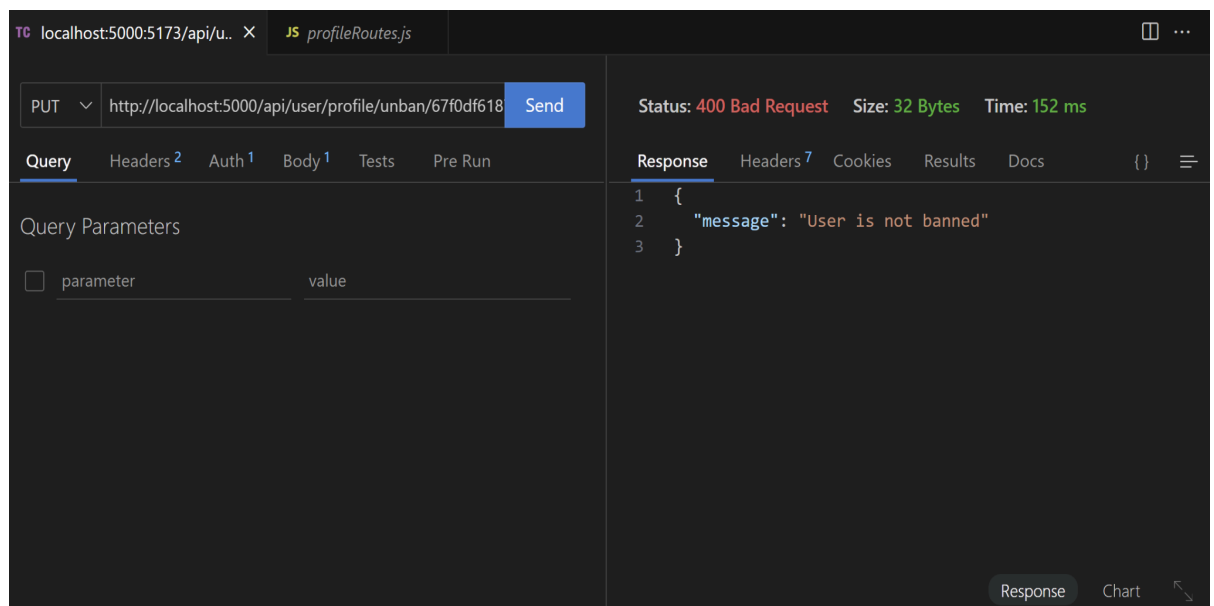
2) Unbanning unknown user:

Incorrect error is thrown (should have thrown "User not Found" instead threw "Server Error")



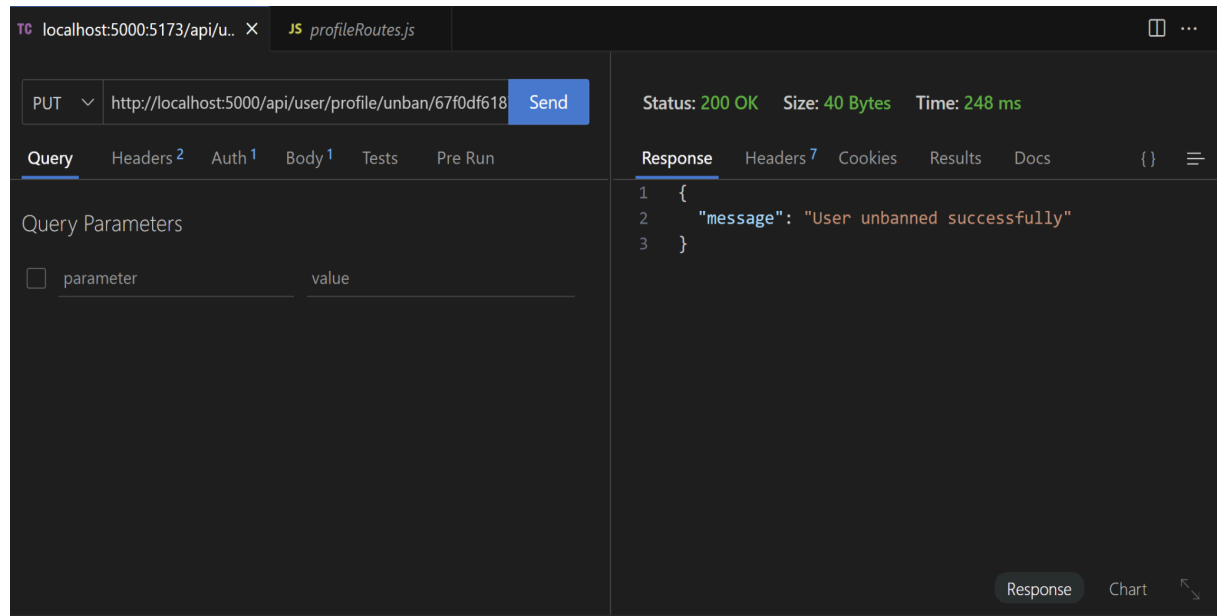
3) Unbanning a user who isn't banned:

Error is thrown



4) Normal function:

Ban is successfully lifted

c) Promoting user to admin:

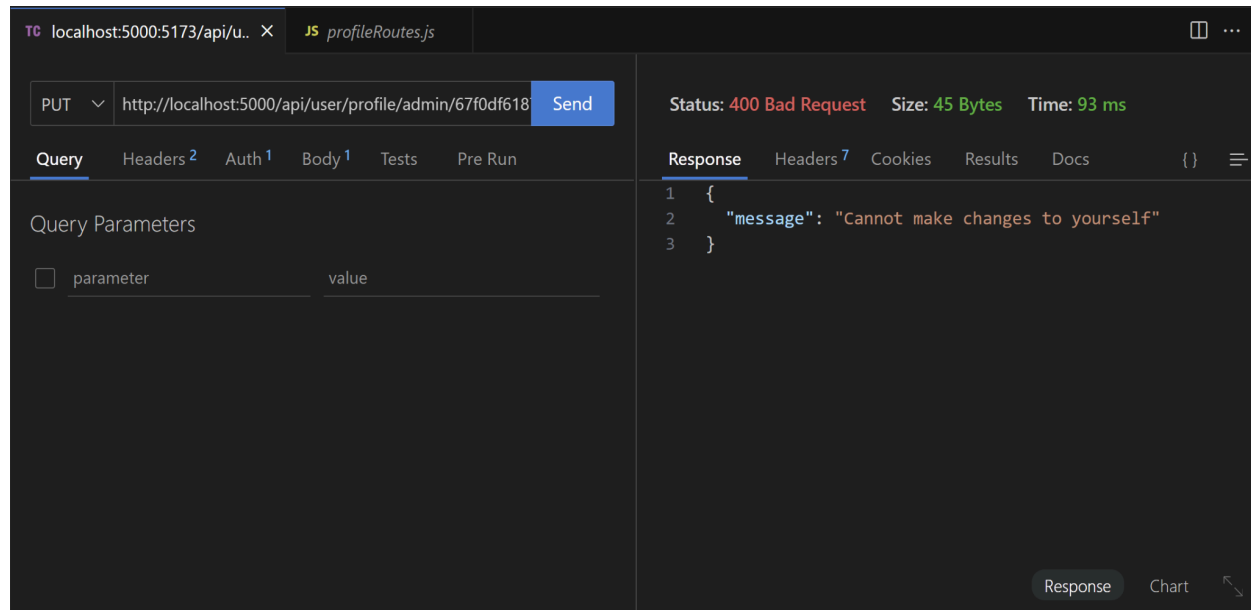
Test Owner: Suyash Kapoor

Test Date: 4/4/25

Test Results:

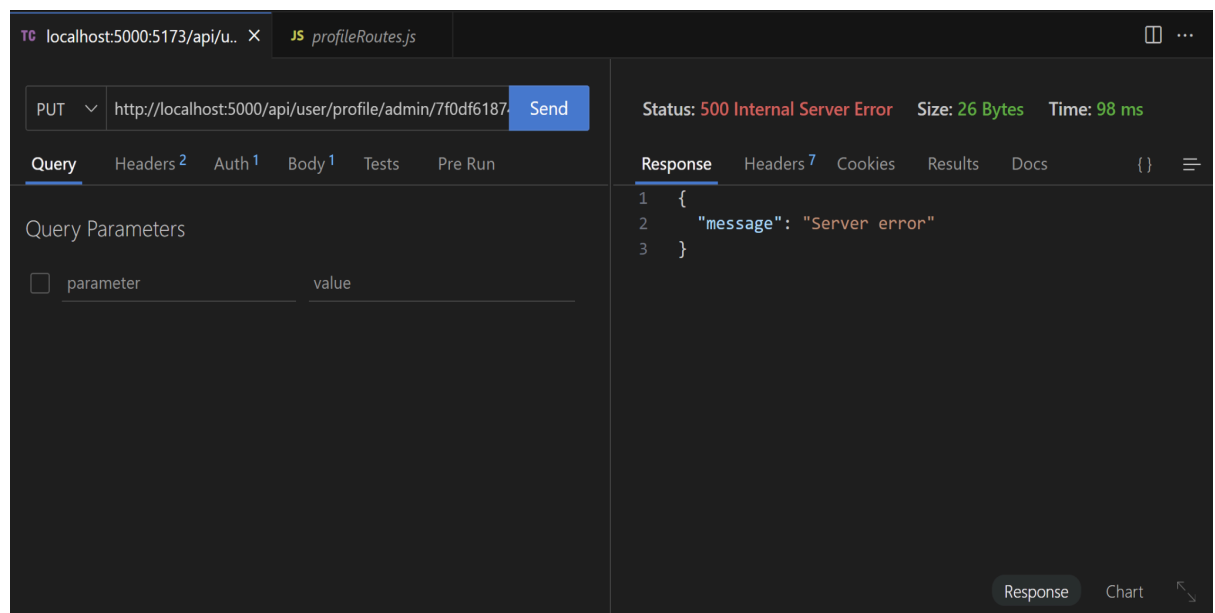
1) Promoting yourself to Admin:

Error is thrown



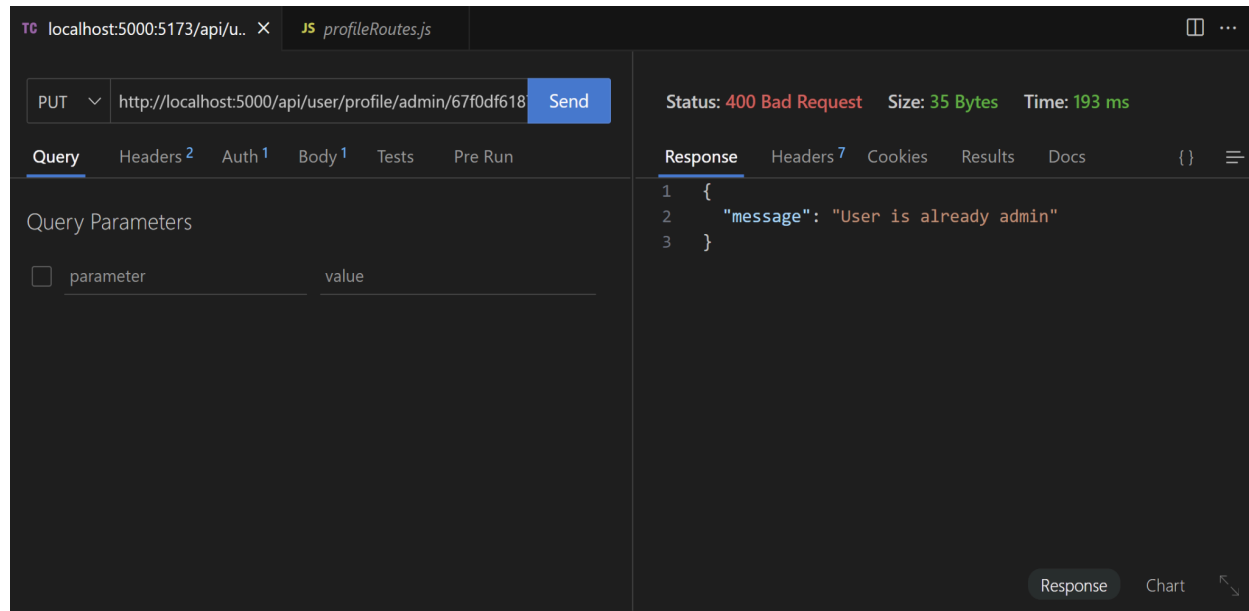
2) Promoting unknown user:

Incorrect error is thrown (should have thrown “User not Found” instead threw “Server Error”)



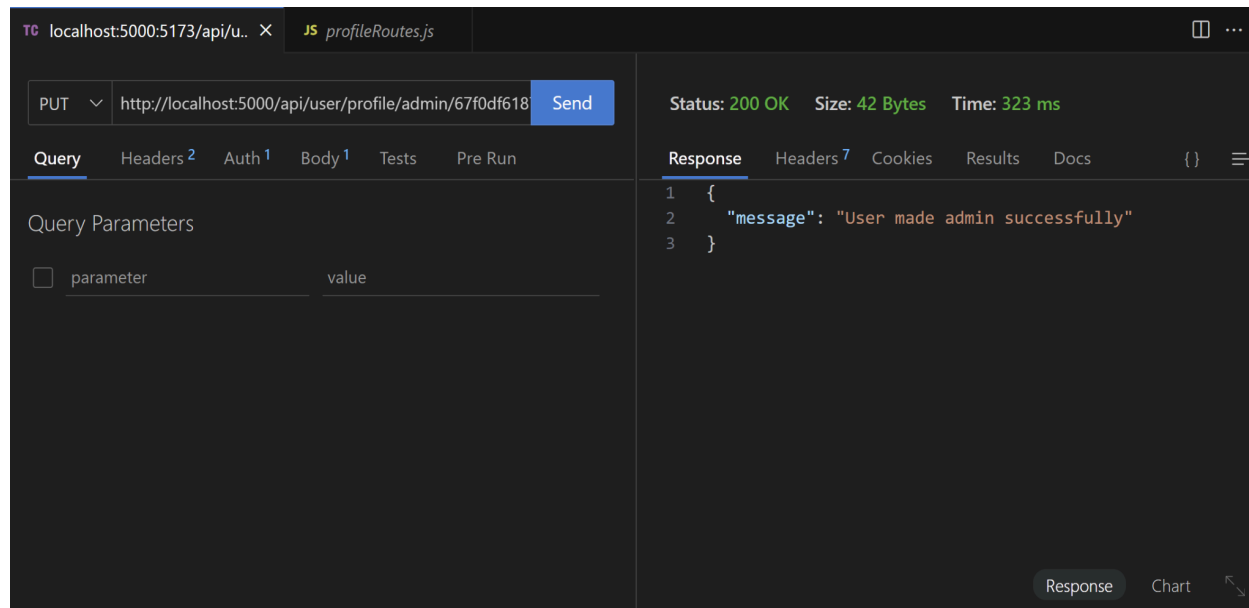
3) Promoting a user who is already Admin:

Error is thrown



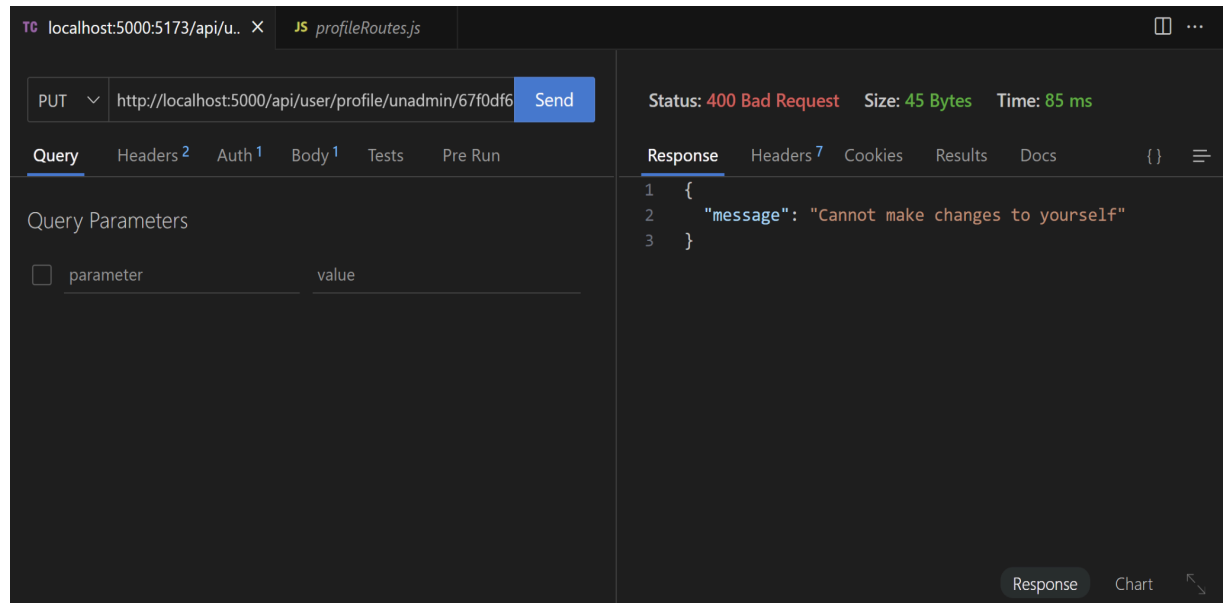
4) Normal Function:

User is successfully promoted

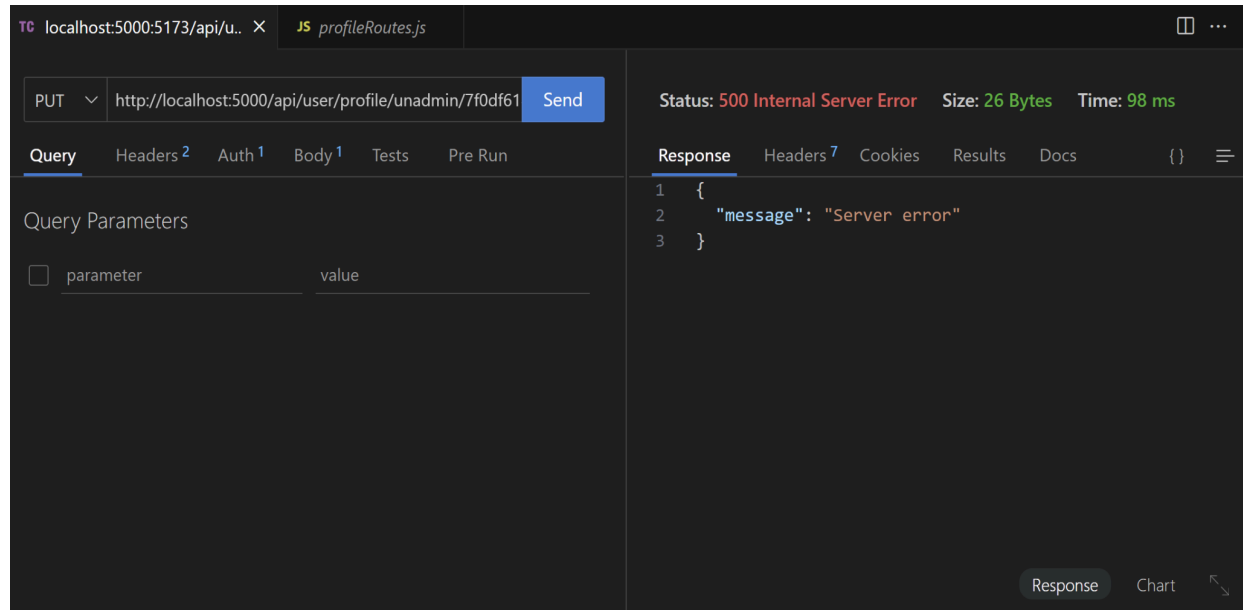


d) Demoting admins:**Test Owner:** Aarush Ghate**Test Date:** 4/4/25**Test Results:**1) Demoting Yourself:

Error is thrown

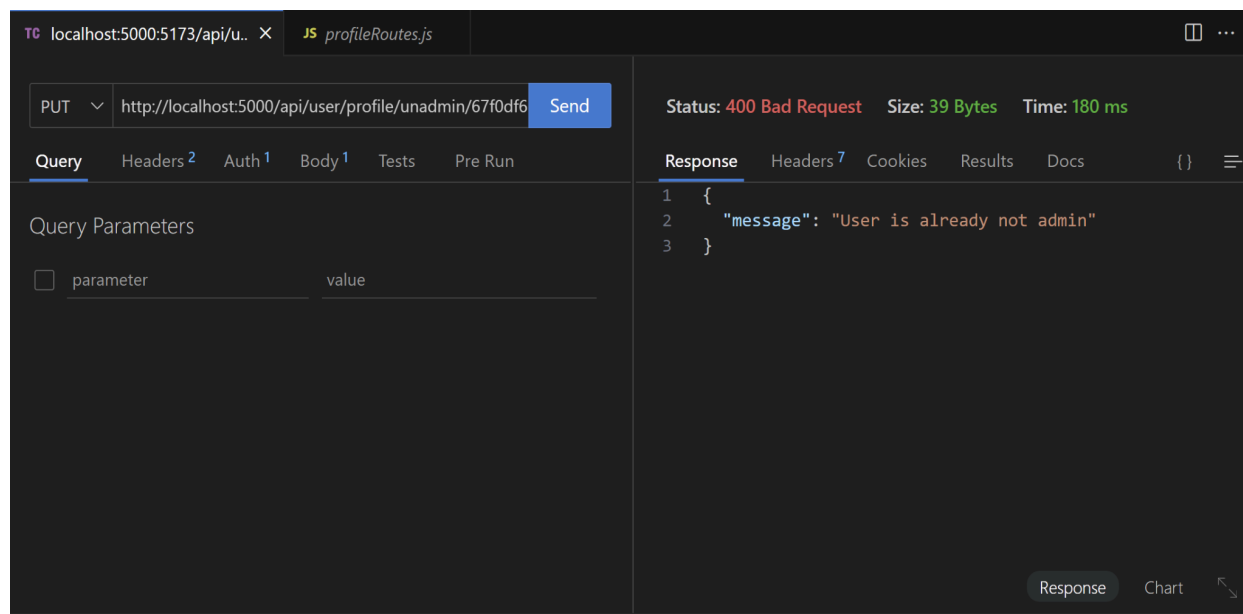
2) Demoting Unknown User:

Incorrect error is thrown (should have thrown "User not Found" instead threw "Server Error")



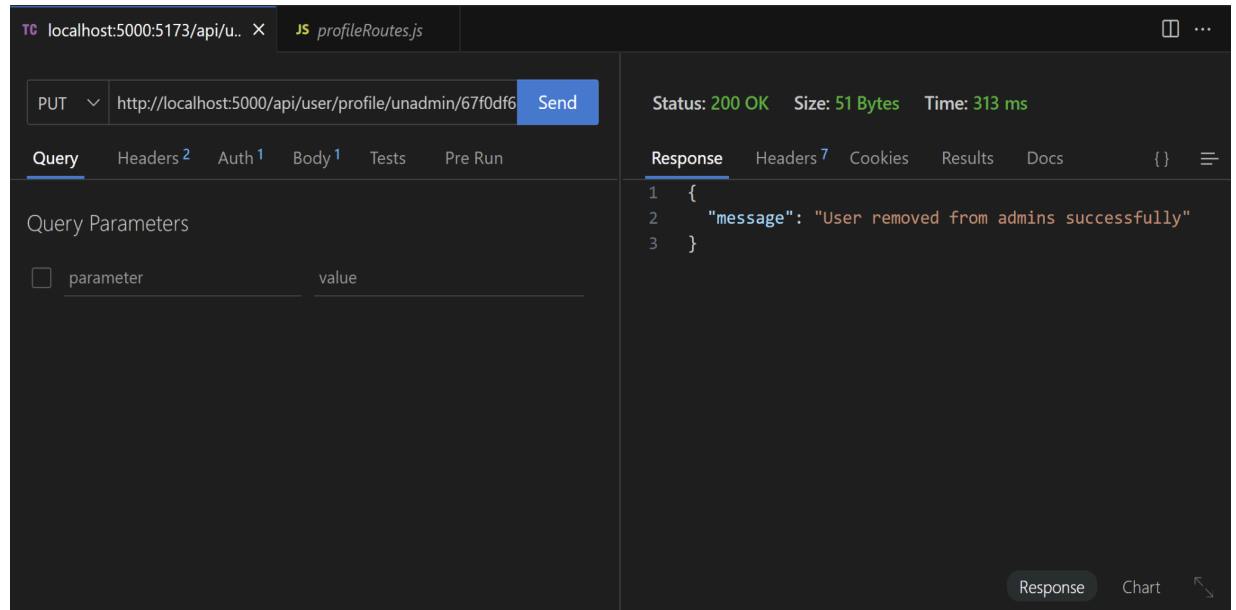
3) Demoting a Regular user:

Error is thrown



4) Normal Function:

User is successfully demoted



Structural Coverage: Complete branch coverage is achieved for all admin functions

Additional Comments:

Minor bug was discovered where, if the given userID is invalid, instead of returning a “User not Found” error, it returns a “Server Error”. The rest of the functionality is as expected

3 Integration Testing

Integration testing involves testing whether the backend and frontend interact the right way and produce the desired behaviour. Both backend and frontend units have been individually tested by the unit testing team. We have performed integration testing as follows. We gave the input to the frontend and observed final changes that occur after the backend processes the request sent by the frontend. Correct final behaviour implies that the interaction between backend and frontend was alright. Apart from this, integration testing also involves the interaction between different components, redirection of pages and updation of pages whenever a change is made to the database from anywhere.

1. Authentication:

Module Details:

The module encompasses testing procedures for user registration through OTP verification, login for already registered users, and the forgot password feature. Additionally, it includes validating the redirection to respective pages upon completion of these actions.

Test Owner: Pratul Koolwal

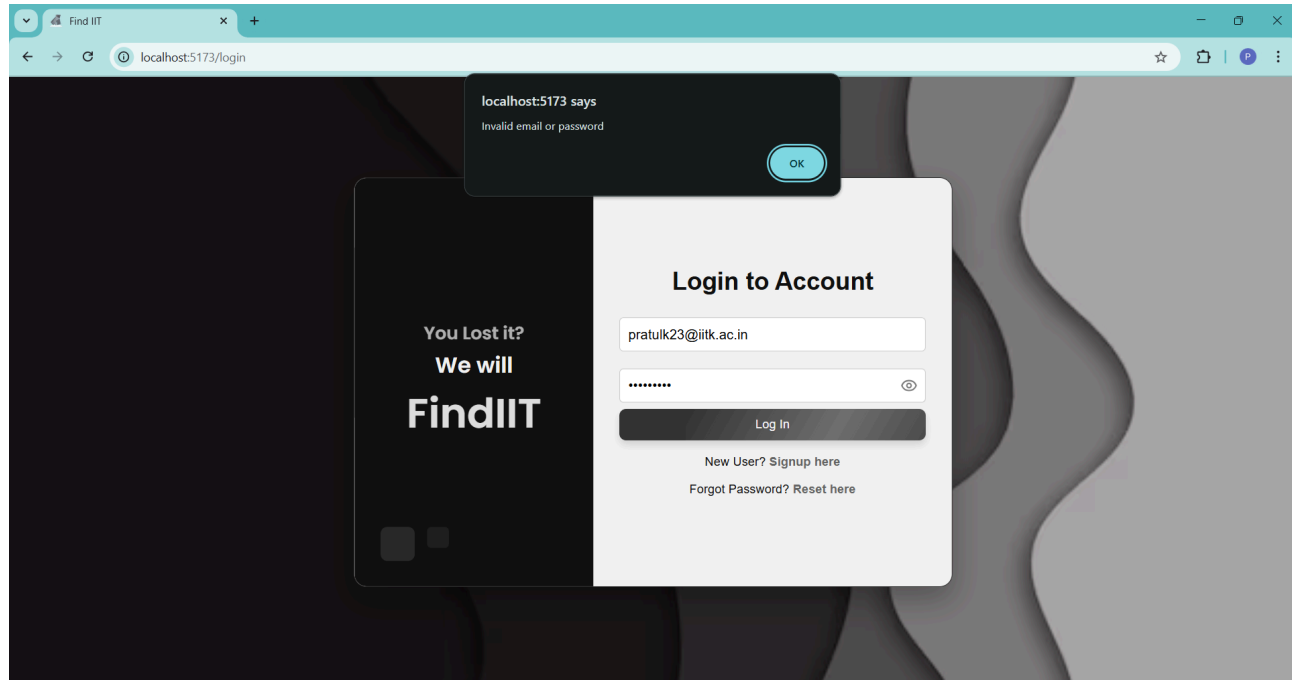
Test Date: 03/04/2025

Test Results:

In this test we validated the successful integration of backend and frontend components of SignUp, Login, Email Verification and Forgot password feature. We also validated that the database is updated on registration of new users and password is stored in encrypted format to maintain the security of the user. Here we are integrating the entire Authentication API using frontend. Throughout the testing process, we also scrutinized the redirection of users to the correct pages upon the completion of these actions.

TEST #1:

Details : Invalid credentials entered while trying to login.



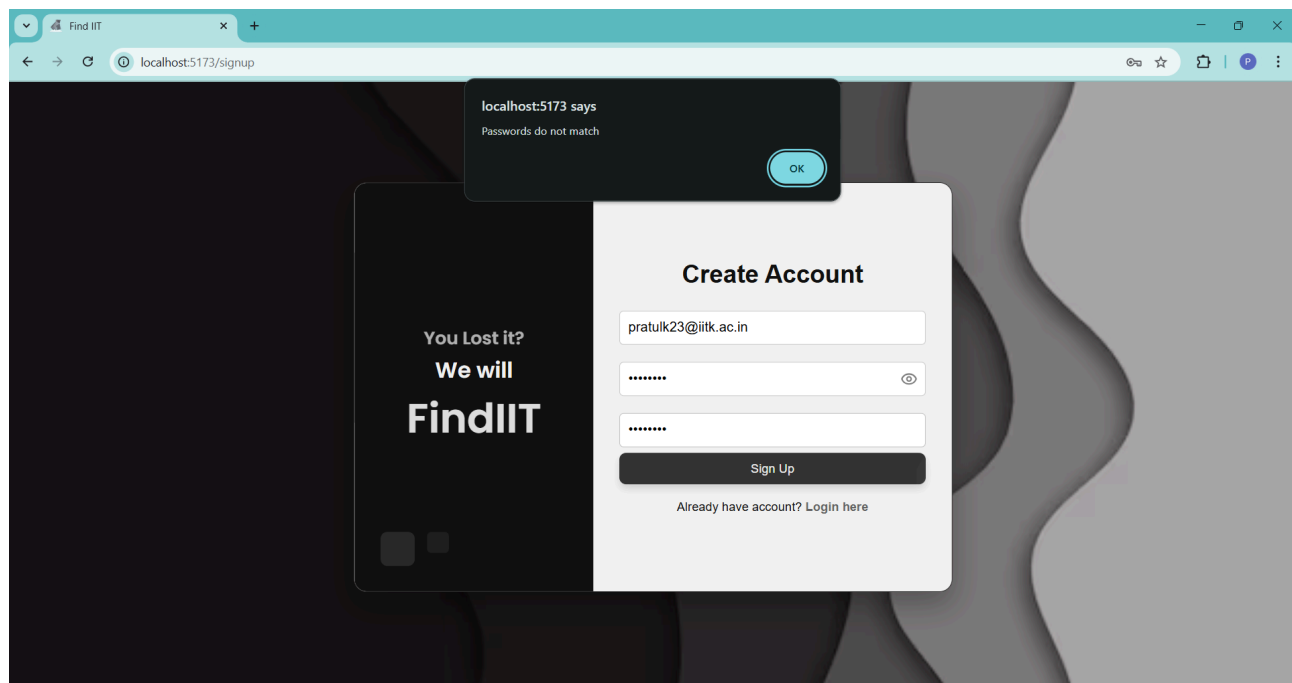
Result : Pop-up displaying “Invalid email or password”. Login is denied.

TEST #2:

Details:OTP Verification and registration of a new user.

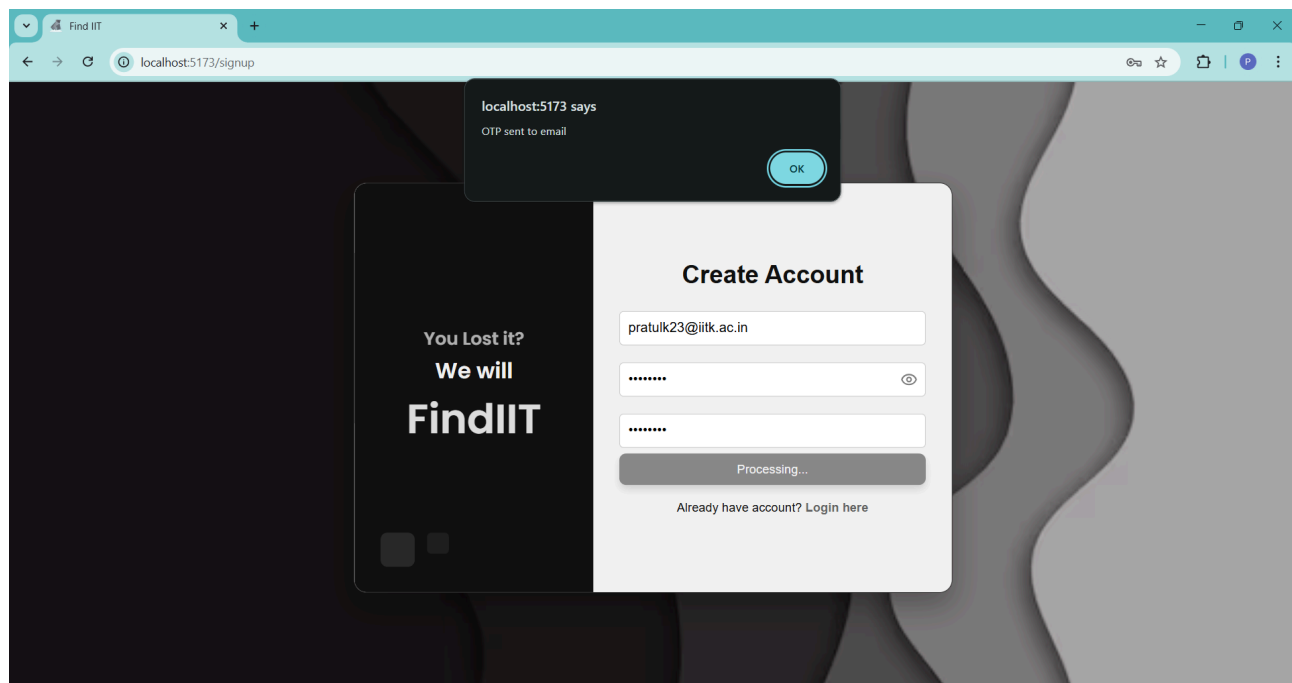
Flow: Email, along with password and confirm password entered in Sign Up page -> OTP mailed to user -> directed to Verify OTP page -> User directed back to login page after registering successfully and user's data updated in MongoDB database

Password and Confirm Password do not match:

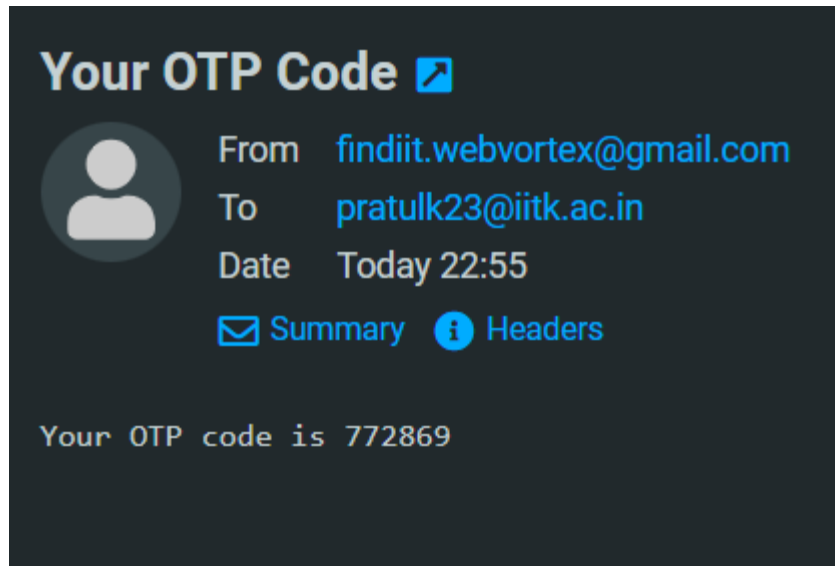


Result : Pop-up displaying “Passwords do not match”. Sign Up is denied.

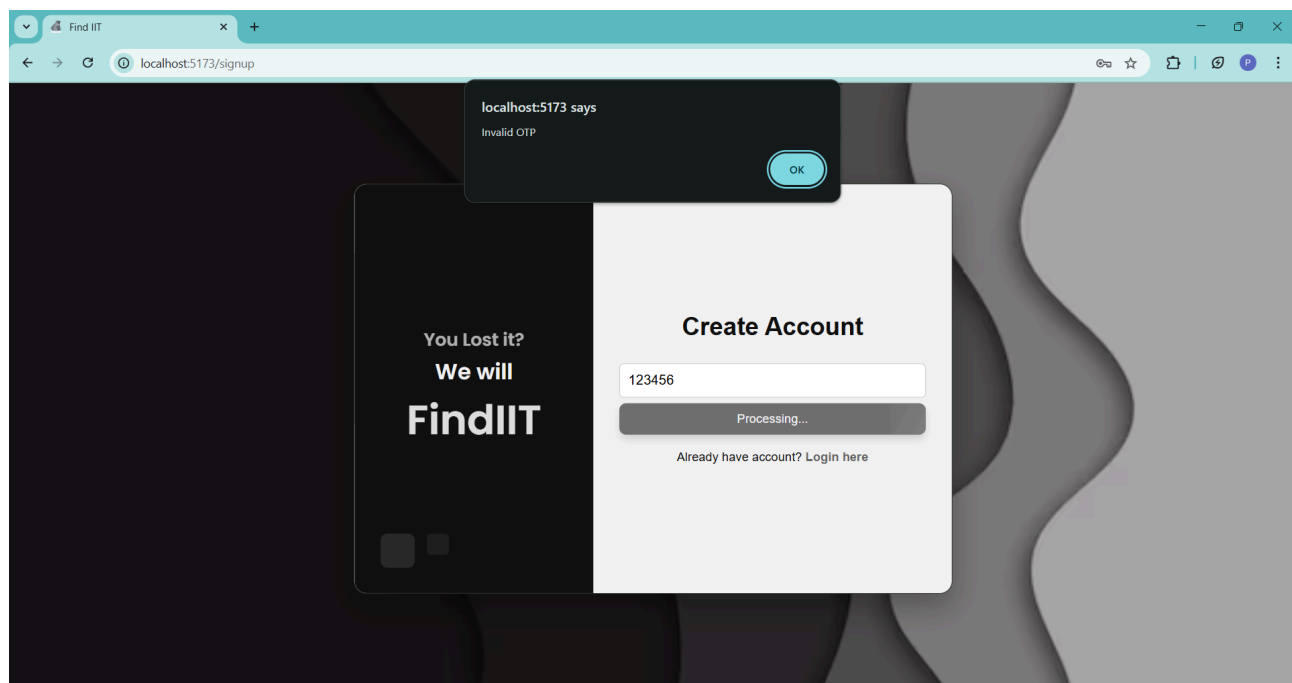
Password and Confirm Password match:



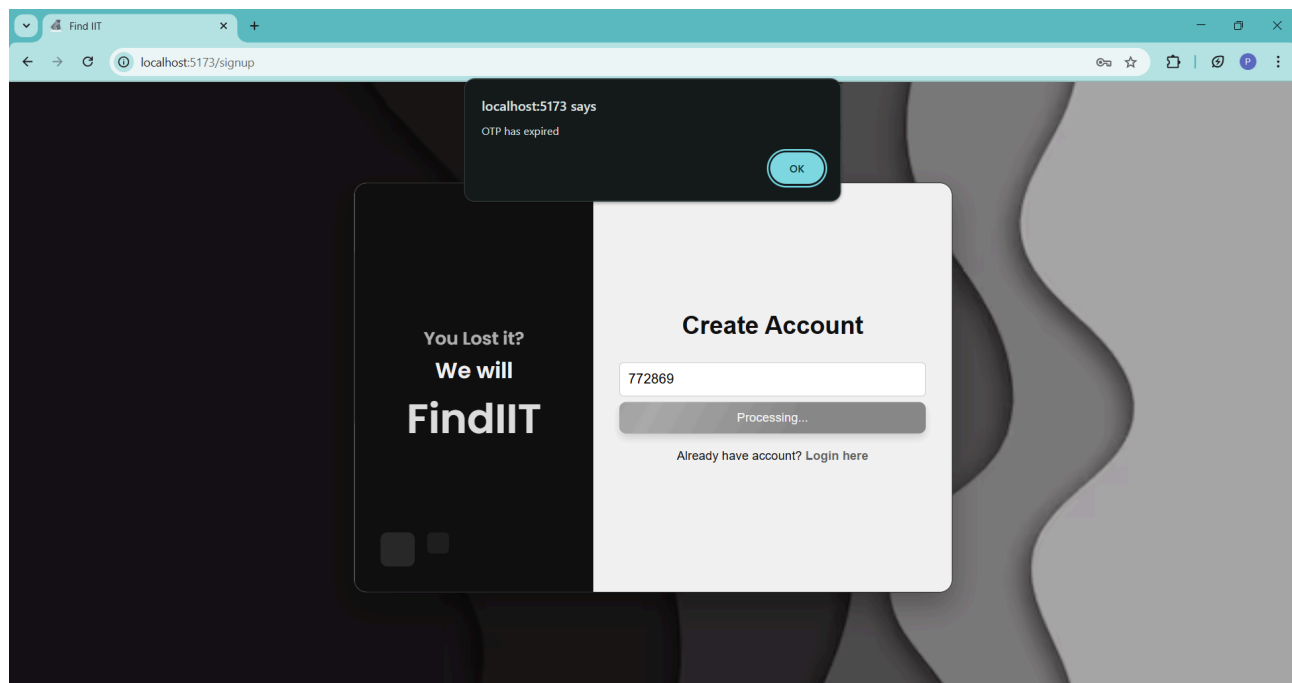
Mail sent for otp verification:



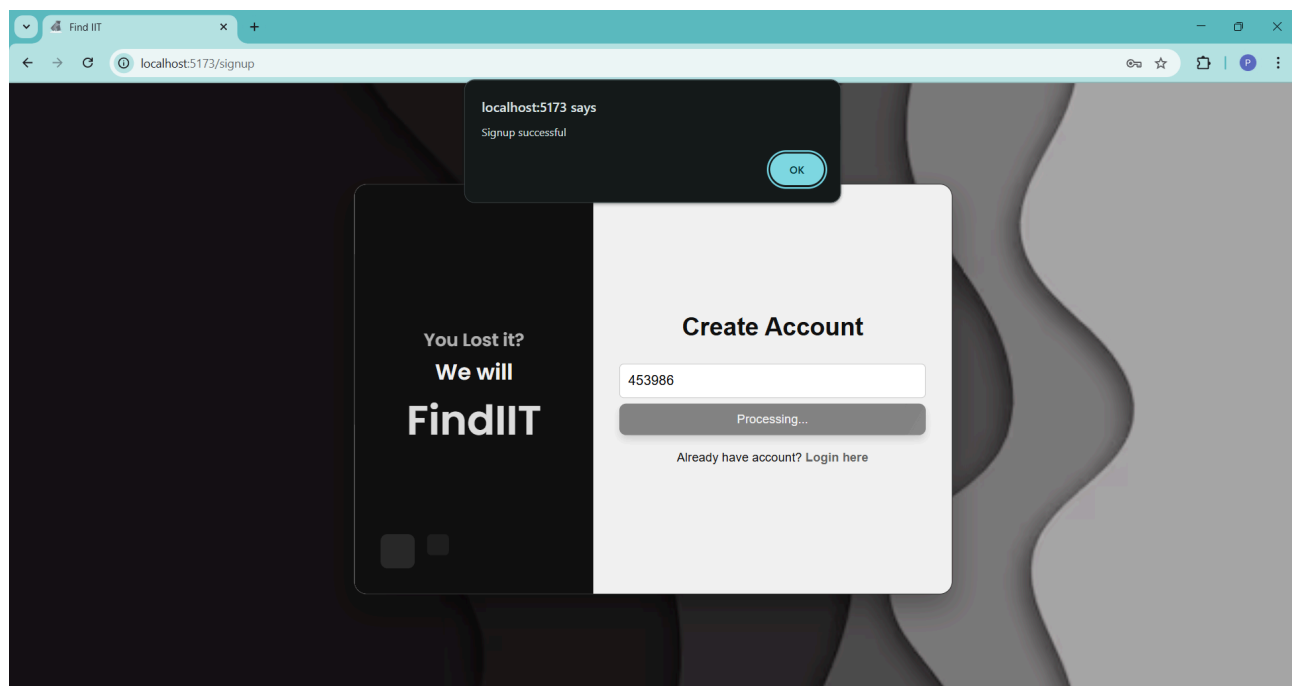
When otp entered does not matches with the sent otp:



When OTP is entered after a delay of 5 minutes (after which OTP is expired):



Upon entering correct details and OTP:



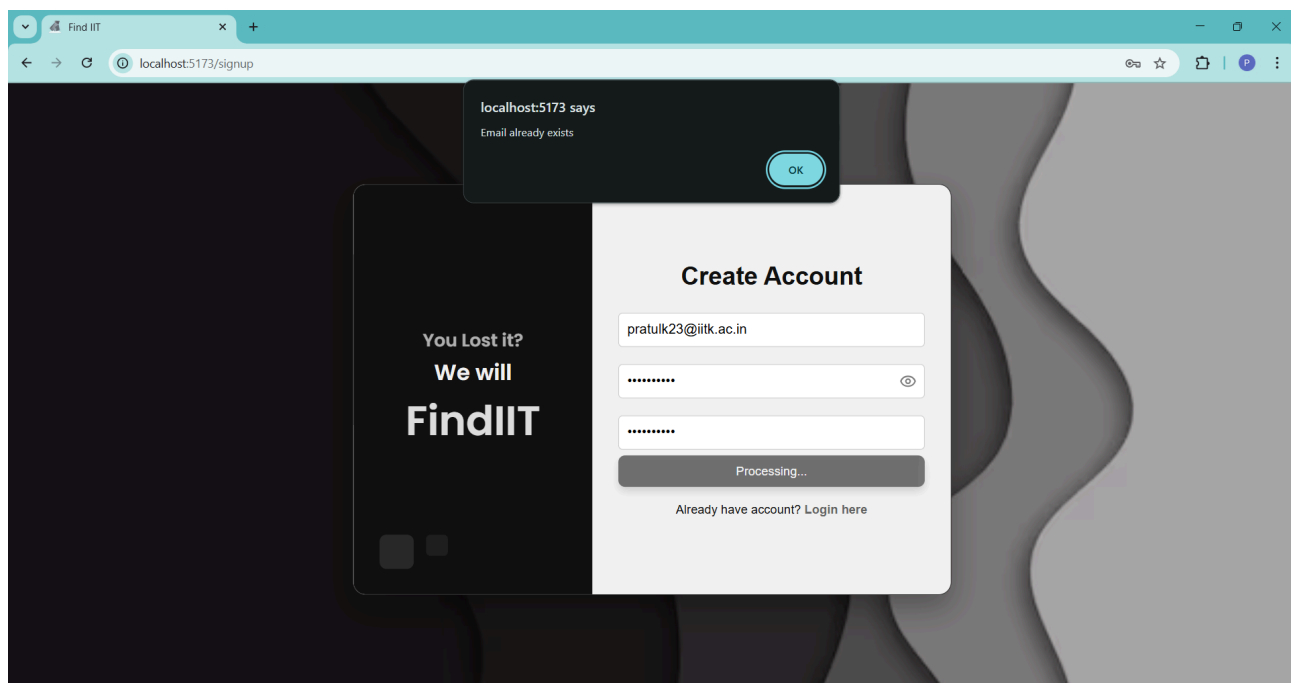
- New User is updated in MongoDB Database -


```
{
  _id: ObjectId('67eec4a7260b162012b3ea03'),
  email: 'pratulk23@iitk.ac.in',
  password: '$2b$10$Pzd23gIln8fkCiccjwuC8.5TdCFYhBkHL4Ww4tYtqdSE26SSRxpvc',
  isVerified: true,
  __v: 0
},
```

Result: Finally when a new user enters everything correctly as described above he/she is successfully registered and ready to login.

TEST #3:

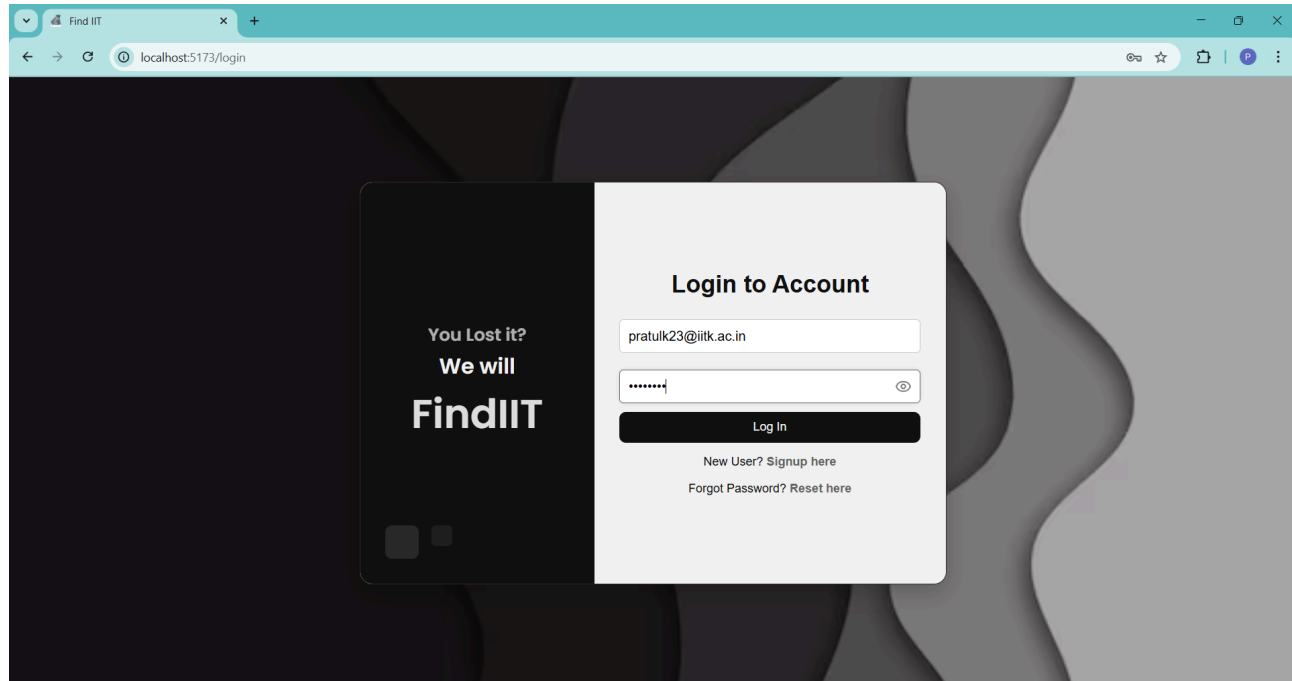
Details: Registering for an email which is already registered on the site.



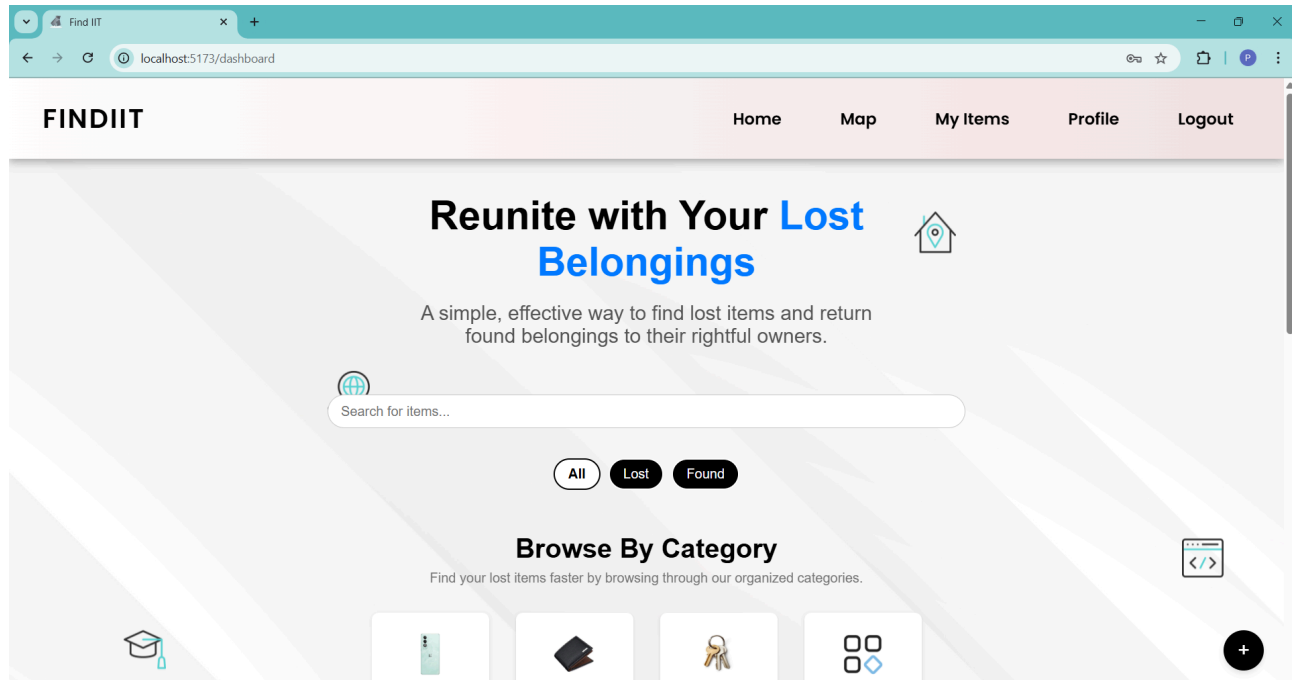
Result : Popup showing “Email already exists”

TEST #4:

Details: Login with right credentials.



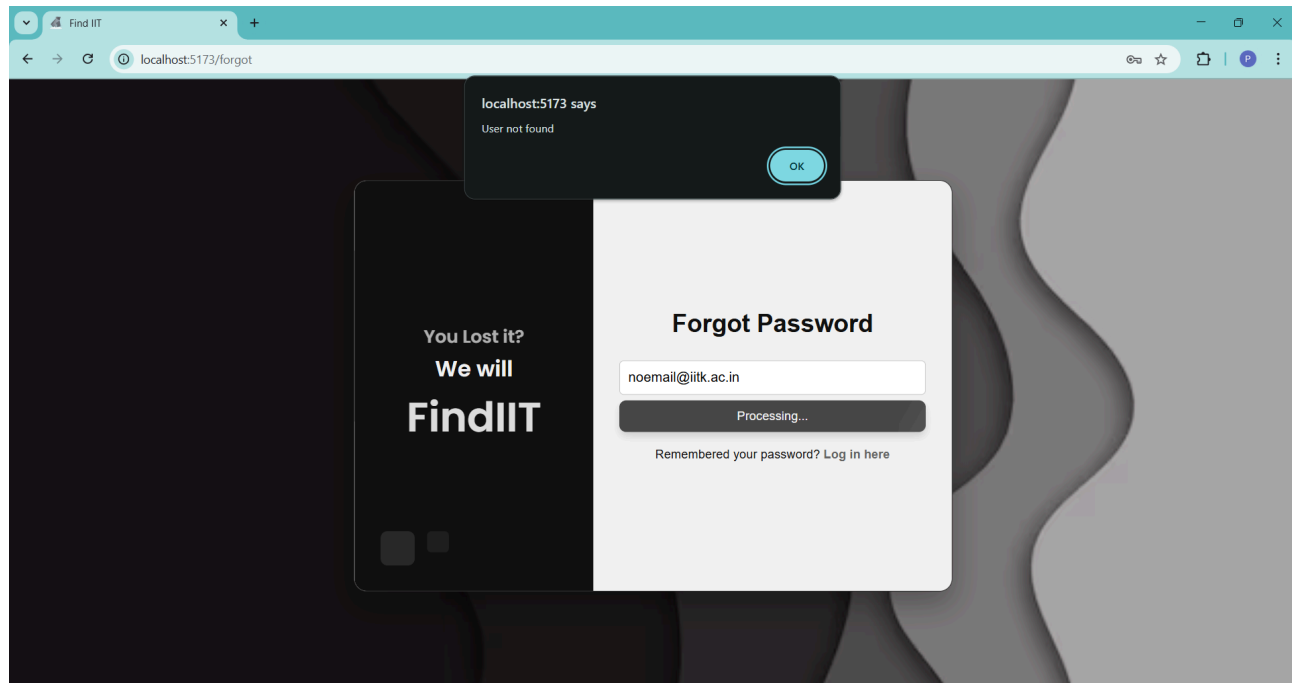
After login user is directed to Dashboard



TEST #5:

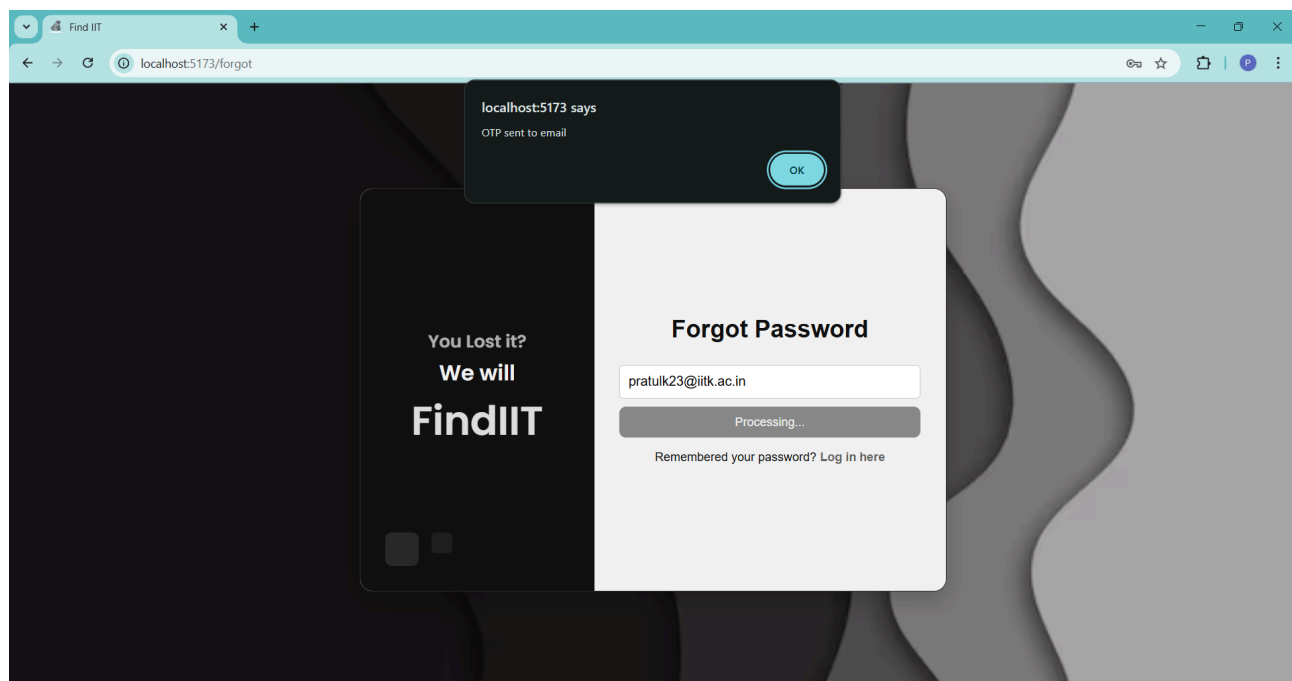
Details: To verify the feature of forgot password and to check if the password is updated correctly.

- On entering an email id for which user does not exist



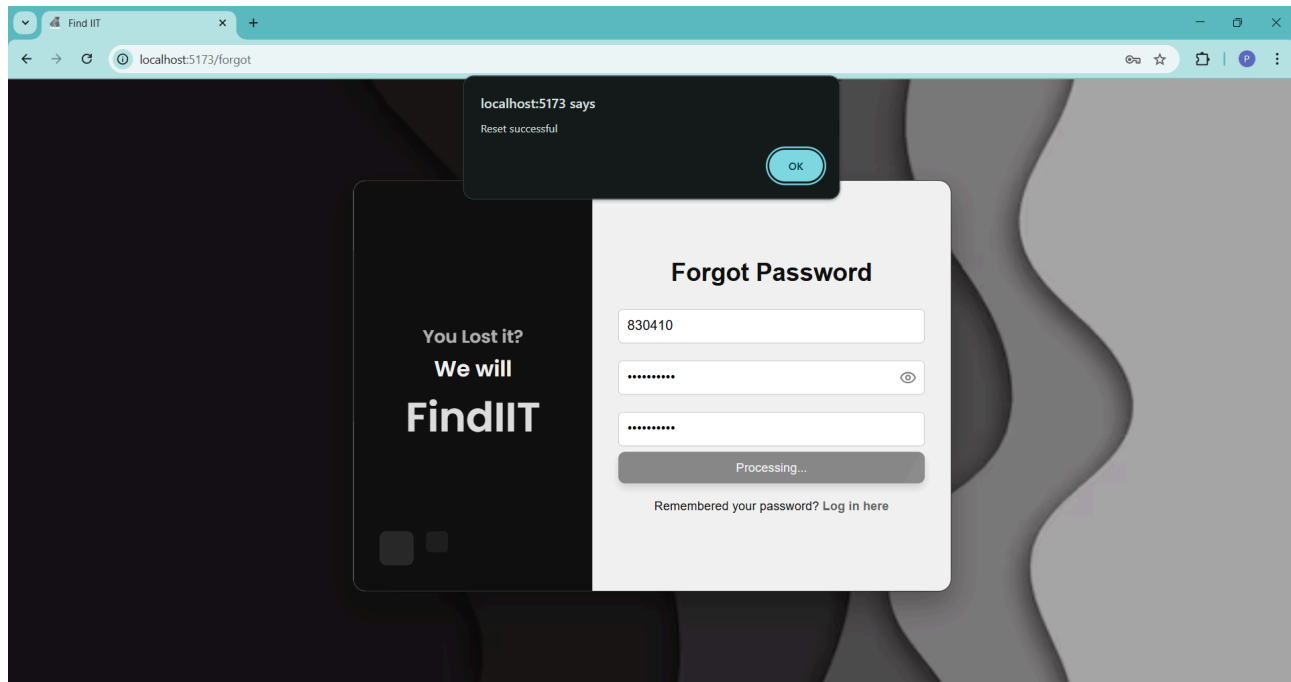
Result: Pop up box which displays "User not found"

On entering registered email-ID otp will be sent to the registered email



Result :OTP sent to the email will be entered and on clicking OK, user will be directed to change your password page.

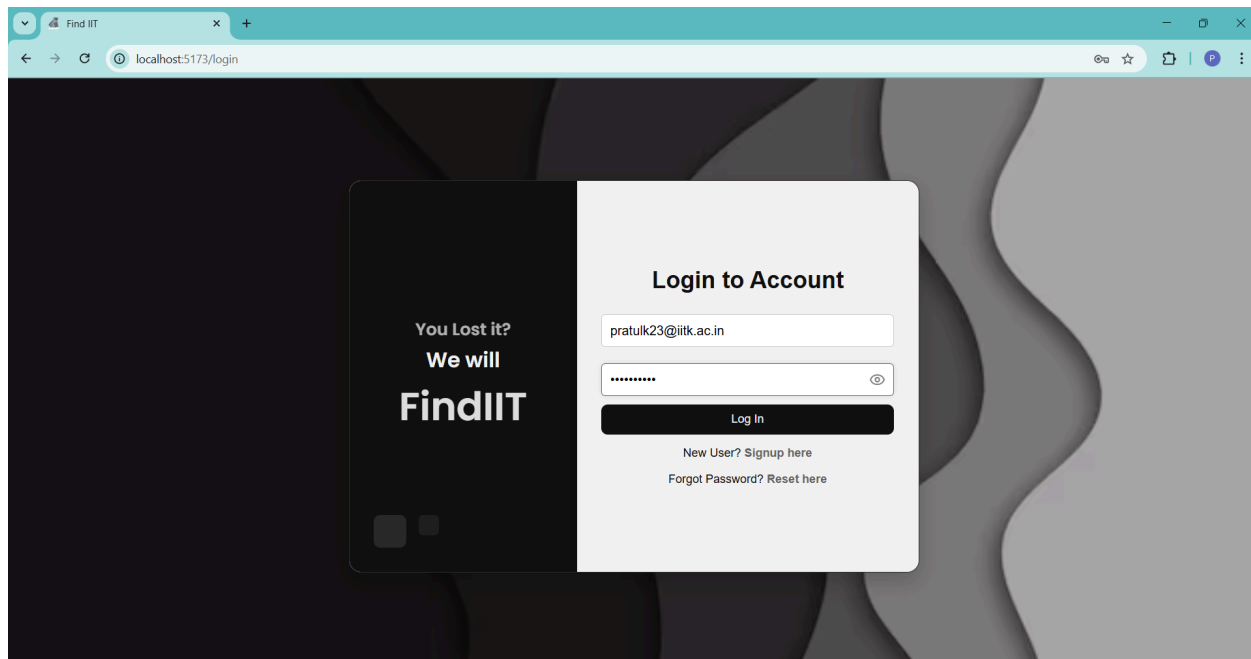
Change your password page:



Result:After Entering new password pop up box which says “Reset successful”

On clicking OK will be directed again to the login page and now the user can login with a new password.

Successful login with new password:



2. Profile Module:

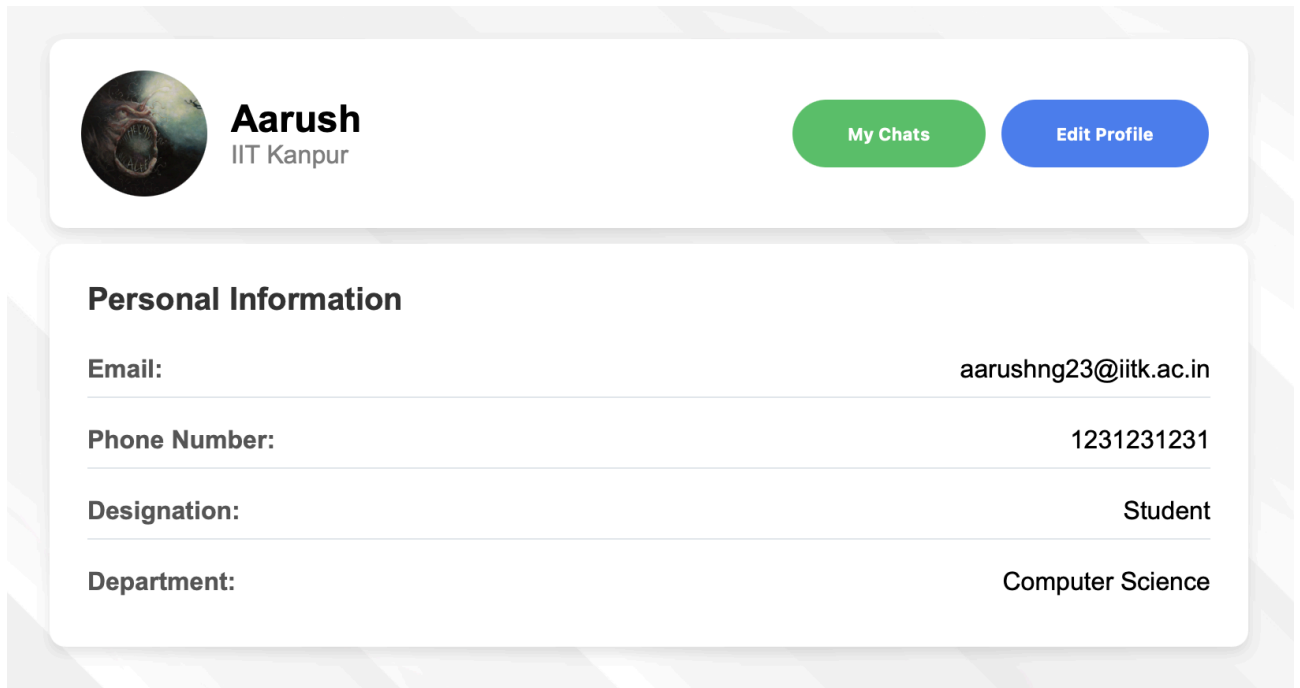
This report provides a comprehensive evaluation of all user functionalities, including profile management and personal information updates. The primary focus was on ensuring the successful integration of backend and frontend components for **Profile View, Edit Profile, and Personal Information Management** (Email, Phone Number, Designation, and Department).

During testing, we verified that users can view and edit their profiles seamlessly, with accurate data storage and retrieval. We also tested edge cases, such as missing or invalid input, to ensure robustness.

Test Cases

Test #1: Viewing Profile Information

- **Details:** Verify whether a user can successfully view their profile details.
- **Test Steps:**
 1. Log in to the application.
 2. Navigate to the **Profile** page.
 3. Check if the user's name, email, phone number, designation, and department are displayed correctly.
 4. Validate that the data is retrieved from the correct backend API (**handled in unit testing**).
- **Expected Output:** The profile should display the correct user details fetched from the database.



 **Aarush**
IIT Kanpur

[My Chats](#) [Edit Profile](#)

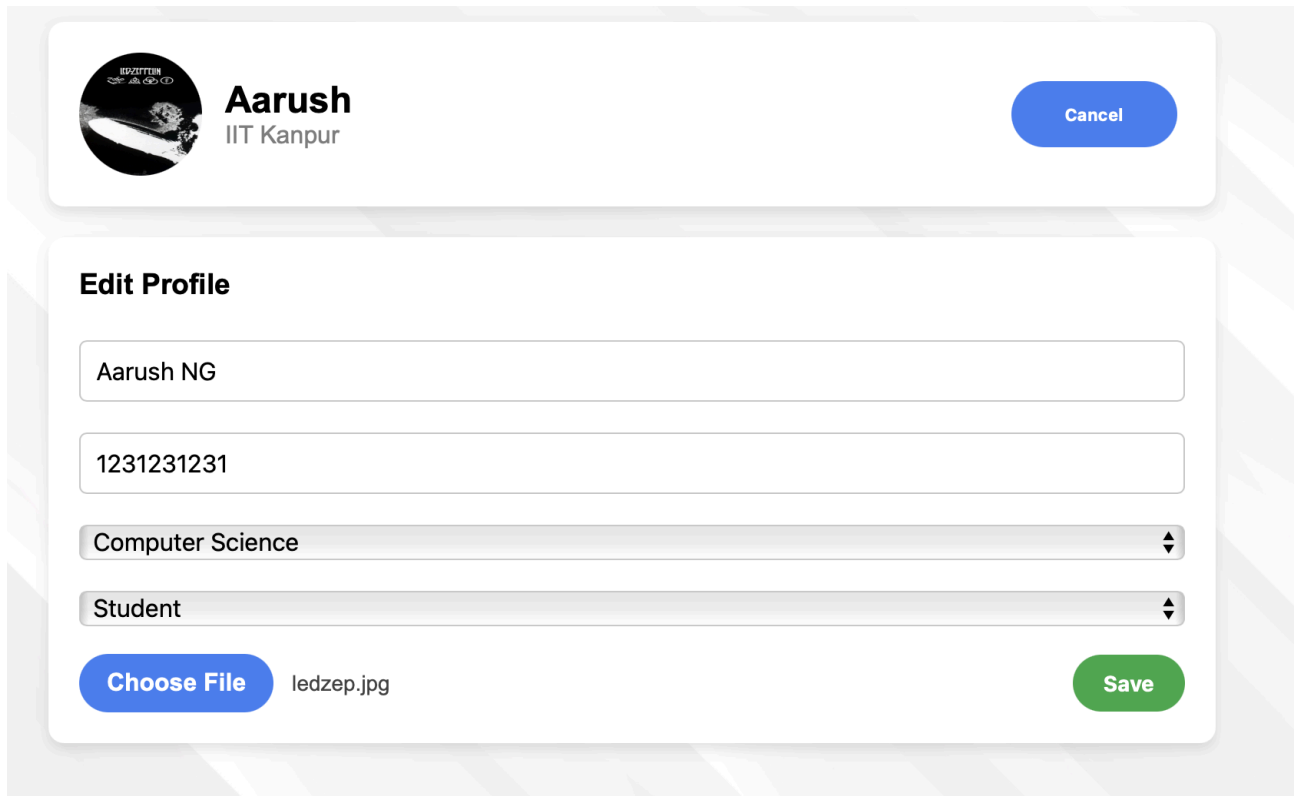
Personal Information

Email:	aarushng23@iitk.ac.in
Phone Number:	1231231231
Designation:	Student
Department:	Computer Science

-
- **Actual Output:** Passed – User details were displayed accurately.

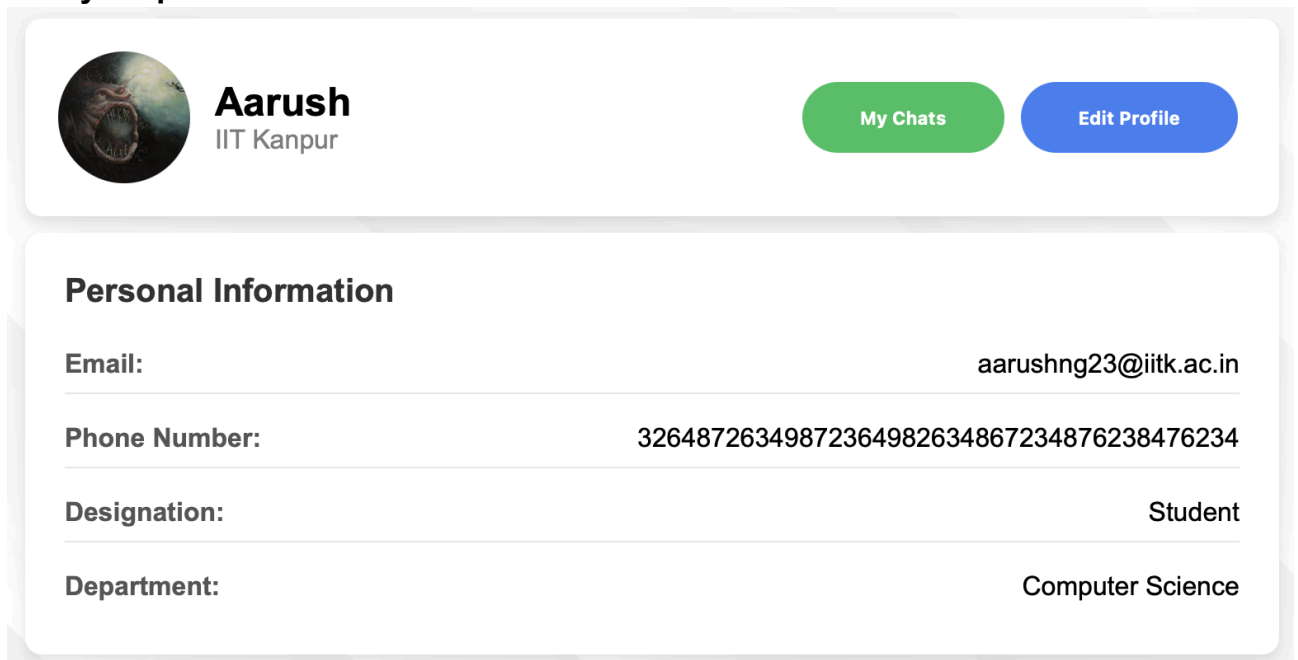
Test #2: Editing Profile Information

- **Details:** Verify that a user can edit their profile and save the changes successfully.
- **Test Steps:**
 1. Navigate to the **Edit Profile** page.
 2. Modify fields such as name, email, phone number, designation, and department.
 3. Click the "Save" button.
 4. Refresh the profile page and verify that the changes persist.
 5. Check the backend request and confirm the database update.
- **Expected Output:** Profile changes should be saved and reflected correctly.



The screenshot shows the 'Edit Profile' form for a user named Aarush from IIT Kanpur. The form includes a profile picture, a 'Cancel' button, and several input fields: a text field for the name 'Aarush NG', a text field for the phone number '1231231231', a dropdown menu for the department 'Computer Science', and another dropdown menu for the designation 'Student'. At the bottom, there is a 'Choose File' button, a preview of the selected file 'ledzep.jpg', and a 'Save' button.

-
- **Actual Output:** Passed with minor exception – Profile updates were successfully stored and displayed but the phone number field wasn't checked for length or region validity.
- **Faulty output:**



The screenshot shows the 'Personal Information' form for the same user, Aarush from IIT Kanpur. The form includes a profile picture, a 'My Chats' button, and an 'Edit Profile' button. Below these, there are four rows of information: 'Email: aarushng23@iitk.ac.in', 'Phone Number: 32648726349872364982634867234876238476234', 'Designation: Student', and 'Department: Computer Science'.

Test #3: Updating Designation and Department

- **Details:** Ensure that users can modify their designation correctly.
- **Test Steps:**
 1. Navigate to **Edit Profile** and change the designation field.
 2. Save the changes and verify that the new designation is reflected in the profile.
 3. Check the database entry to confirm persistence.
- **Expected Output:** The updated designation should be displayed properly.

Aarush
IIT Kanpur

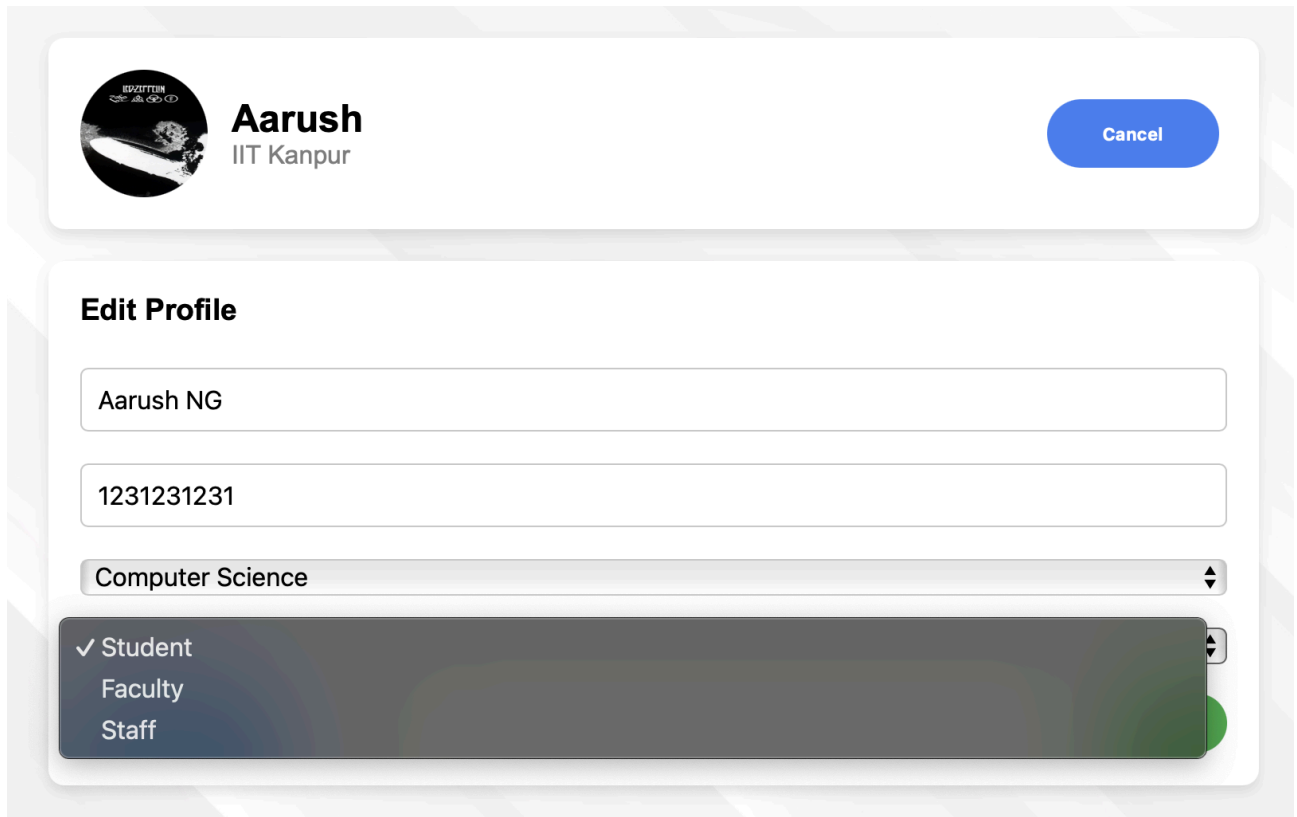
Cancel

Edit Profile

Aarush NG

1231231231

- ✓ Computer Science
- Electrical Engineering
- Statistics and Data Science
- Mathematics and scientific computing
- Mechanical Engineering
- Civil Engineering
- Chemical Engineering
- Aerospace Engineering
- Economics
- Biological Sciences and Biological Engineering
- Chemistry
- Physics
- Earth Sciences
- Other



Aarush
IIT Kanpur

Edit Profile

Aarush NG

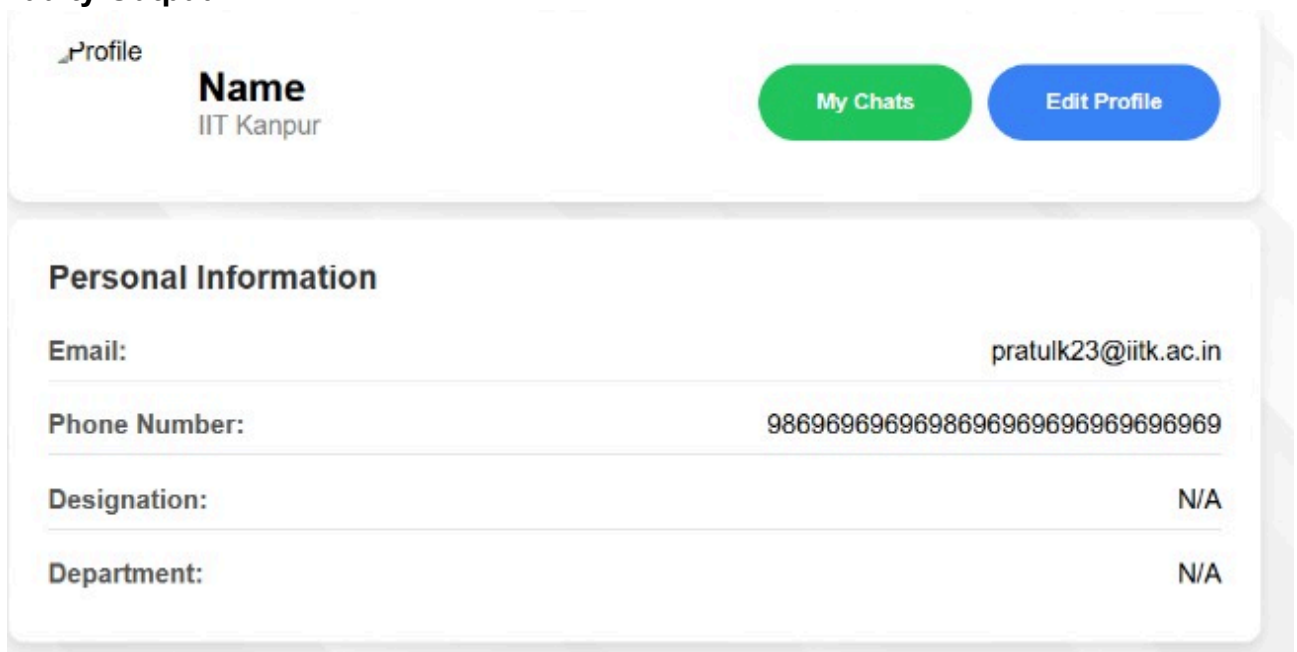
1231231231

Computer Science

- ✓ Student
- Faculty
- Staff

Cancel

-
- **Actual Output:** Passed with minor exception – The designation and department was successfully updated with the minor issue on editing the profile for the first time, where unless department and designation isn;t explicitly selected from the drop down , it shows up as not applicable.
- **Faulty Output:**



Profile

Name
IIT Kanpur

My Chats Edit Profile

Personal Information

Email: pratul23@iitk.ac.in

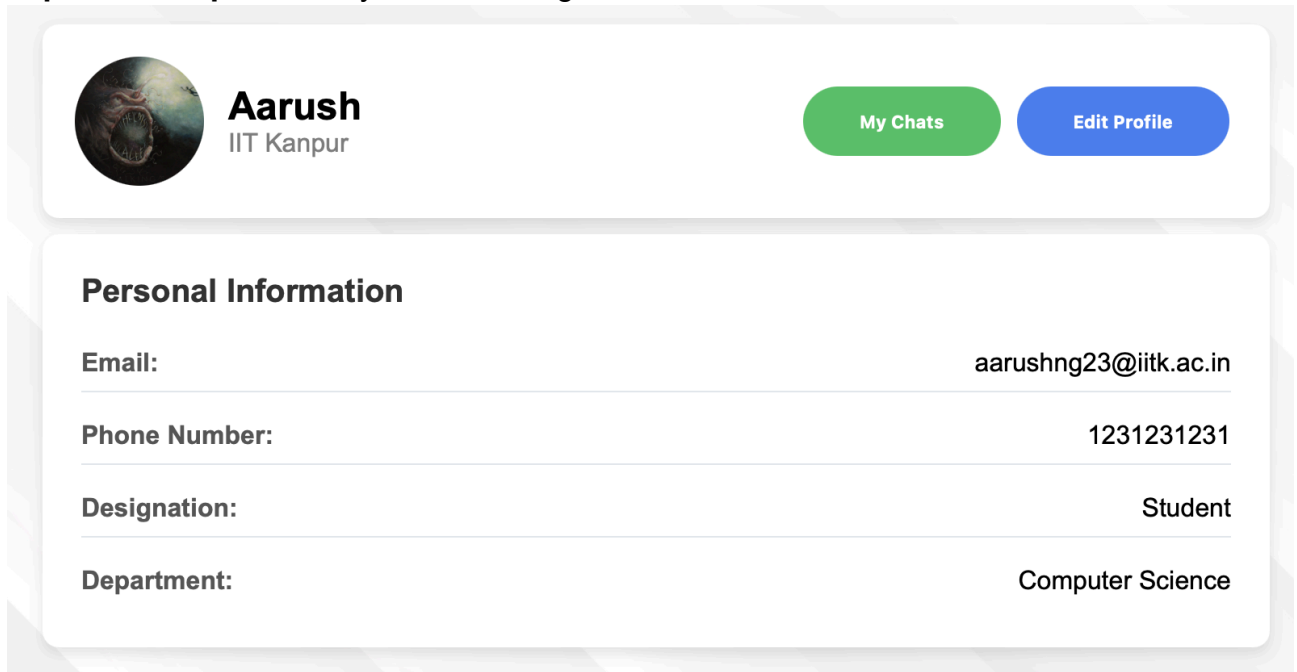
Phone Number: 98696969696969696969696969696969

Designation: N/A

Department: N/A

Test #4: Handling Missing or Empty Fields

- **Details:** Ensure that the system does not allow saving empty required fields.
- **Test Steps:**
 1. Clear all fields in **Edit Profile** and attempt to save.
 2. Verify that it goes back to the last saved/default values.
 3. Ensure that no incorrect data is saved in the database.
- **Expected Output:** The system should go back to its last saved/default state.



 **Aarush**
IIT Kanpur

[My Chats](#) [Edit Profile](#)

Personal Information

Email:	aarushng23@iitk.ac.in
Phone Number:	1231231231
Designation:	Student
Department:	Computer Science

-
- **Actual Output:** Passed – The application successfully rolled back to its last saved/default state.

Conclusion

This test suite successfully validated all core user functionalities related to profile management. The system correctly retrieved, updated, and displayed user information without errors. The **manual testing approaches** thorough validation, given the limited number of failure points.

Key Findings:

1. **Data Persistence:** All changes were accurately stored and retrieved.
2. **User Experience:** Profile modifications were intuitive and smooth.
3. **Security & Integrity:** The system prevented unauthorized changes and enforced input constraints.

Overall, the testing process confirmed that the **Profile Management Module** is **functionally complete, user-friendly, and secure**. Future improvements can include adding **automated validation tests** and **independent testing** for further robustness.

3. Chat Option Testing Module Report

Overview

This test module evaluates the chat functionality within the application. The chat option is designed to:

- Allow users to access the chat page by clicking on **"My Chats"** from their profile.
- Display a list of previous chats with contacts.
- Provide a **"Start Chat"** button that, when clicked, requires the user to enter an email address to initiate a new chat.

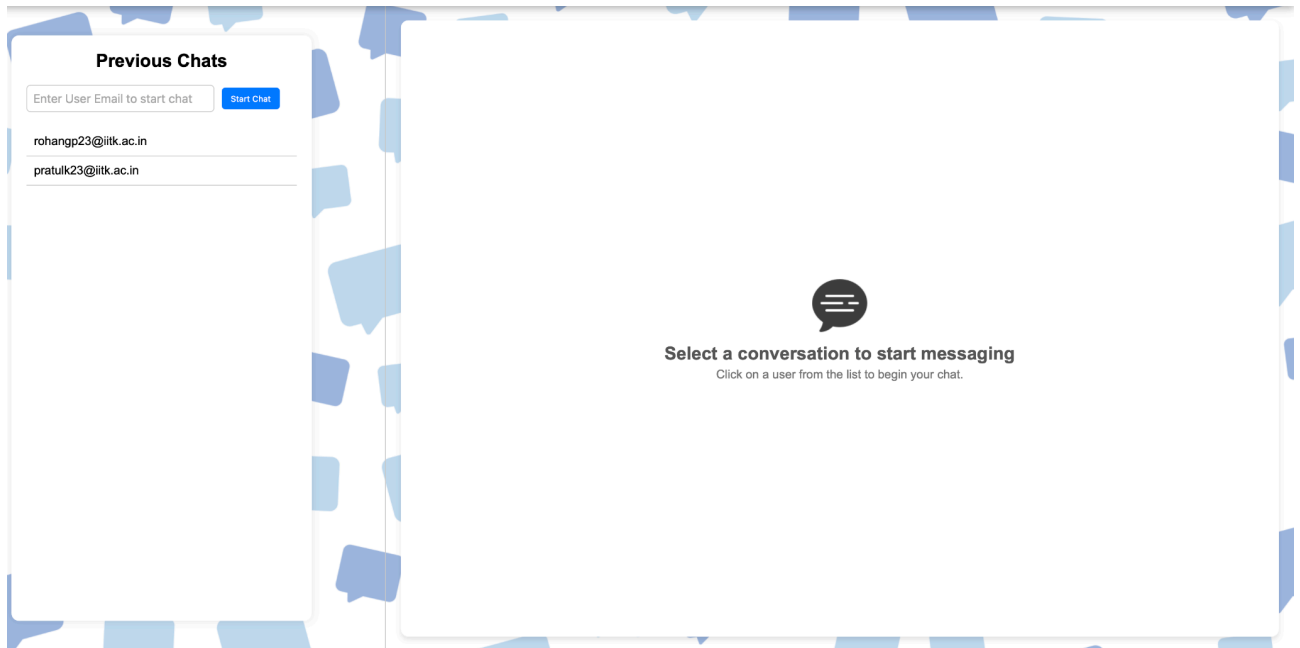
The tests validate that:

- The chat page loads correctly.
- The list of previous chats is accurate.
- New chats are initiated properly.
- Error handling is in place for invalid or non-existent email inputs.

Test Cases

Test #1: Access Chat Page via Profile

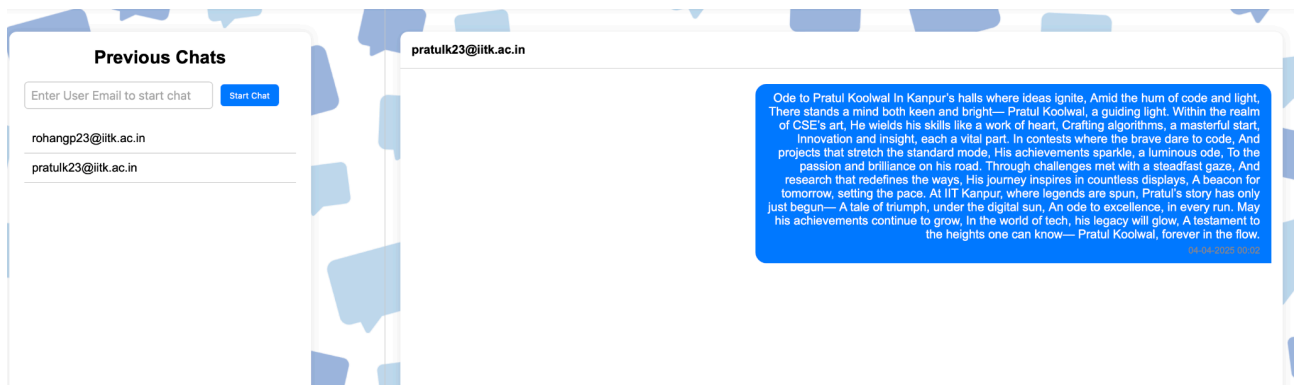
- **Details:** Verify that clicking the "My Chats" button on the profile page correctly redirects the user to the chat page.
- **Test Steps:**
 1. Log in to the application.
 2. Navigate to the Profile page.
 3. Click on the **"My Chats"** button.
 4. Confirm that the user is redirected to the chat page.
- **Expected Output:** The chat page should load, displaying the list of previous chats and a **"Start Chat"** button.



-
- **Actual Output:** Passed – The chat page was accessed successfully from the profile.

Test #2: Verify Previous Chats List

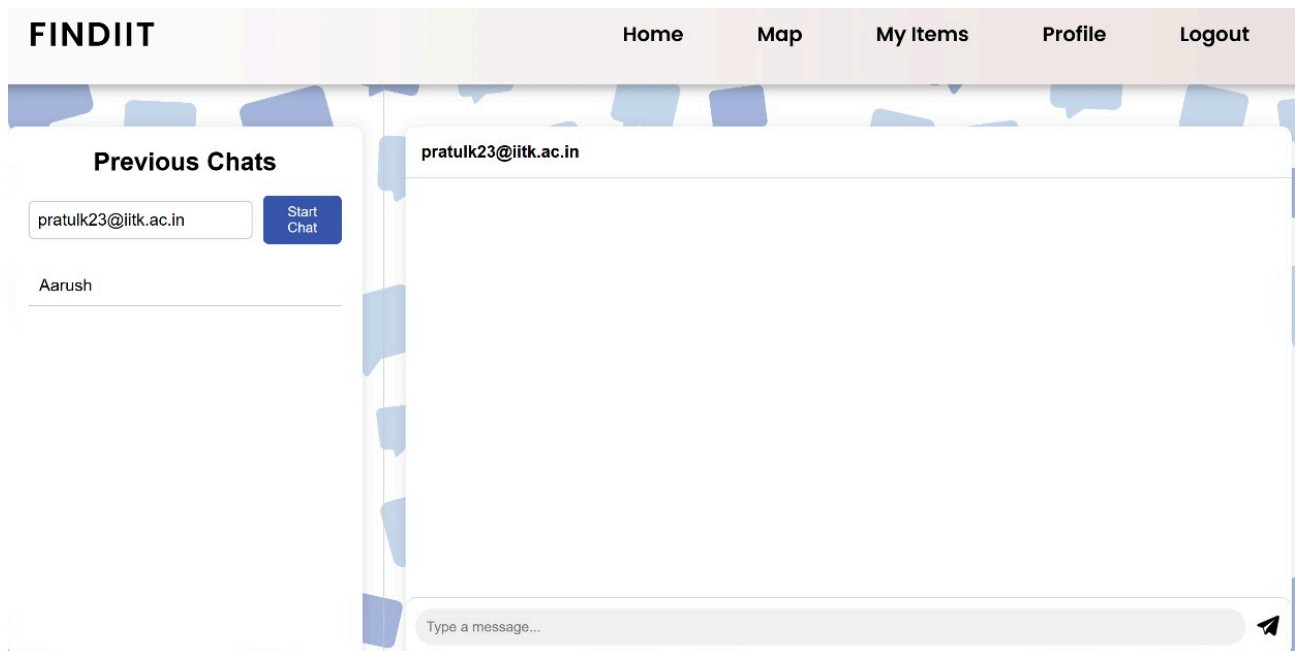
- **Details:** Validate that the chat page displays a correct list of users with whom the current user has had previous chats.
- **Test Steps:**
 1. On the chat page, inspect the list of previous chats.
 2. Compare the displayed contacts with expected chat history (e.g., based on test data).
- **Expected Output:** The list should accurately display all contacts with previous conversations with email addresses.



-
- **Actual Output:** Passed – All previous chat contacts were displayed as expected.

Test #3: Start a New Chat with a Valid Email

- **Details:** Ensure that a user can successfully initiate a new chat by entering a valid recipient's email address.
- **Test Steps:**
 1. Enter a valid email address (e.g., `pratulk23@iitk.ac.in`) in the provided input field.
 2. Click the **"Start Chat"** button to initiate the chat.
 3. Verify that the system redirects to a new chat conversation page.
- **Expected Output:** A new chat conversation should be initiated, and the chat interface should load for the entered email.

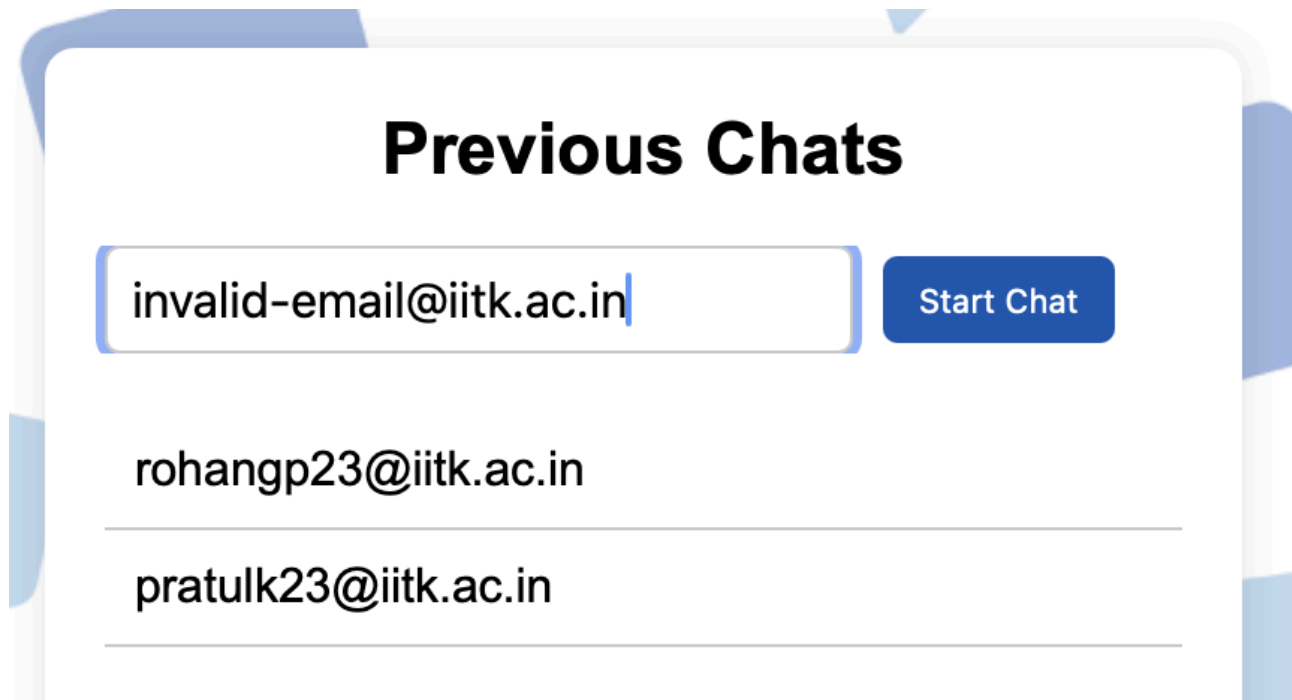


- **Actual Output:** Passed – A new chat was successfully initiated with the provided email address.

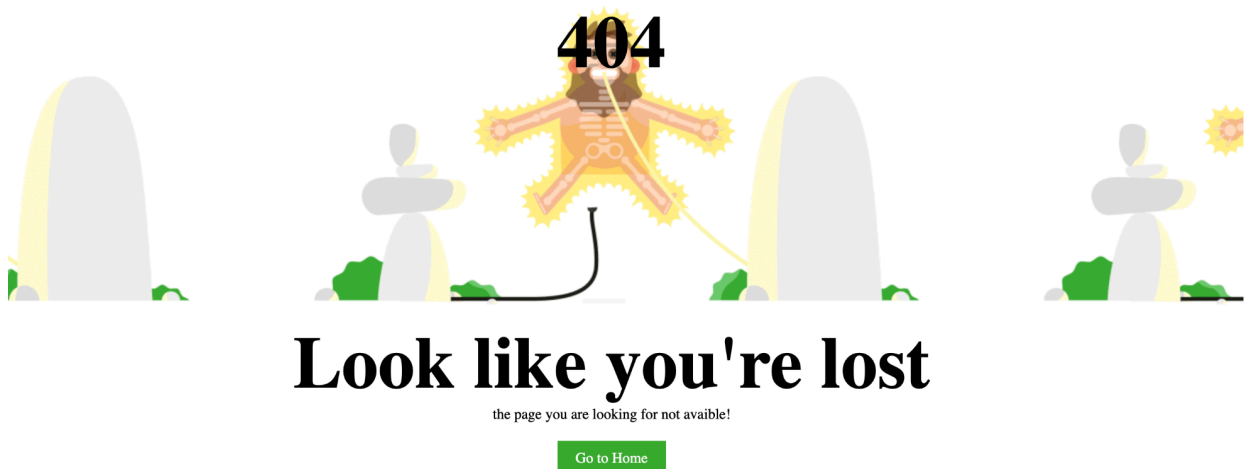
Test #4: Error Handling for Invalid Email

- **Details:** Check that the system handles cases where the user enters an invalid email (non existent user) when attempting to start a new chat.
- **Test Steps:**
 1. Click the **"Start Chat"** button on the chat page.

2. Enter an invalid email format (`invalid-email@iitk.ac.in`) in the input field.
 3. Attempt to start the chat.
- **Expected Output:** The system should display an error message indicating that the user doesn't exist, and the chat should not be initiated.



- redirects to :



-
- **Actual Output:** Passed with minor exception – instead of an error message being displayed, the website redirected immediately to the 404 page and the chat was not initiated.

Test #5: UI/UX Consistency on the Chat Page

- **Details:** Verify that the chat page layout is consistent, intuitive, and responsive across different devices.
 - **Test Steps:**
 1. Inspect the chat page layout, including the placement of the previous chats list and the **"Start Chat"** button.
 2. Verify responsiveness by simulating different screen sizes.
 3. Check that navigation elements (e.g., back button, header) function as intended.
 - **Expected Output:** The chat page should maintain a consistent and user-friendly layout, and all navigation components should function correctly.
 - **Actual Output:** Passed – The chat page layout was consistent and responsive across tested screen sizes.
-

Conclusion

The chat functionality was thoroughly tested to ensure that:

- Users can access the chat page from their profile.
- The previous chat contacts list is accurate.
- New chats are initiated correctly when a valid email is provided.
- Appropriate error handling is in place for invalid or non-existent emails.
- The user interface is consistent and responsive.

All test cases were executed successfully, and the system performed as expected with **no major issues** observed during testing. Future enhancements may include automated testing for regression purposes and further UI/UX refinements based on user feedback.

This detailed test module provides comprehensive coverage of the chat option and verifies that all key components work seamlessly to provide a smooth and reliable user experience.

Lost and Found Items Testing Module Report

Test owner : Pratul Koolwal

Test Date : 04/04/2025

Overview

This test module evaluates the core functionalities of the FindIIT website. The system is designed to:

- Allow users to report lost and found items.
- Enable users to search for reported items.
- Provide a secure authentication system for reporting and claiming items.
- Enables users to see all their reported items at one place.
- Allows users to delete items in case they want to.

The tests validate that:

- The reporting and searching features function correctly.
- Items are stored and retrieved accurately from the database.
- Authentication and authorization mechanisms work properly.
- Users can successfully delete any previously reported item.

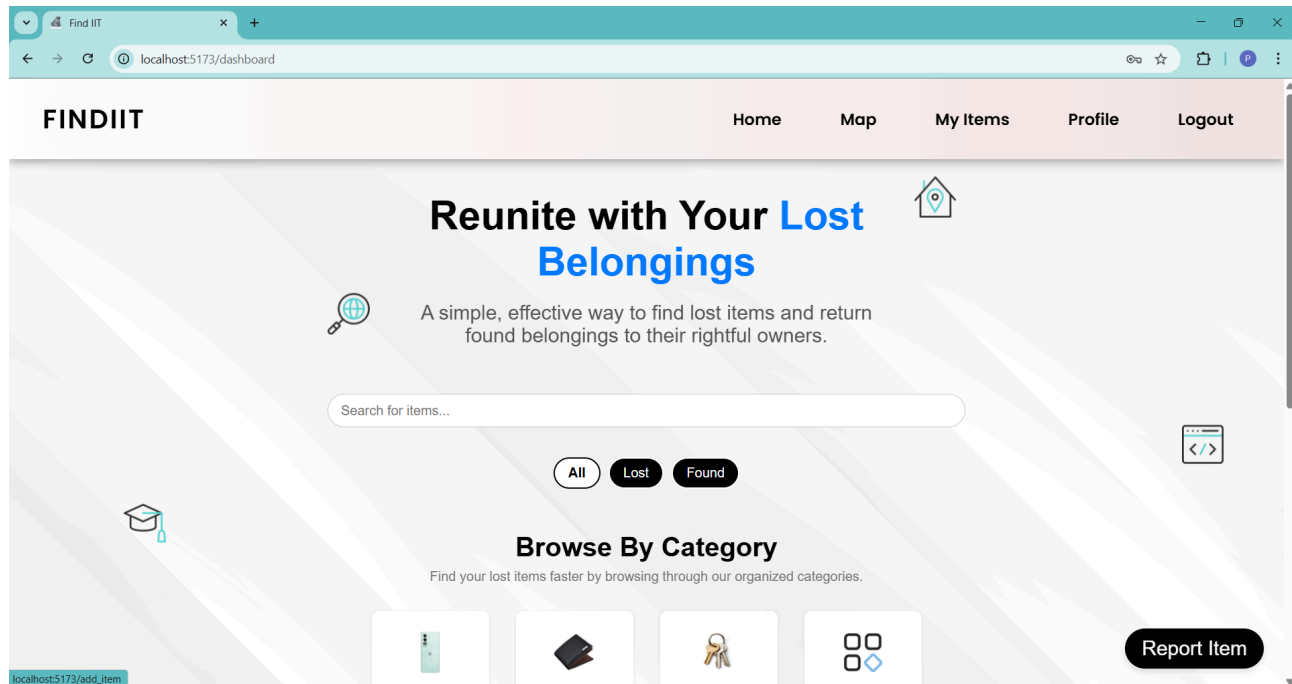
Test Cases

Test #1: Reporting a Lost or Found Item

Details: Verify that a user can successfully report a lost item and that the item is stored in the database.

Test Steps:

1. Log in to the application.
2. Navigate to the "Report Lost Item" page.



3. Fill in the required details (item name, description, date lost, location, and contact information).

Find IIT

localhost:5173/add_item

Home

Map

My Items

Profile

Logout

Report a Product

Product Status
☒ Lost Product ☐ Found Product

Select Product Category

Phone

Product Description

iPhone s24 Ultra

Add tags (separate by commas)

Black,transparent_case

Location or Gate

L-20, Academic area

The screenshot shows a web browser window with the URL `localhost:5173/add_item`. The page has a header with the logo "FINDIIT" and navigation links: "Home", "Map", "My Items", "Profile", and "Logout". The main content area is a form for adding a new item. It features a map of IIT Kanpur with a red pin at "LHC 18, 19 and 20". A pop-up window shows the location details: "G66M+9M3, Indian Institute of Technology Kanpur, Kalyanpur, Kanpur, Uttar Pradesh 208016, India". Below the map, the form includes a "Contact Details" section with a text input field containing "9876543210", an "Image URL" section with a "Choose file" button and a text input field containing "phone.jpeg", and a "Submit Report" button.

FINDIIT Home Map My Items Profile Logout

Lecture Hall 7
Lecture Hall mplex, IIT Kanpur
Lecture Hall mplex, IIT Kanpur

Southern Labs

LHC 18, 19 and 20

G66M+9M3, Indian Institute of Technology Kanpur
Kalyanpur
Kanpur, Uttar Pradesh 208016
India
[View on Google Maps](#)

Main Dr
Main Dr

Selected coordinates: 26.510725394859804, 80.23440957069397

Contact Details

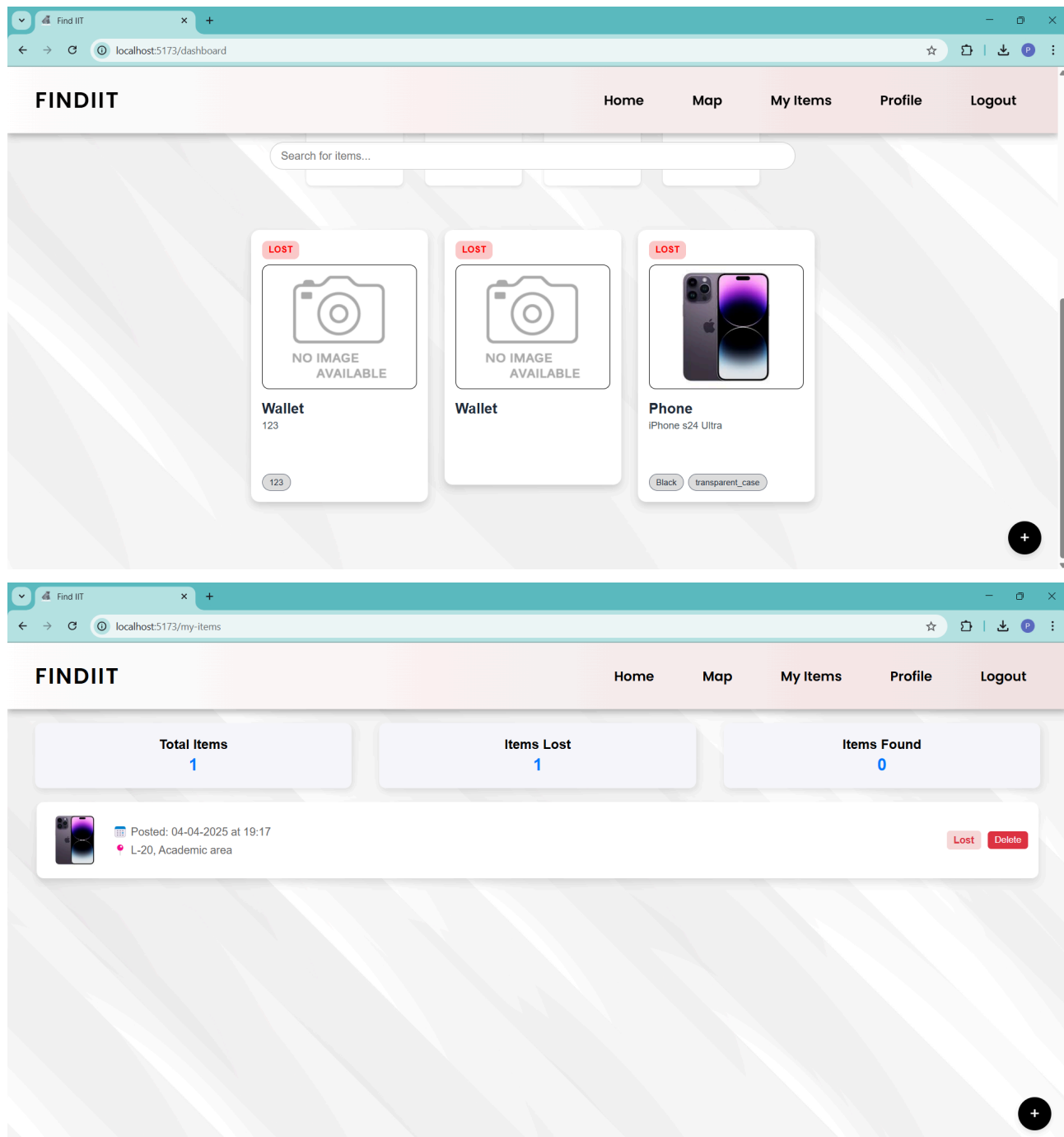
9876543210

Image URL

Choose file phone.jpeg

Submit Report

4. Click the "Submit Report" button.
5. The item appears on the home page, as well as the my items page.



Expected Output: The lost item should be successfully saved and displayed at the relevant places on the website.

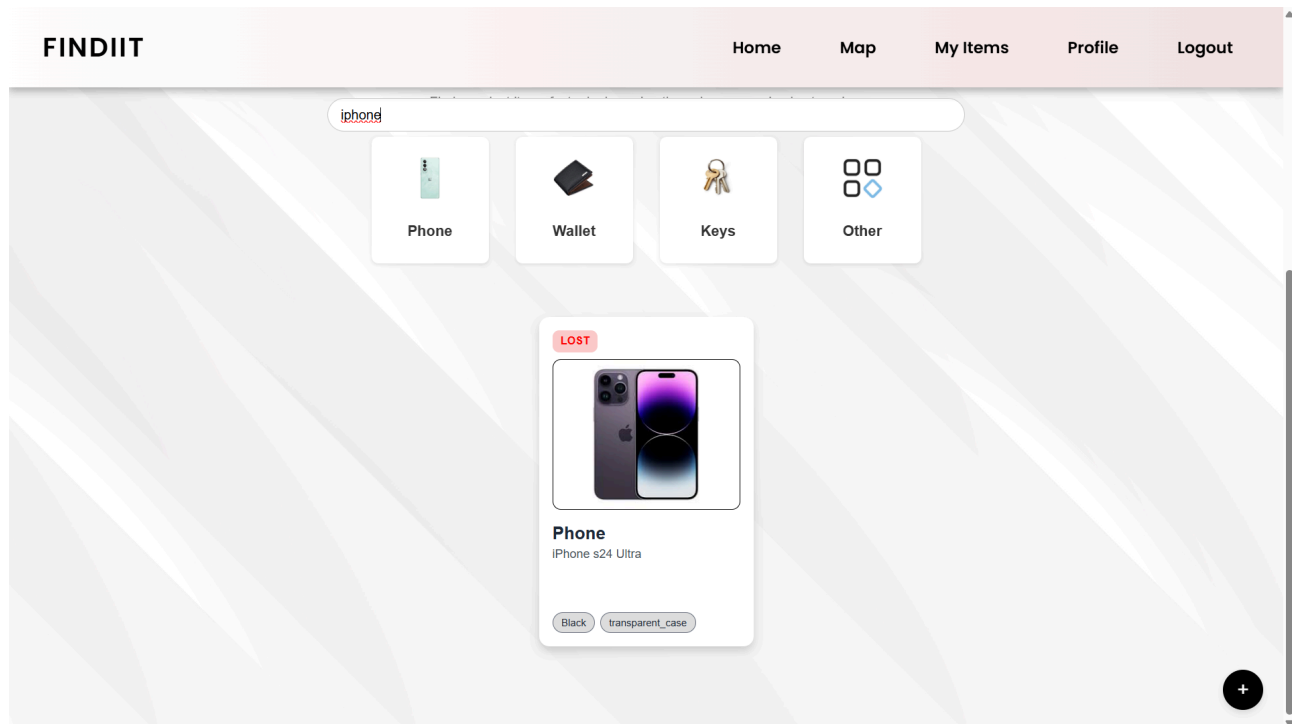
Actual Output: ☒ Passed – The item was stored and retrieved successfully.

Test #2: Searching for a Lost Item

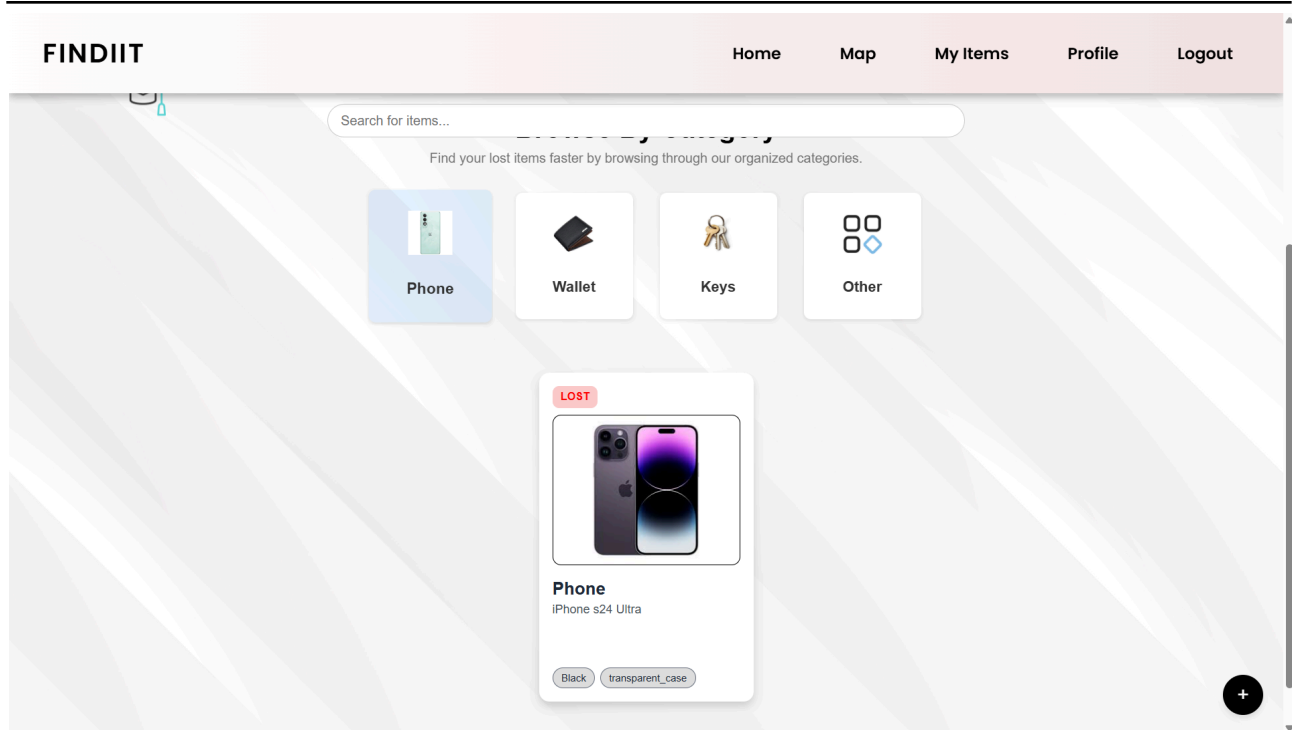
Details: Validate that users can search for lost items based on keywords and filters.

Test Steps:

1. Navigate to the Homepage.
2. User can proceed in different ways -
 - a. Enter a keyword (e.g., "phone") in the search bar.



- b. Browse by category



3. Relevant lost and found items appear in the results.

Expected Output: The system should return all matching items based on the entered criteria.

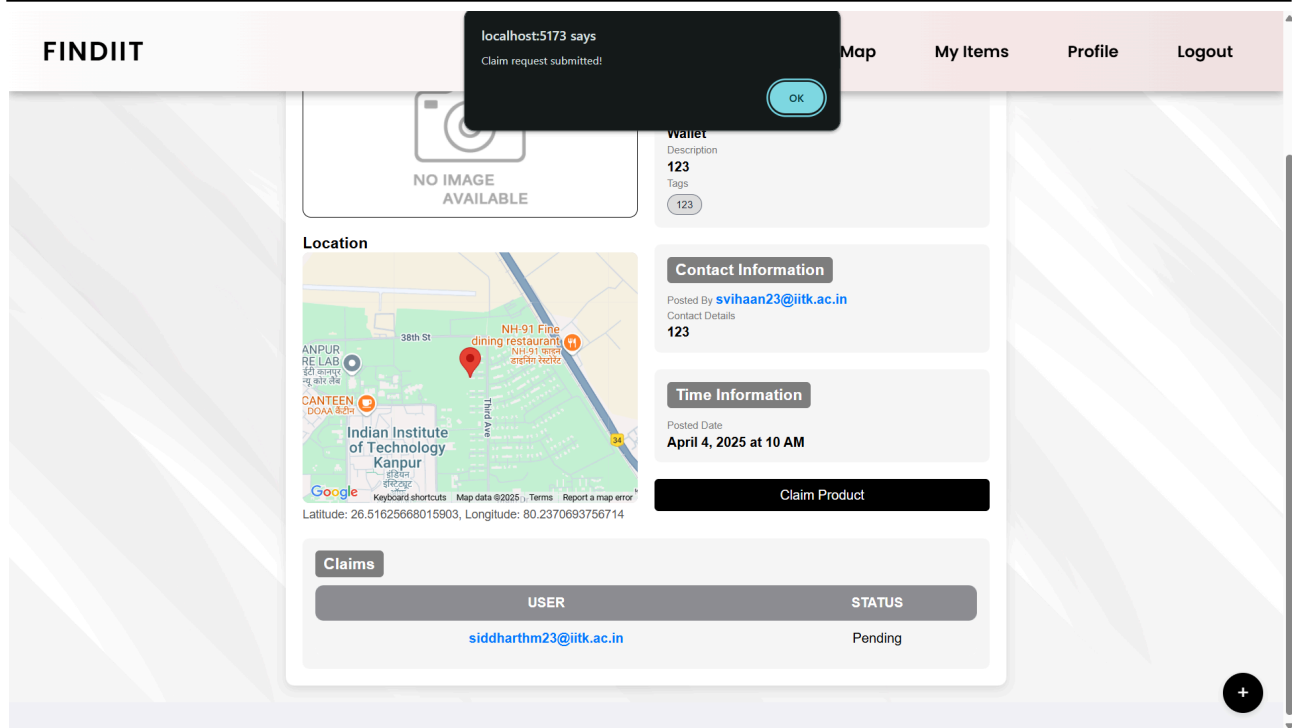
Actual Output: ☒ Passed – Relevant results were displayed accurately.

Test #3: Claiming a Found Item

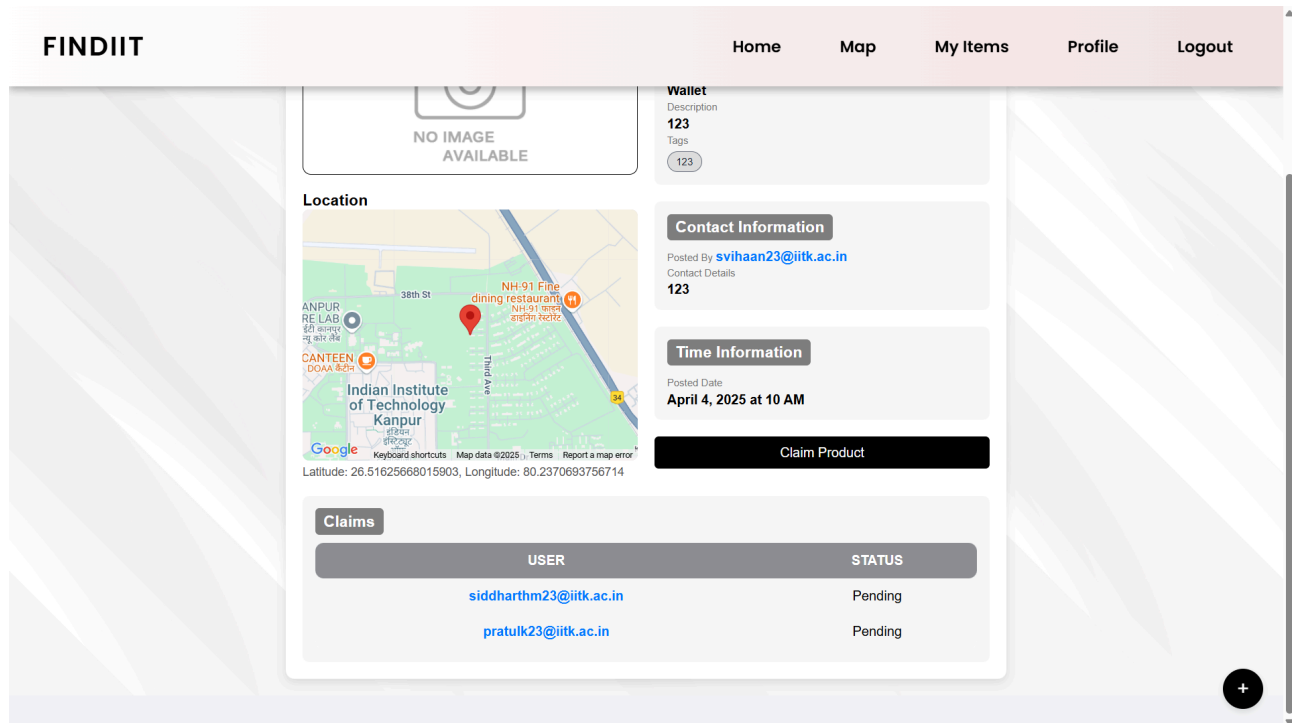
Details: Ensure that a user can successfully claim an item reported as found.

Test Steps:

1. Locate a reported item that matches a lost item.
2. Click the "Claim Product" button.



3. An alert appears "Claim Request Submitted".
4. all users will be able to see your claim request.
5. The user who initially posted that found query will be able to accept or reject this claim request.

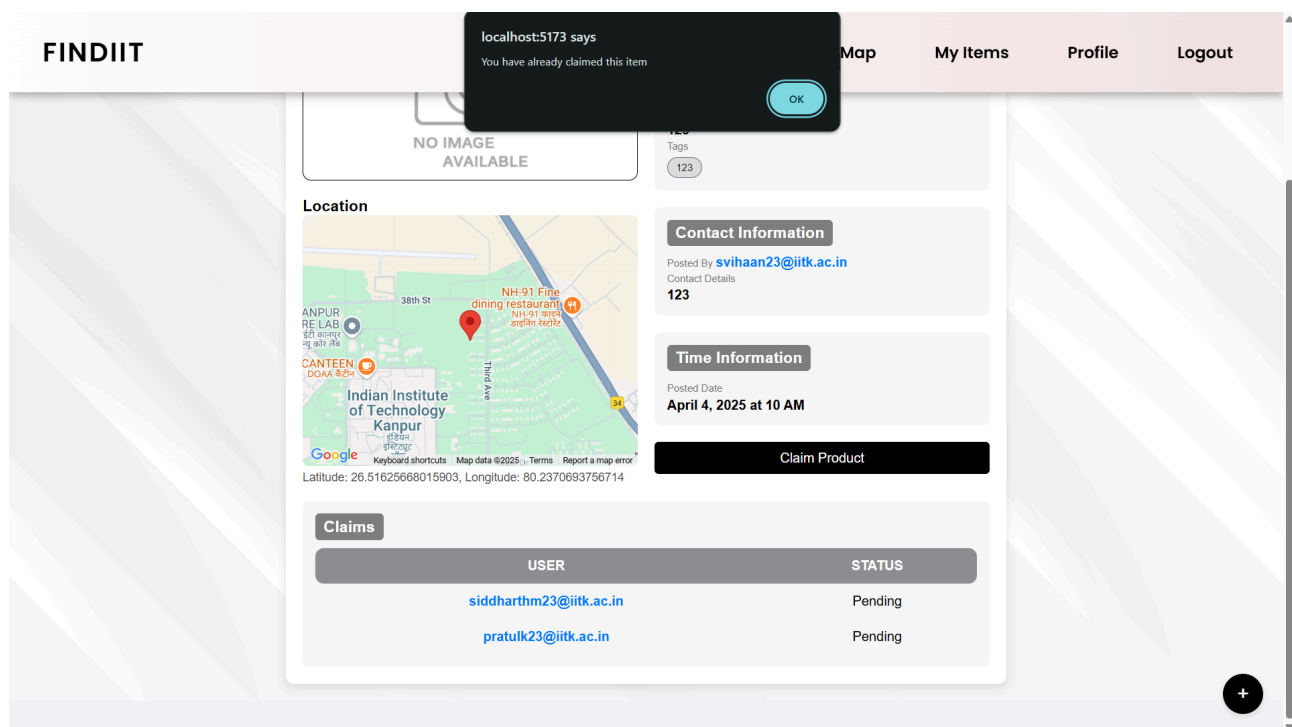


Expected Output: If the verification is successful, the user should receive a confirmation message via popup alert.

Actual Output: ☒ Passed – Claim request submitted successfully, and status updated upon approval.

Test #4: Claiming an item already claimed by same user

Expected Output: The user should receive an alert saying “You have already claimed this item”



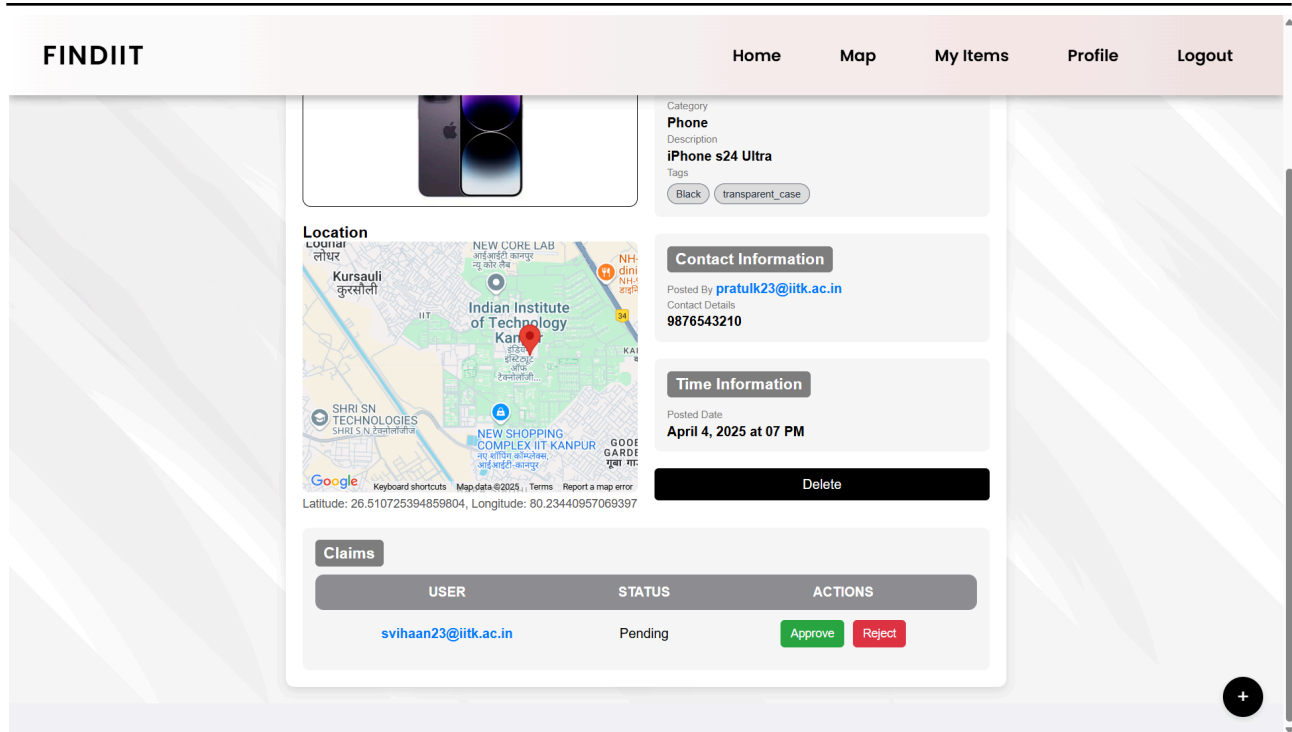
Actual Output: ☒ Passed – Alert displayed and no change was made to the product page.

Test #5: Accepting or rejecting claims made for items reported by user

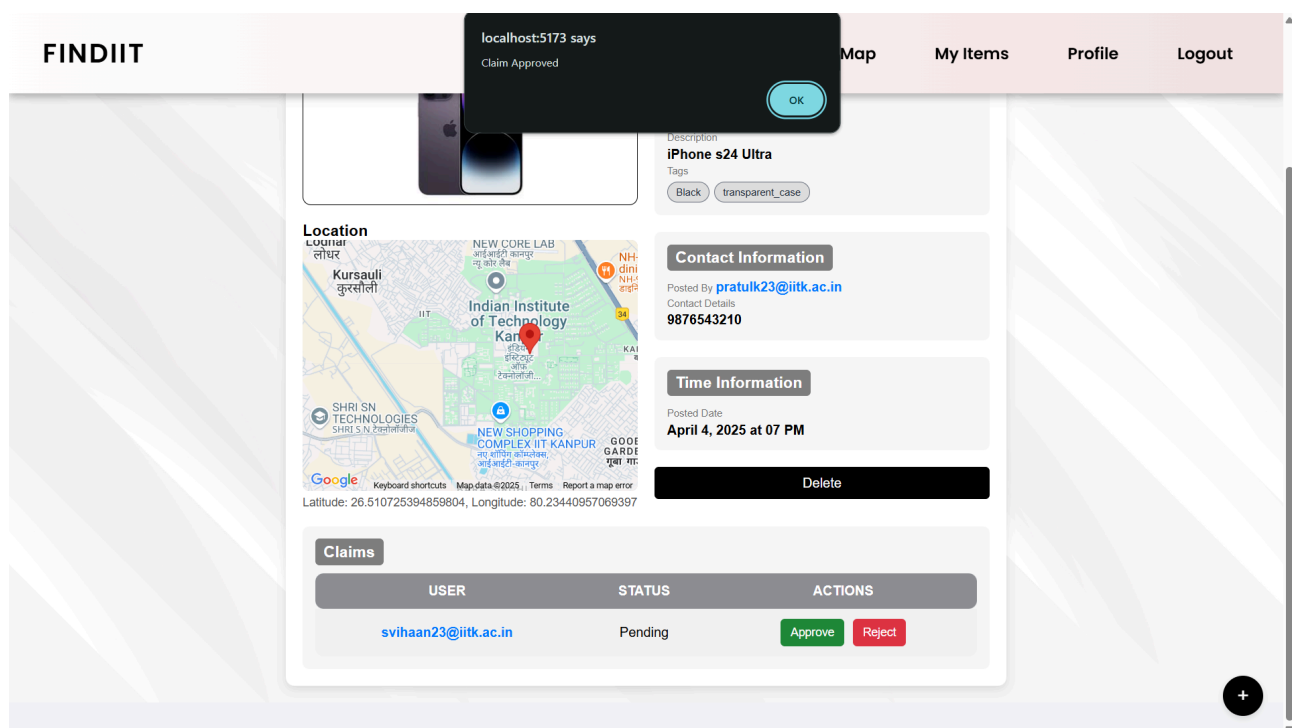
Details: Verify that users can accept or reject claims made by other users.

Test Steps:

1. Go to My Items page and open the concerned query.

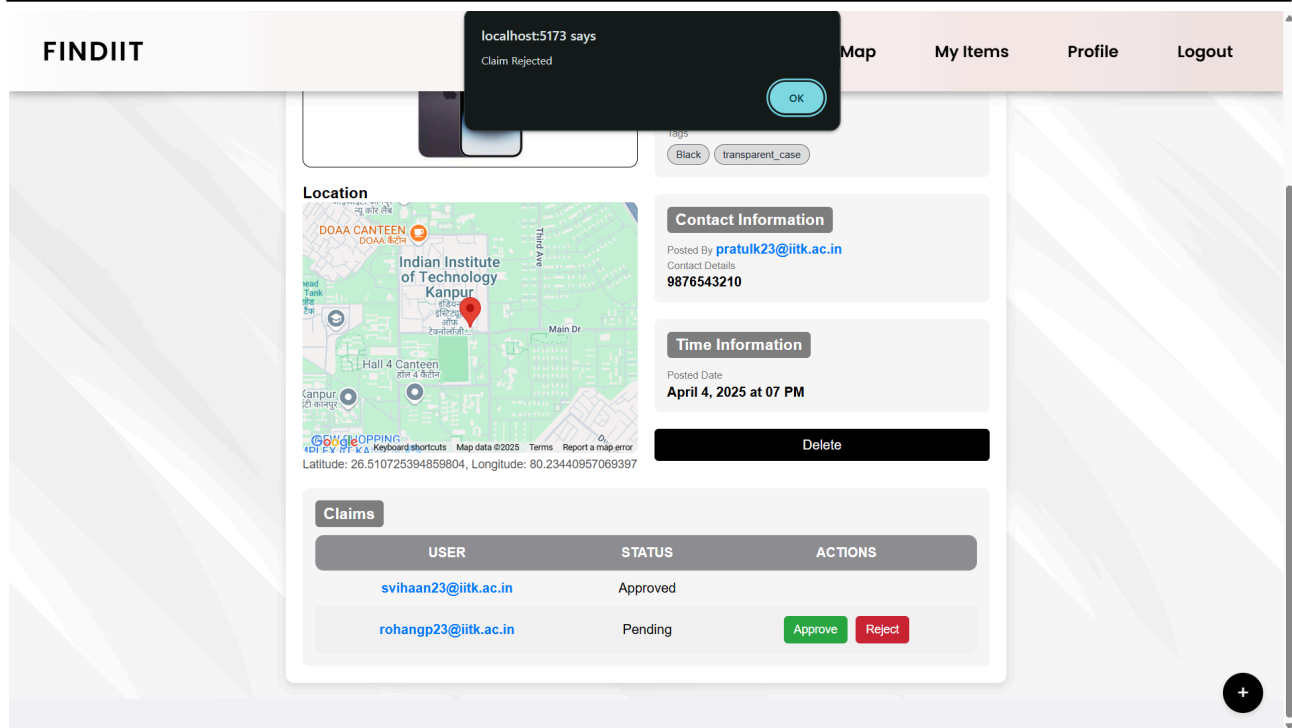


a. Click on Accept button



An alert appears saying "Claim Approved".

b. Or click on Reject button



An alert appears saying “Claim Rejected”.

Expected Output: An alert appears depending on user prompt.

Actual Output: ☒ Passed – Alert generated successfully.

Conclusion

The Lost and Found Items website was tested to ensure:

- Users can report and search for lost and found items seamlessly.
- The claiming process works with proper authentication and verification.
- Deletion of items works without bugs.

All test cases were executed successfully, with no major issues observed. Future improvements may include notification alerts on successful matches and implementation of relevance-based matching of items.

Admin functionalities test Module Report

Test owner : Rohan Potukuchi

Test Date : 05/04/2025

Overview

This test module evaluates the functionalities that can be exercised only by admins of the FindIIT website. The system allows admins to:

- Ban any user (including other admins)
- Unban any previously banned user
- Assign an existing user Admin role.
- Revoke admin role of any admin.
- Delete any item reported by any user.

The tests validate that:

- The banned user cannot login to the website.
- The user can use the website as usual after being unbanned.
- Users being assigned the admin role can exercise admin functionalities.
- After being revoked from the admin role, users cannot exercise admin functionalities.
- Items are successfully deleted from the database after the admin deletes them.

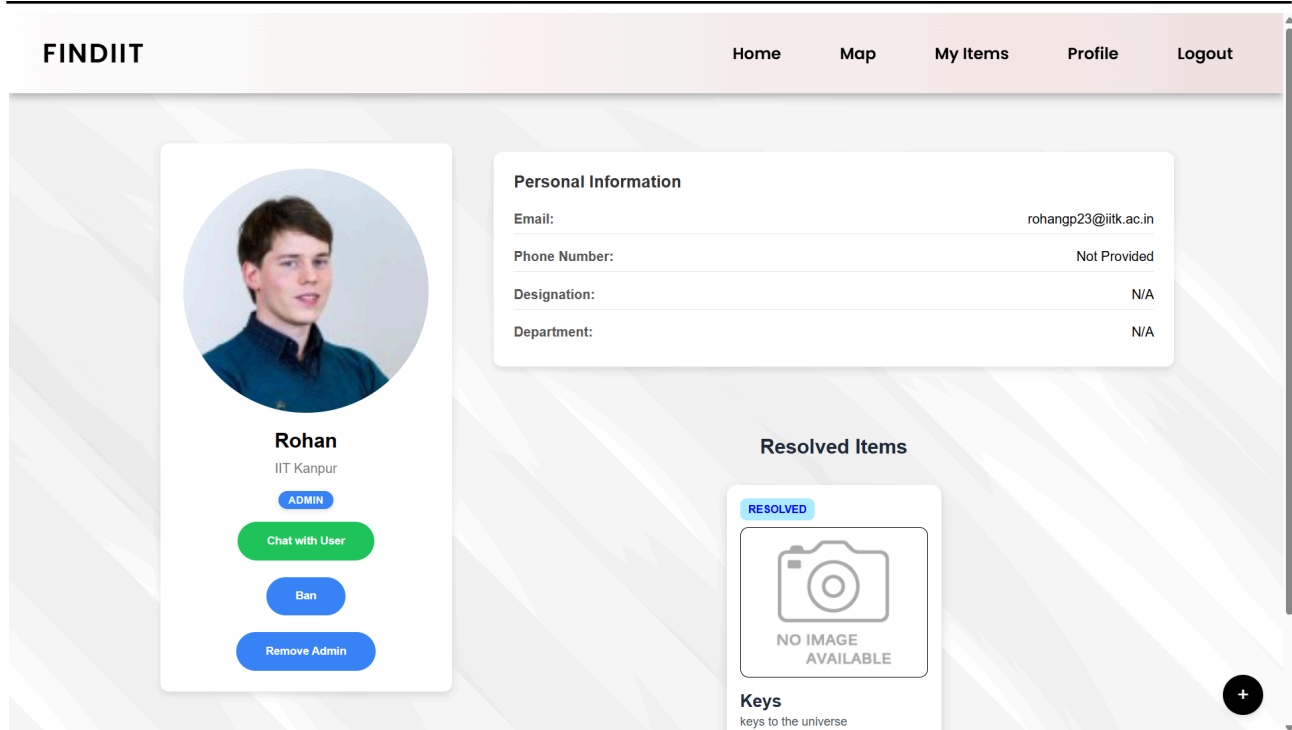
Test Cases

Test #1: Ban any existing user

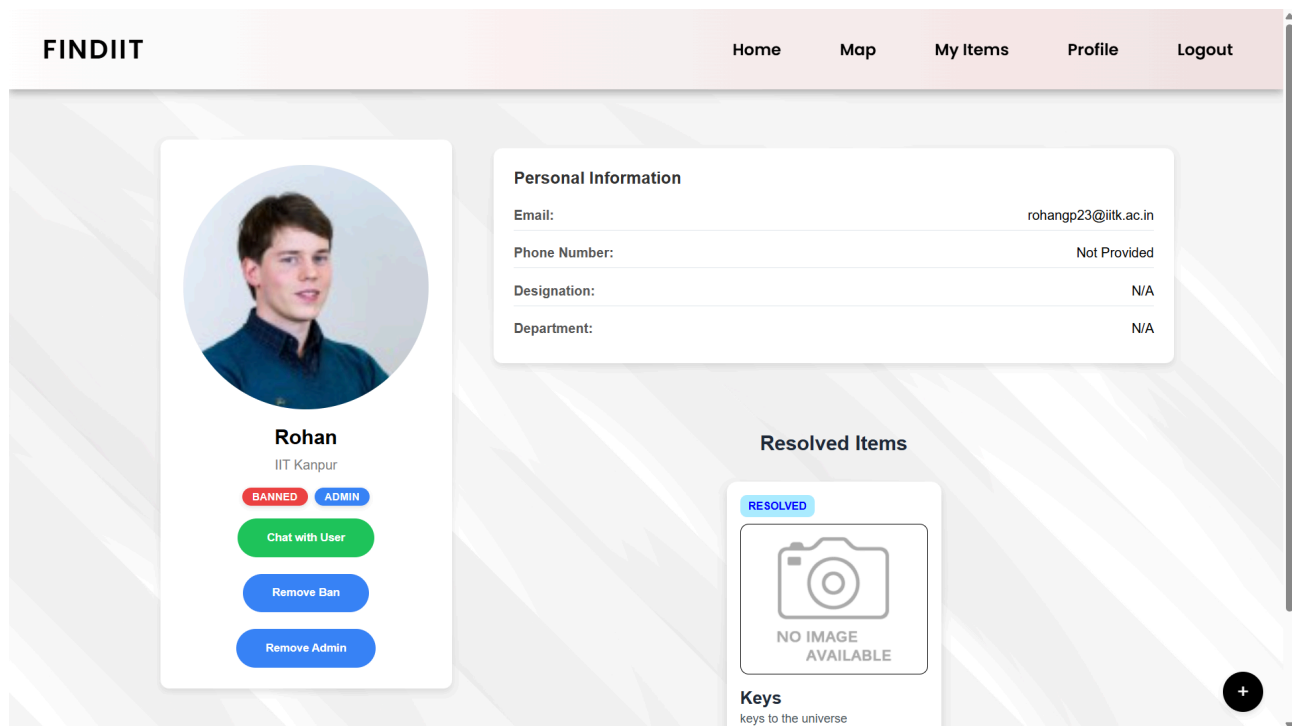
Details: Verify that an Admin can successfully ban an existing user.

Test Steps:

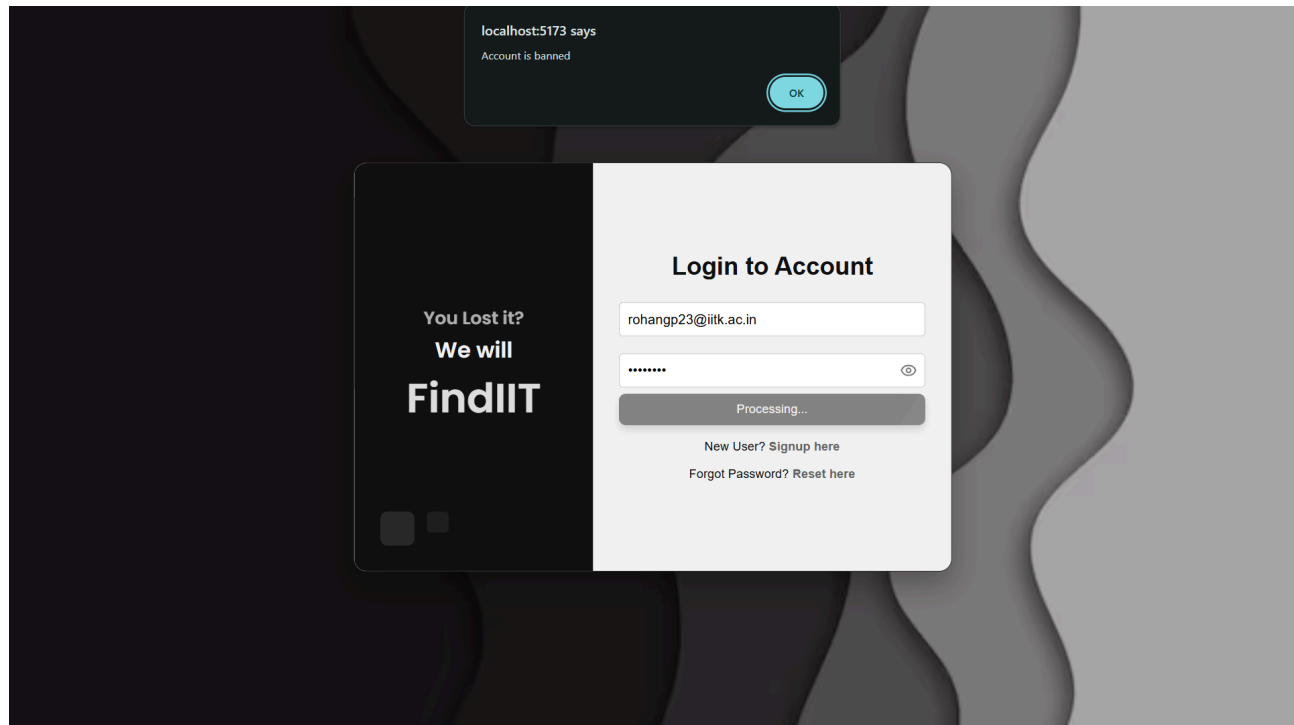
1. Go to the profile page of the user who has to be banned.



2. Click the “Ban” button.



Expected Output: The banned user should not be able to log into his/her account



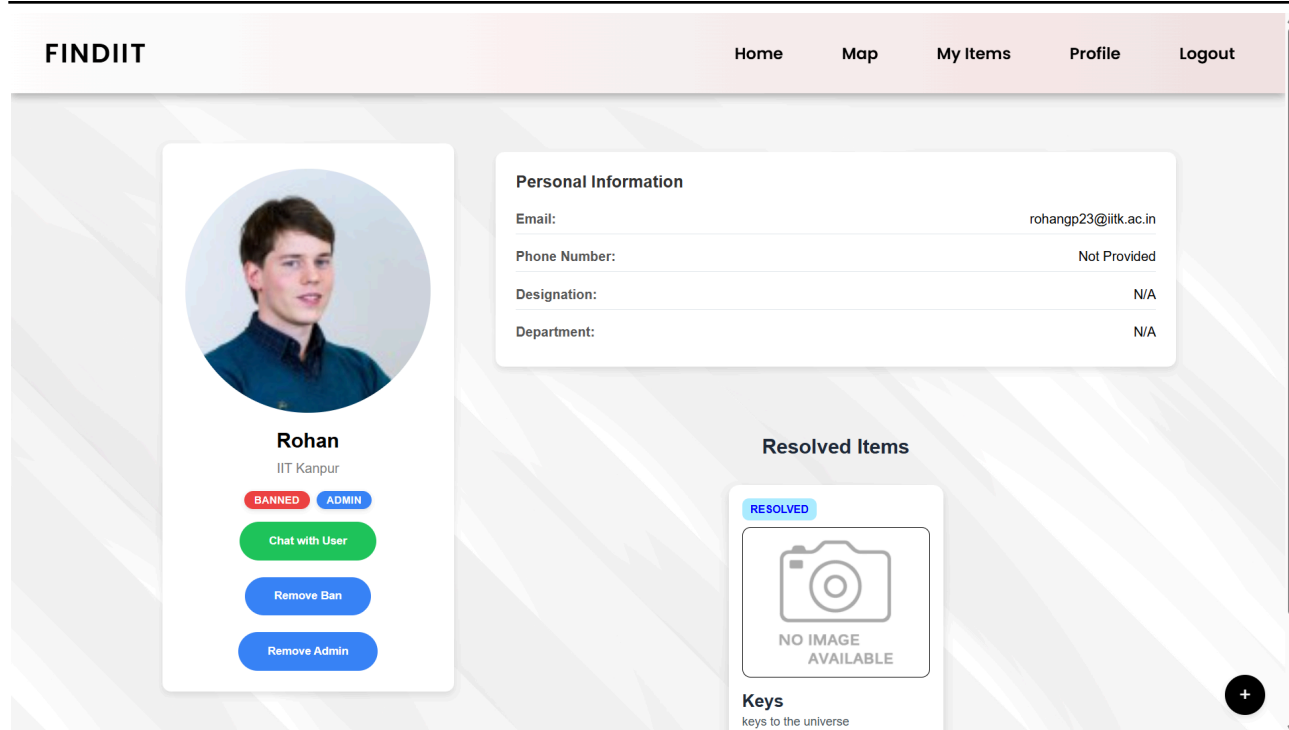
Actual Output:  Passed – The banned user receives an alert “Account is banned” and is not allowed to login.

Test #2: Unban a banned user

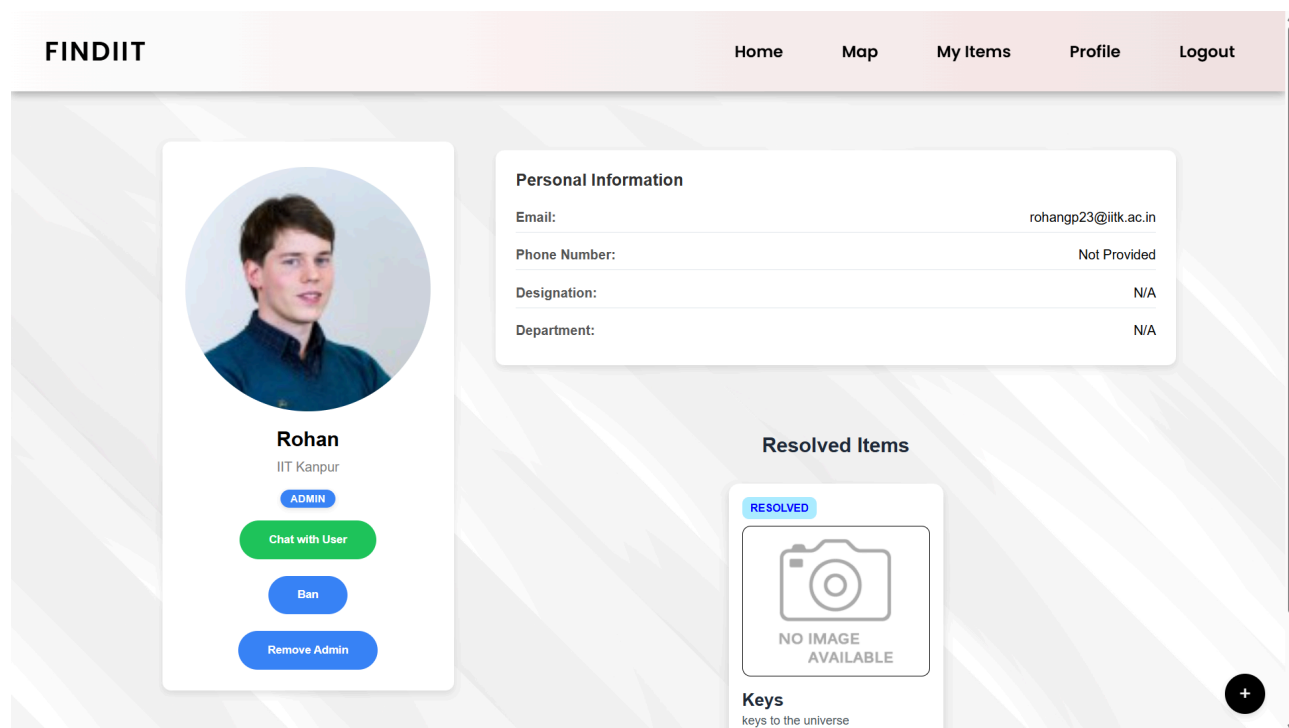
Details: Verify that an Admin can successfully unban a banned user.

Test Steps:

1. Go to the profile page of the user who has to be unbanned.



2. Click the “Remove Ban” button.



Expected Output: The banned user should now be able to log into his/her account

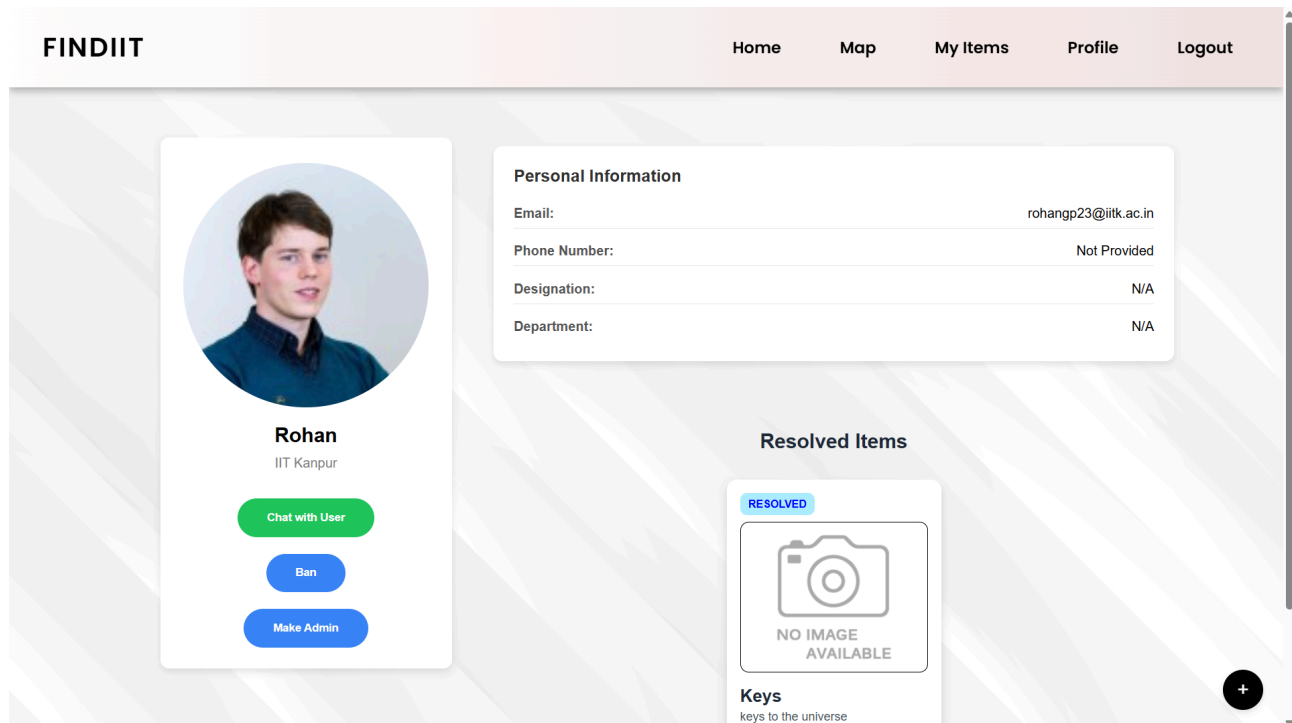
Actual Output: ☒ Passed – The banned user is now allowed to login as usual.

Test #3: Assign admin role of any existing user

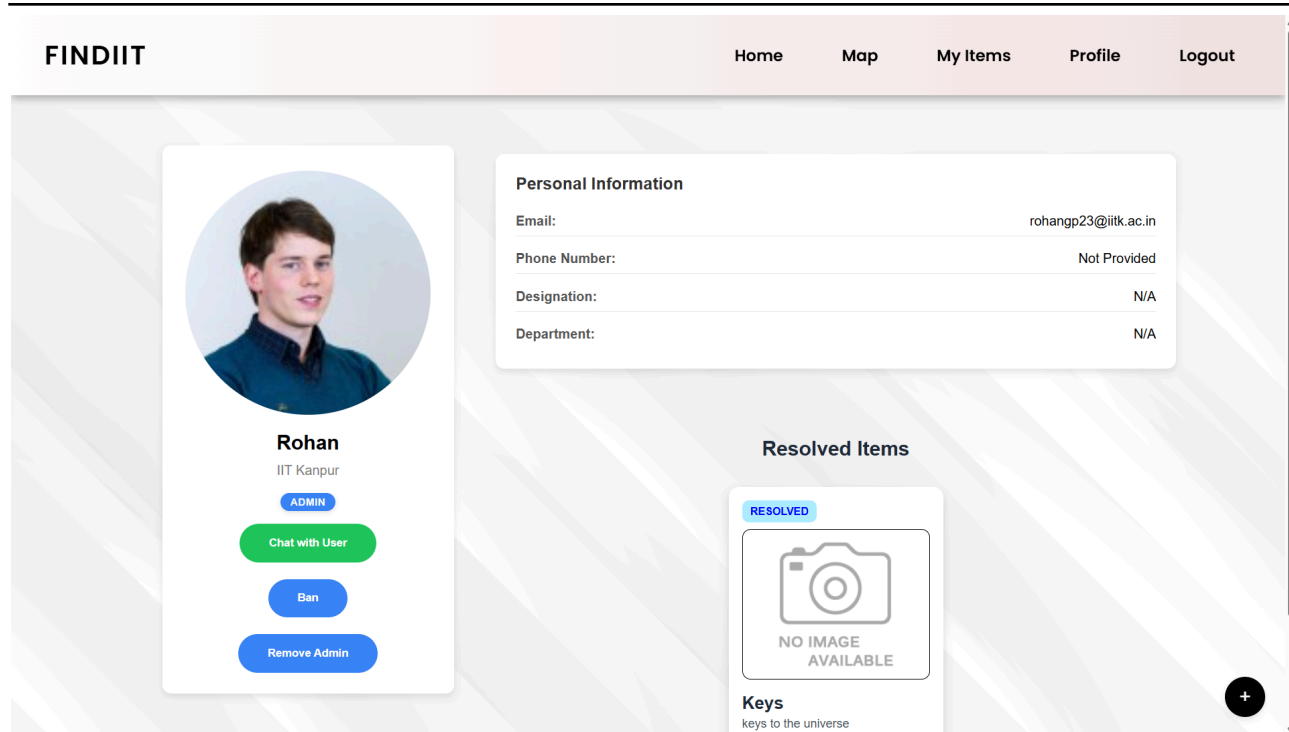
Details: Verify that an Admin can successfully revoke the admin role of any admin.

Test Steps:

1. Go to the profile page of the user who has to be assigned the admin role.



2. Click the "Make Admin" button.



Expected Output: The user should now be able to exercise admin functionalities.

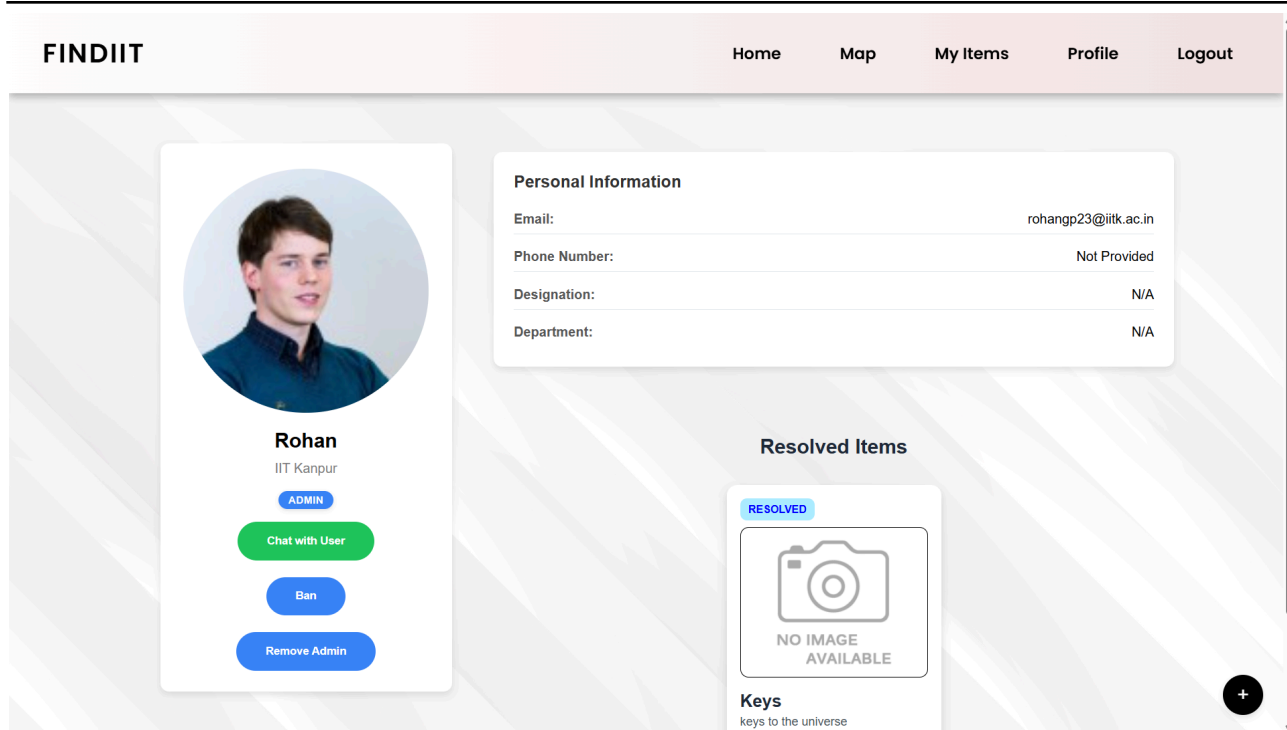
Actual Output: ☒ Passed – The user can exercise admin functionalities.

Test #4: Revoke admin role of any admin

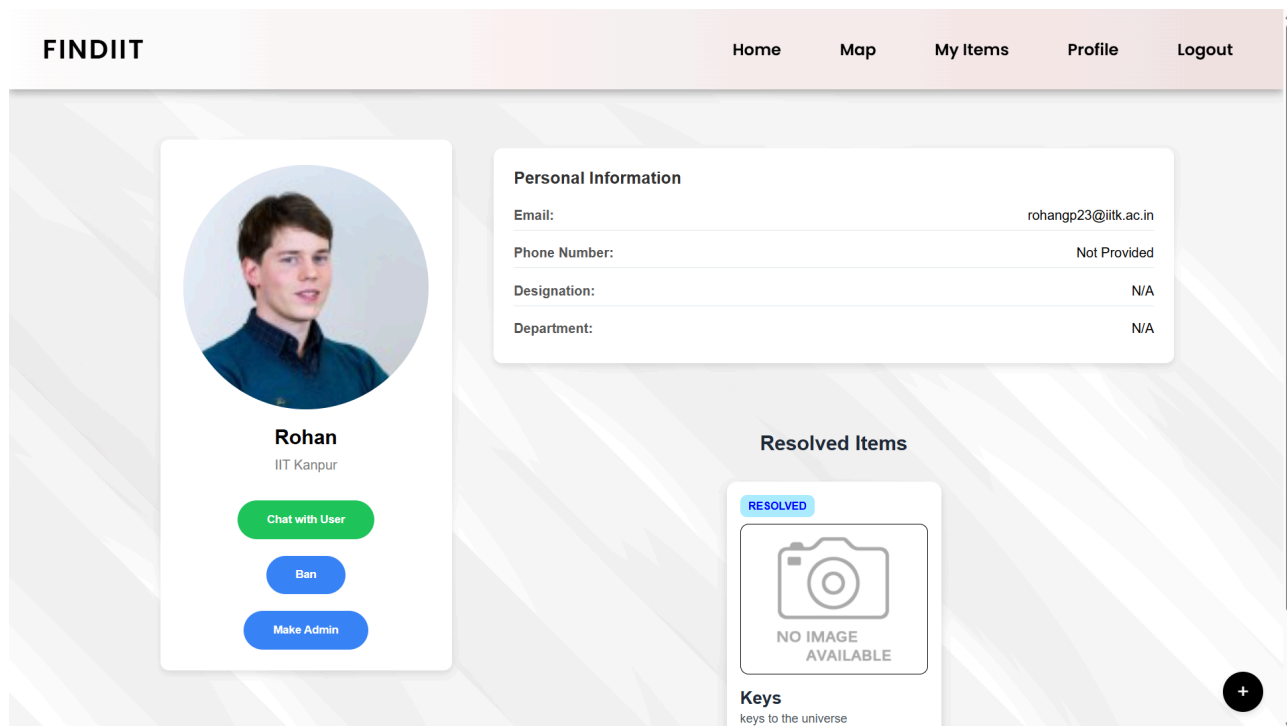
Details: Verify that an Admin can successfully revoke the admin role of any admin.

Test Steps:

1. Go to the profile page of the user whose admin role has to be revoked.



2. Click the “Remove Admin” button.



Expected Output: The user should not be able to exercise admin functionalities.

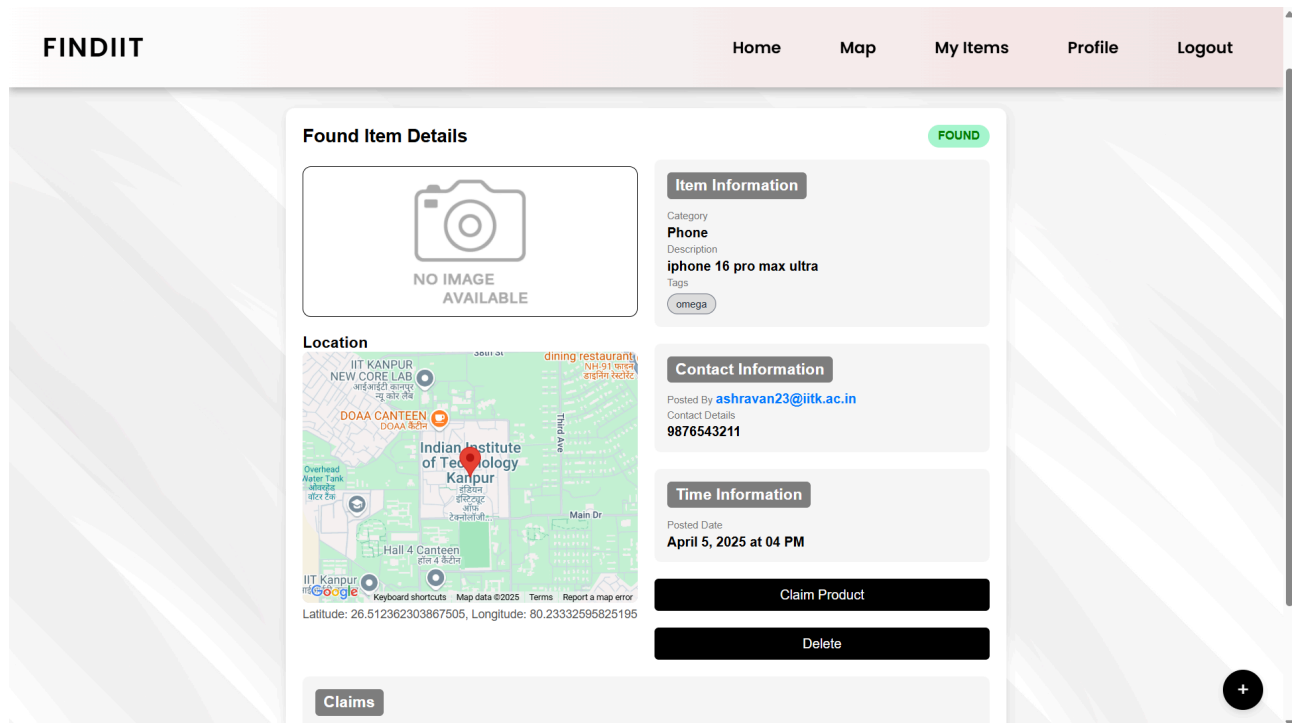
Actual Output: ☒ Passed – The user now performs actions like a normal user only.

Test #2: Deleting an item reported by some other user

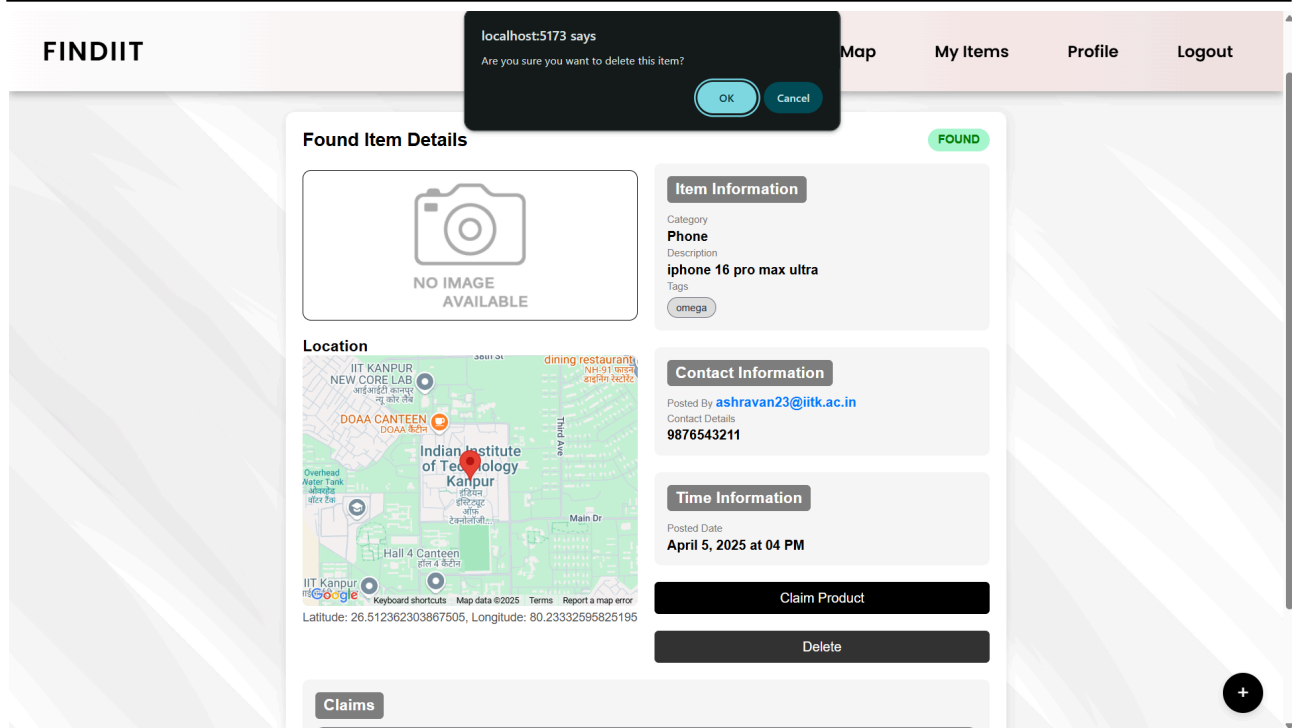
Details: Validate that admin can delete any existing reported item query.

Test Steps:

1. Go to the page of time which has to be deleted.



2. Click the "Delete" button.



3. The System raises an alert asking for confirmation. Click OK.

Expected Output: The system should delete this item from the database.

Actual Output: ☒ Passed – The deleted query was successfully removed from database.

Conclusion

The FindIIT website was tested to ensure that admins can:

- Ban any user (including other admins)
- Unban any previously banned user
- Assign an existing user Admin role.
- Revoke admin role of any admin.
- Delete any item reported by any user.

All test cases were executed successfully, with no major issues observed.

4 System Testing

Functional Requirements

1. Sign-In Functionality

Requirement:

The user should be able to sign in through their email ID (or phone number) and password if they are already a member.

Testing Approach:

This sign-in functionality underwent thorough testing to ensure its reliability and effectiveness. We created test cases covering various scenarios, including valid and invalid inputs.

These test cases were executed by:

- Manually navigating to the **Sign-In** page.
- Inputting different combinations of email IDs and passwords.
- Observing the system's responses.

Test Scenarios:

- **Valid Credentials:** Entered correct email and password → Successfully redirected to the **Dashboard** page.
- **Invalid Inputs:**
 - Entered an incorrect email format → Displayed appropriate error message.
 - Entered a non-registered email ID → Displayed "**Wrong credentials**" error.
 - Entered an incorrect password → Displayed "**Wrong credentials**" error.

Throughout the testing process, no bugs or issues were encountered, confirming that the **sign-in functionality** operates as intended and meets the specified requirements.

Test Details:

- **Test Owner:** Aarush Ghate
 - **Test Date:** 1/04/2025
 - **Test Results:** The **Sign-In** page worked as expected. We tested it on our deployed website, and **no bugs** were reported.
 - **Additional Comments:** NA
-

2. Sign-Up Functionality

Requirement:

A new user should be able to sign up by creating their account by filling in the required details, which include:

- Email
- Password
- OTP from email
- Setting a valid password

Testing Approach:

The **Sign-Up** functionality enables new users to create accounts by providing the required details. To ensure the effectiveness of this feature, we devised a series of test cases covering various scenarios.

These tests were executed by:

- Manually navigating to the **Sign-Up** page.
- Inputting valid and invalid data into respective fields.
- Observing the system's responses.

Test Scenarios:

- Successfully signing up with **all required details and a valid password** → Account was created successfully.
- Entering **incomplete or incorrect information** → Displayed validation errors.
- Using **duplicate email IDs** → Displayed error message.
- Entering an **invalid password (less than required minlength or without the required special characters)** → Displayed validation error.

No bugs were encountered for the same.

Test Details:

- **Test Owner:** Pratul Koolwal
 - **Test Date:** 03/04/2025
 - **Test Results:** The **Sign-Up** page worked as expected.
 - **Additional Comments:** NA
-

3. Forgot Password Functionality

Requirement:

There should be an option of '**Forgot Password?**' in case the user forgets their password. Authentication should be carried out through **OTP verification** on the registered email ID.

Testing Approach:

The **Forgot Password** functionality allows users to reset their passwords securely. Authentication for password reset is done through **OTP verification**, which is sent to the registered email ID provided during signup.

These tests were executed by:

- Manually navigating to the **Forgot Password** page.
- Inputting the registered email ID.
- Initiating the password reset process.

Test Scenarios:

1. Valid Email ID:

- Entered a valid registered email → OTP was successfully generated and sent.
- Entered the correct OTP → Allowed to reset the password.
- Logged in with the new password → Successfully logged in.

2. Invalid Scenarios:

- Entered an **invalid or non-registered email ID** → Displayed error message.
- Entered an **incorrect OTP** → Displayed error message.

Throughout the testing process, no issues or bugs were encountered, confirming that the **Forgot Password** functionality operates as expected and provides users with a **secure and reliable password recovery option**.

Test Details:

- **Test Owner:** Pratul Koolwal
- **Test Date:** 03/04/2025
- **Test Results:** The **Forgot Password** page worked as expected. We tested it on our deployed website, and **no bugs** were reported.
- **Additional Comments:** NA

4. View Profile Functionality

Requirement:

The user should be able to **view their profile**, which includes details such as **photo, name, roll number, and contact details**.

Testing Approach:

The **View Profile** functionality underwent thorough testing to ensure its proper operation. Test cases covered various scenarios, including valid and invalid profile information.

These test cases were executed by:

- Navigating to the **Profile Page**.
- Checking if all **user details (Photo, Name, Roll Number, Contact Details, etc.)** were displayed correctly.
- Verifying that the profile loads consistently without any missing or corrupted information.

Throughout the testing process, **no issues or bugs** were encountered, confirming that the **View Profile** feature functions as expected.

Test Details:

- **Test Owner:** Rohan Potukuchi
 - **Test Date:** 01/04/2025
 - **Test Results:** The **View Profile** functionality worked as expected. **No bugs** were reported.
 - **Additional Comments:** NA
-

5. Edit/Update Profile

Requirement:

Users should be able to **edit and update** their profile details, including modifying their **photo, and contact details**.

Testing Approach:

The **Edit Profile** feature was tested by:

- Modifying different fields like **photo, and contact details**.
- Ensuring that changes were **saved successfully** and reflected correctly.

- Testing **invalid inputs**, such as incorrect phone numbers, to check error handling.

Bugs Found:

- Phone number of arbitrary length was being accepted as valid
- Default designation and departments did not reflect in final if no changes were made (showed N/A)

Test Details:

- **Test Owner:** Aarush Ghatе
 - **Test Date:** 01/04/2025
 - **Test Results:** The **Edit Profile** functionality worked as expected with a minor bug as mentioned above , that will be patched in the next bug fix update.
 - **Additional Comments:** NA
-

6. Chat Functionality

Requirement:

A chat system should allow users to **view past conversations** and **start new chats**.

Testing Approach:

The **Chat Page** was tested using multiple scenarios:

- Sending and receiving messages between users.
- Checking if **previous chat history** was displayed correctly.
- Attempting to start a **new conversation** with another user.

Testing confirmed that **all chat messages were sent, received, and stored properly**.

Bugs Found:

- Starting a chat with an invalid user redirects immediately to the 404 page before showing a user not found error

Test Details:

- **Test Owner:** Rohan Potukuchi
- **Test Date:** 02/04/2025
- **Test Results:** The **Chat Functionality** worked as expected except for the minor bug mentioned , which will be fixed in the next bug fix update.
- **Additional Comments:** NA

7. Report Items Functionality

Requirement:

Users should be able to **report lost or found items** by providing details such as **item description, time, location, and an optional image**.

Testing Approach:

The **Report Items** feature was tested using the following scenarios:

- Creating a **new report** by entering valid item details.
- Leaving fields empty to check **validation prompts**.
- Uploading images and verifying proper display.
- Adding **tags to reports** for improved searchability.

The system correctly stored and displayed reported items without issues.

Bugs Found:

- Phone number length was not checked (since optional field).

Test Details:

- **Test Owner:** Rohan Potukuchi
 - **Test Date:** 02/04/2025
 - **Test Results:** The **Report Items** functionality worked as expected except for the minor bug mentioned , which will be patched in an upcoming update.
 - **Additional Comments:** NA
-

8. My Items Functionality

Requirement:

Users should be able to view a **list of items they have reported as lost or found**.

Testing Approach:

The **My Items** page was tested by:

- Reporting multiple items and checking if they appeared in the list.
- Ensuring that items were **sorted correctly**.
- Checking that clicking on an item opened its **detailed view**.
- Deleting lost/found queries and checking if they reflected on all relevant pages.

The feature performed well, with all reported items **accurately listed** and displayed.

Test Details:

- **Test Owner:** Rohan Potukuchi
 - **Test Date:** 02/04/2025
 - **Test Results:** The **My Items** functionality worked as expected. **No bugs** were reported.
 - **Additional Comments:** NA
-

9. Map Functionality

Requirement:

Users should be able to view a **map of the campus** where lost items can be seen at their reported locations.

Testing Approach:

The **Map Feature** was tested by:

- Checking if the **map is loaded properly**.
- Clicking on **locations** to see lost/found item details.
- Verifying **marker placement accuracy**.

The map **displayed all items correctly**, and no technical issues were found.

Test Details:

- **Test Owner:** Pratul Koolwal
 - **Test Date:** 01/04/2025
 - **Test Results:** The **Map Feature** worked as expected. **No bugs** were reported.
 - **Additional Comments:** NA
-

10. Search Bar Functionality

Requirement:

Users should be able to **search for lost or found items** in the feed.

Testing Approach:

The **Search Bar** functionality was tested by:

- Searching using **item names and tags**.
- Searching with **partial keywords** to check fuzzy matching.
- Verifying that relevant items **appeared in the results**.

The search function **accurately retrieved relevant items** without issues.

Test Details:

- **Test Owner:** Aarush Ghate
 - **Test Date:** 01/04/2025
 - **Test Results:** The **Search Bar** worked as expected. **No bugs** were reported.
 - **Additional Comments:** NA
-

11. Item Feed Functionality

Requirement:

A feed should display **lost and found items** along with their **photo, tags, and short descriptions**.

Testing Approach:

The **Item Feed** was tested by:

- Checking if new **lost/found items** **appeared** correctly.

- Clicking on items to view their **detailed description and comments section**.

The feed loaded and displayed all items properly without lag or missing information.

Test Details:

- **Test Owner:** Aarush Ghatе
 - **Test Date:** 01/04/2025
 - **Test Results:** The **Item Feed** functionality worked as expected. **No bugs** were reported.
 - **Additional Comments:** NA
-

5 Conclusion

Conclusion: Testing Effectiveness and Future Improvements

1. Effectiveness and Exhaustiveness of Testing

The testing process covered a **wide range of functional aspects** of the system, ensuring that core features operated as expected. Key areas of focus included **profile management, chat functionality, lost-and-found item reporting, map integration, and search capabilities**. Each feature was evaluated using multiple test cases, addressing various **edge cases and user interactions**.

The testing approach was **effective** in identifying potential issues and ensuring that all functionalities worked smoothly. **Both positive and negative test scenarios were considered**, including valid and invalid inputs, stress testing, and boundary testing. As a result, we can conclude that the system is **robust and performs well under normal user conditions**.

Despite the effectiveness of the testing, **100% exhaustive testing is impractical** due to the vast number of possible input combinations and edge cases. However, the test cases covered all the **critical functionalities** that impact user experience.

2. Components That Have Not Been Tested Adequately

While most functionalities were tested thoroughly, **some areas could benefit from further testing**, including:

- **Scalability Testing:** The system was tested with a moderate number of users and reported items, but it has not been tested under **high traffic conditions**. Future tests should include **load testing** to evaluate system performance when handling thousands of users simultaneously.
- **Security Testing:** While basic security measures were tested (such as input validation and user authentication), **penetration testing and vulnerability assessments** were not conducted. These tests are crucial to ensure data privacy and prevent unauthorized access.
- **Error Handling and Recovery:** The system's ability to **recover from failures** (e.g., server crashes, network issues, or database downtimes) was not extensively tested. Future testing should simulate **server failures, network outages, and database crashes** to assess system resilience.
- **Cross-Platform Testing:** The system was primarily tested on a **single platform and browser**. More comprehensive testing should include **various devices**,

screen sizes, and browsers to ensure compatibility across different user environments.

3. Difficulties Faced During Testing

Several challenges were encountered during the testing process:

- **UI Rendering Delays:** Some testers reported minor delays when **loading the map feature**. This was linked to high-resolution images being loaded dynamically.
- **Testing Environment Constraints:** The system was tested in a **controlled environment**, meaning that real-world conditions (such as network latency, device compatibility, and multi-user interactions) were difficult to simulate accurately.

4. Future considerations for Improving the Testing Process

To enhance future testing cycles, the following improvements will be considered:

- **Enhanced Automated Testing:** Implementing **enhanced automated test scripts** for regression testing can save time and reduce human errors. Tools like **Selenium, JUnit, or Jest** could be used for web testing.
- **Load and Performance Testing:** Conducting **stress tests** using tools like **Apache JMeter** can help analyze system behavior under high traffic conditions.
- **Security Audits:** A thorough **penetration test** should be performed to detect vulnerabilities, such as **SQL injection, XSS attacks, and authentication flaws**.
- **User Testing and Feedback Collection:** Real users should be engaged in testing to **identify usability issues** that automated tests may miss. A structured **beta-testing phase** can help refine UI/UX design.
- **Continuous Integration (CI/CD):** Implementing **automated deployment and testing pipelines** can ensure smooth updates without introducing new bugs.

Final Conclusion

The system underwent **rigorous functional testing** across multiple modules, ensuring that essential features worked as expected. **No major bugs** were identified, and core functionalities like profile management, item reporting, chat, and search performed reliably. However, **certain areas require further testing**, such as **scalability, security, and cross-platform compatibility**.

Moving forward, it is recommended to:

1. **Conduct performance and load testing** to evaluate system behavior under high traffic conditions.
2. **Implement security audits** to safeguard user data and prevent potential vulnerabilities.
3. **Optimize UI/UX through user testing** to enhance ease of use and accessibility.
4. **Automate testing processes** to improve efficiency and reduce manual effort.

By addressing these areas, the system can be further refined into a **fully robust and scalable platform**, ensuring a **seamless user experience**.

NOTE: The bugs that have been discovered during the testing process were addressed and fixed for the next version of our website.

Appendix A - Group Log

- We have made a WhatsApp group for effective communication amongst ourselves.
- We have created a private repository on GitHub on which we would be regularly pushing our codes to maintain efficiency and collaboration.
- We have broadly divided the team into parts, all members would be working on different aspects of frontend and backend so that all members understand the mechanism of software development and are able to implement it.

The following is the group log for the development of this Testing Document.

Date	Meet type	Final outcome of meet
29/03/2025	Offline	Discussed the various test cases based on and distributed the tests to members of the team.
30/03/2025	Offline	Discussed and resolved the doubts faced by everyone in their respective tasks.
01/04/2025	Online	All members updated each other with their work and made a basic draft of the Testing Document.
03/04/2025	Online	Discovered a few minor test cases and assigned them among the members.
04/04/2025	Offline	Final meet to review the Testing document and finalize it.